

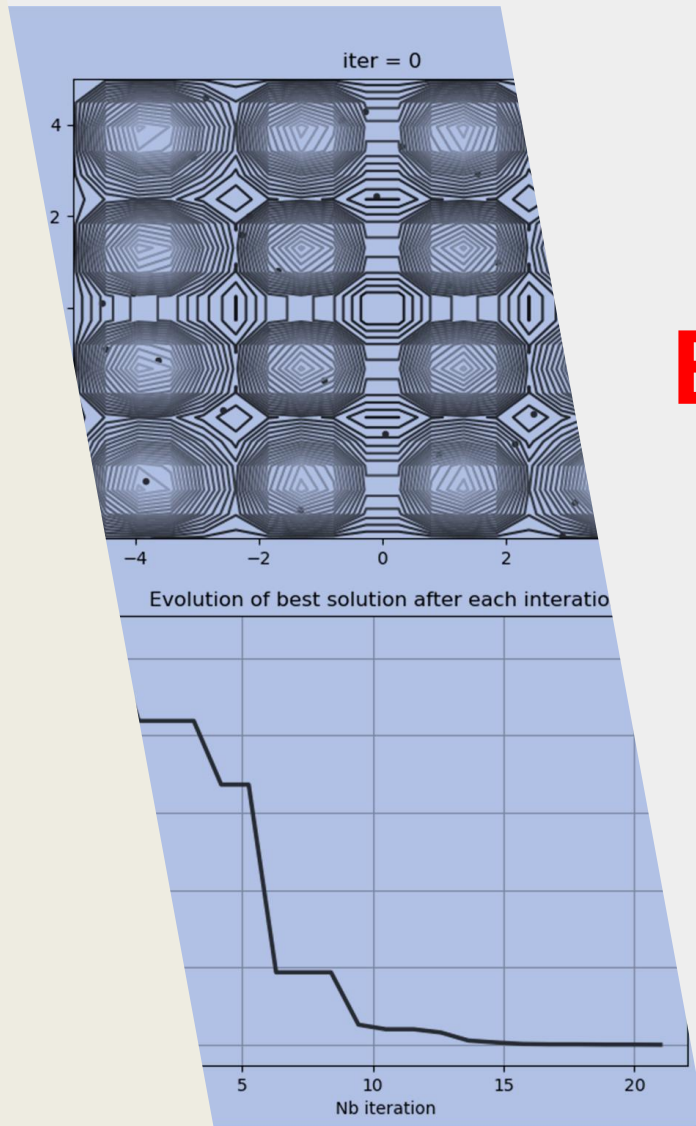
5SEE – MACHINE LEARNING FOR ELECTRONIC DESIGN

Alexandre Boyer

alexandre.boyer@insa-toulouse.fr

November 2025

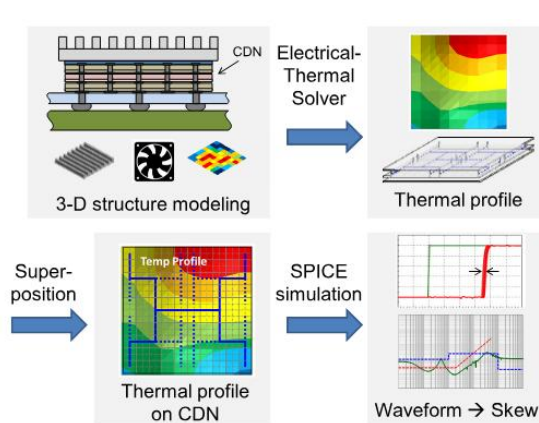
moodle.insa-toulouse.fr



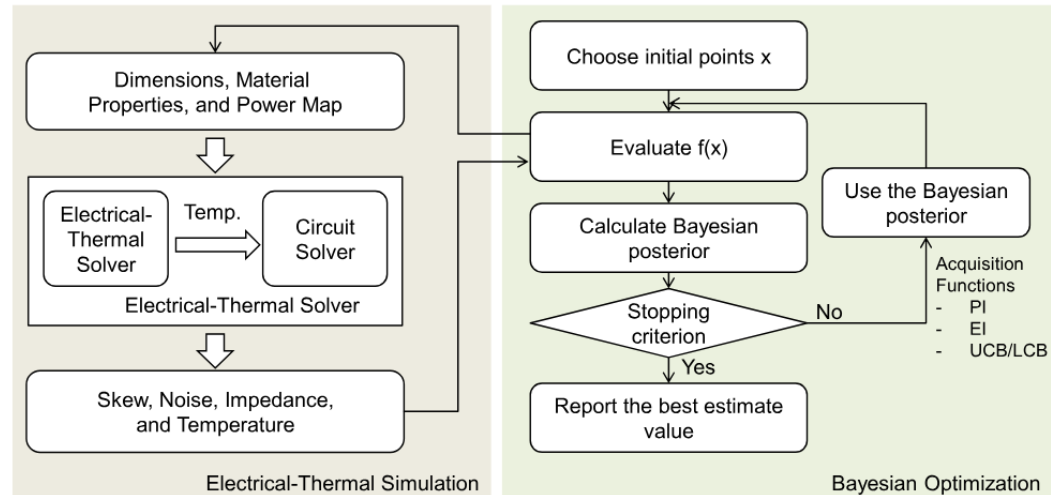
Introduction

AI/ML usage in electronic design ? Some examples

- ✓ Electro-thermal simulation to optimize clock delivery network in 3D IC module (clock skew and thermal gradient)

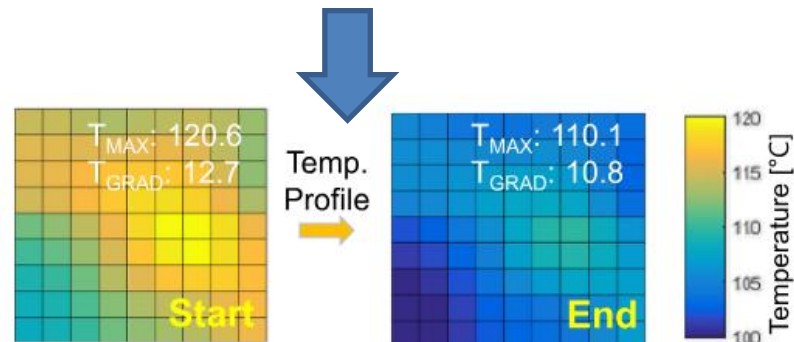


Expensive simulation



Bayesian-driven flow for electrical-thermal optimization

S. J. Park, B. Bae, J. Kim, M. Swaminathan, "Application of Machine Learning for Optimization of 3-D Integrated Circuits and Systems," IEEE Trans. On VLSI Systems, vol. 25, no. 6, June 2017.

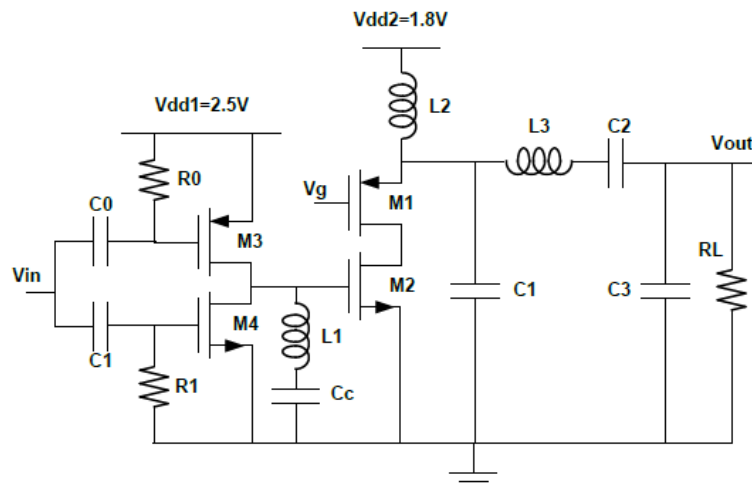


Improvement of temperature distribution on die surface

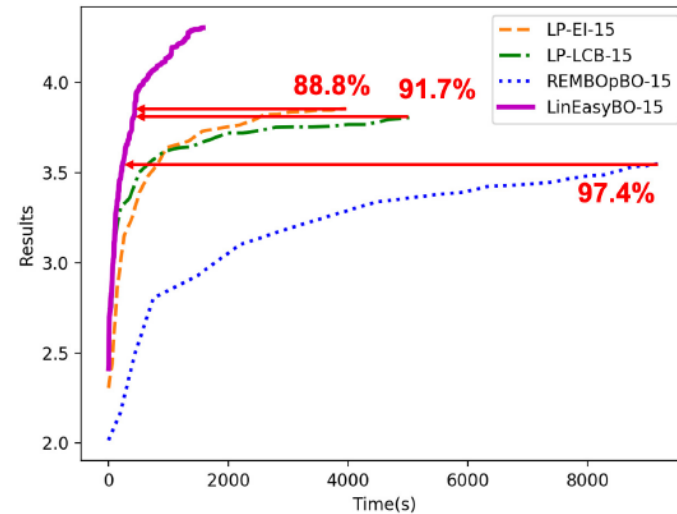
Introduction

▶ AI/ML usage in electronic design ? Some examples

- ✓ Use of machine-learning method to accelerate analog circuit sizing.
- ✓ Example: use of Bayesian optimization to optimize the output power and efficiency of a class-E RF power amplifier without human intervention.



12 design variables



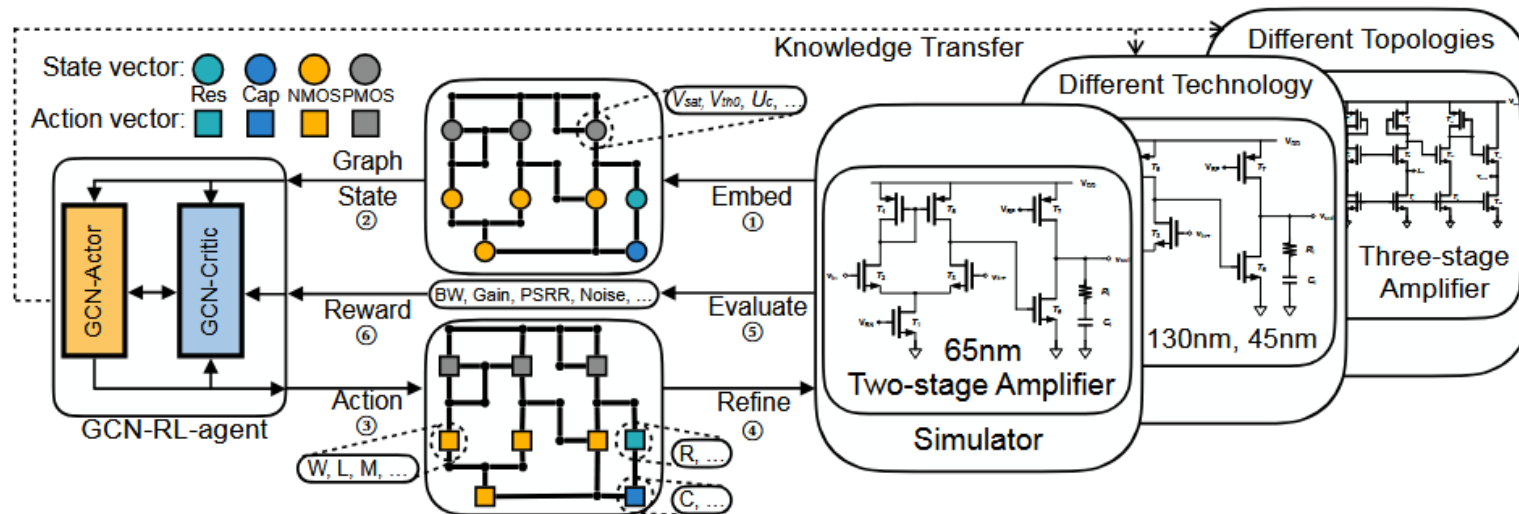
LP-EI-15	4.34	3.49	3.86	0.23	1h6m3s
LP-LCB-15	4.08	3.41	3.81	0.15	1h23m40s
REMBOpBO-15	4.10	2.92	3.55	0.22	2h33m23s
LinEasyBO-15	5.96	3.21	4.30	0.74	27m23s

S. Zhang, F. Yang, C. Yan, D. Zhou, X. Zeng, "LinEasyBO: Scalable Bayesian Optimization Approach for Analog Circuit Synthesis via One-Dimensional Subspaces," 4th Workshop on Machine Learning for CAD (MLCAD), Sept. 2022.

Introduction

► AI/ML usage in electronic design ? Some examples

- ✓ Automatic transistor sizing for analog circuits when transfer to another technological nodes ?
- ✓ Limit of manual design to ensure similar figures-of-merit (intensive and time-consuming approach due to large design space, slow simulation tool and sophisticated trade-off)



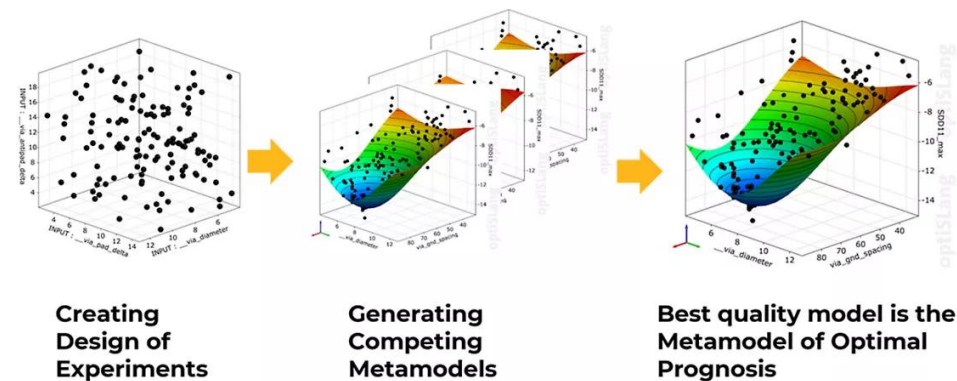
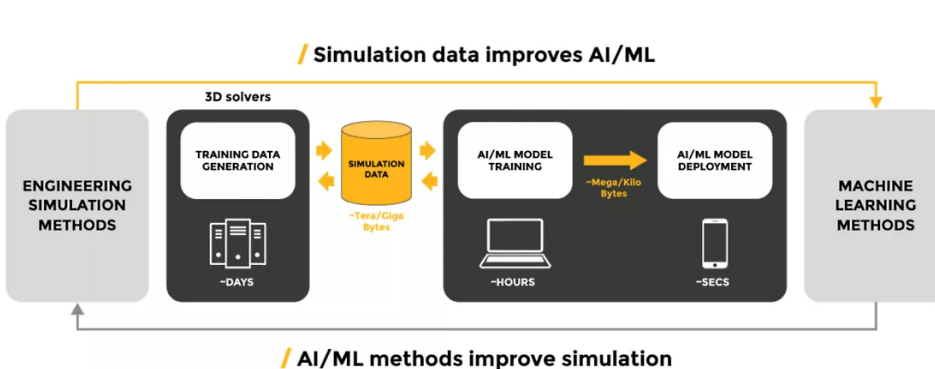
Graph Convolutional Neural Networks based Reinforcement Learning for Automatic Transistor Sizing

H. Wang, K. Wang, J. Yang, L. Shen, N. Sun, H. S. Lee, S. Han, "GCN-RL Circuit Designer: Transferable Transistor Sizing with Graph Neural Networks and Reinforcement Learning," IEEE Design Automation Conference, 2020.

Introduction

AI/ML usage in electronic design ? Some examples

- ✓ Integration of artificial intelligence (AI) and machine learning (ML) methods in commercial EDA simulation tools to enhance design exploration and optimization, accelerate simulations.



<https://www.ansys.com/fr-fr/blog/optimize-design-simulation-with-ai-ml-metamodeling>

Purpose of this course:

- ✓ What are the on-going trends?
- ✓ What are the typical AI/ML algorithms to accelerate the electronic design process?
- ✓ Experiment AI/ML algorithms for a typical electronic design optimization (coupling an electronic design tool to an AI/ML Python library)

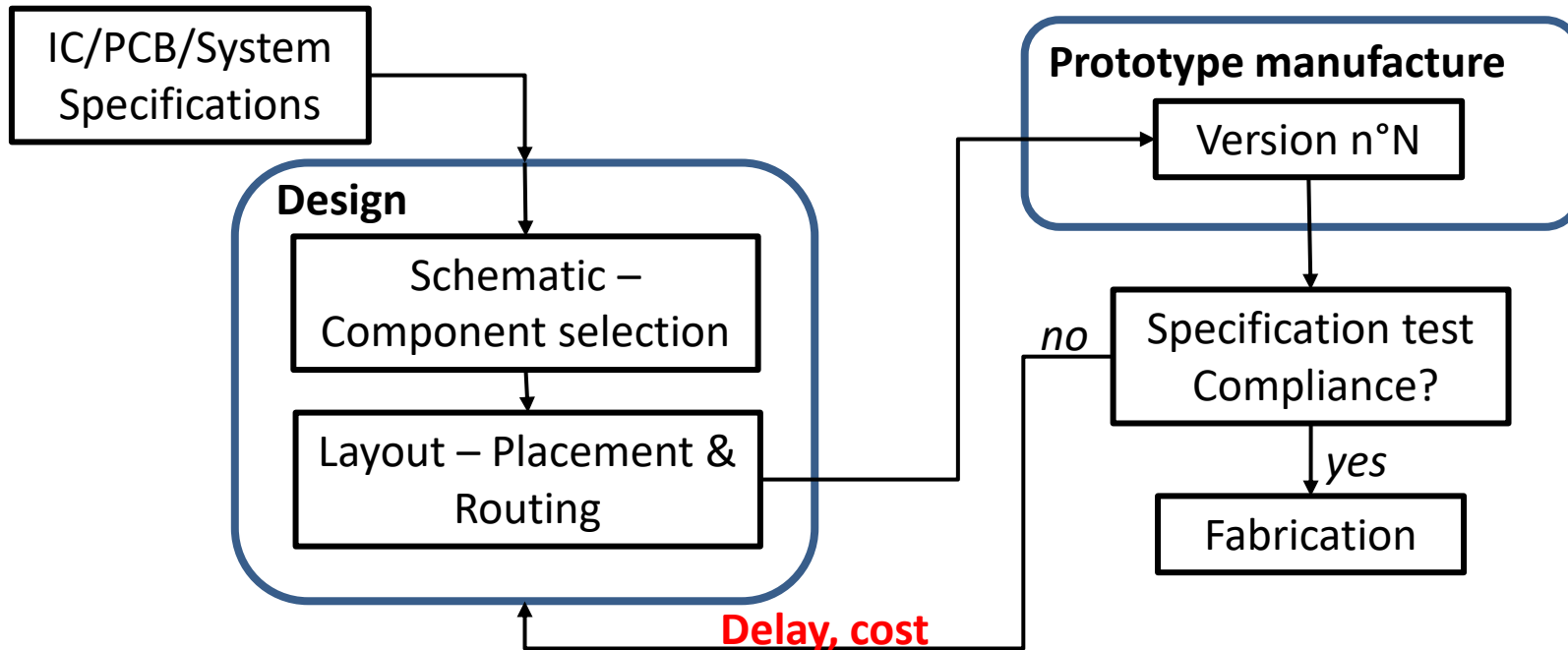
Summary

- 1. Overview of Electronic Design Automation (EDA) Tools and Usage**
- 2. Current challenges of EDA – Benefits of AI/ML for EDA**
- 3. Overview of AI/ML methods**
- 4. Metaheuristic for electronic design optimization**
- 5. Supervised learning methods for surrogate modeling**
- 6. Reinforcement learning for electronic design optimization**

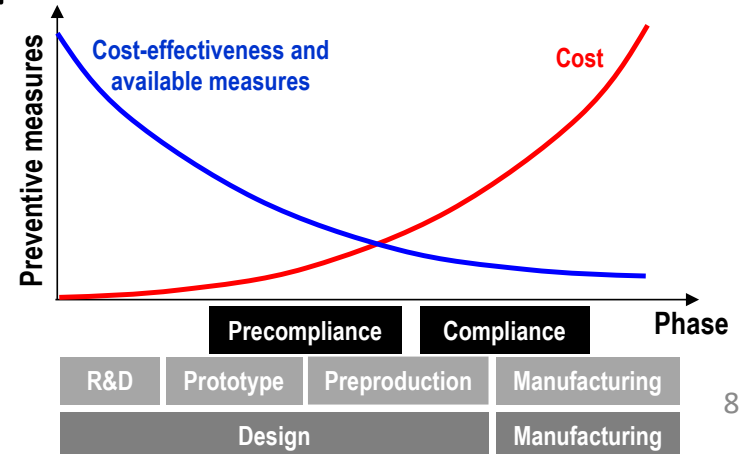
1. Overview of Electronic Design Automation (EDA) Tools and Usage

Overview of EDA Tools and Usage

► Electronic Design Flow – Trial-and-error design flow

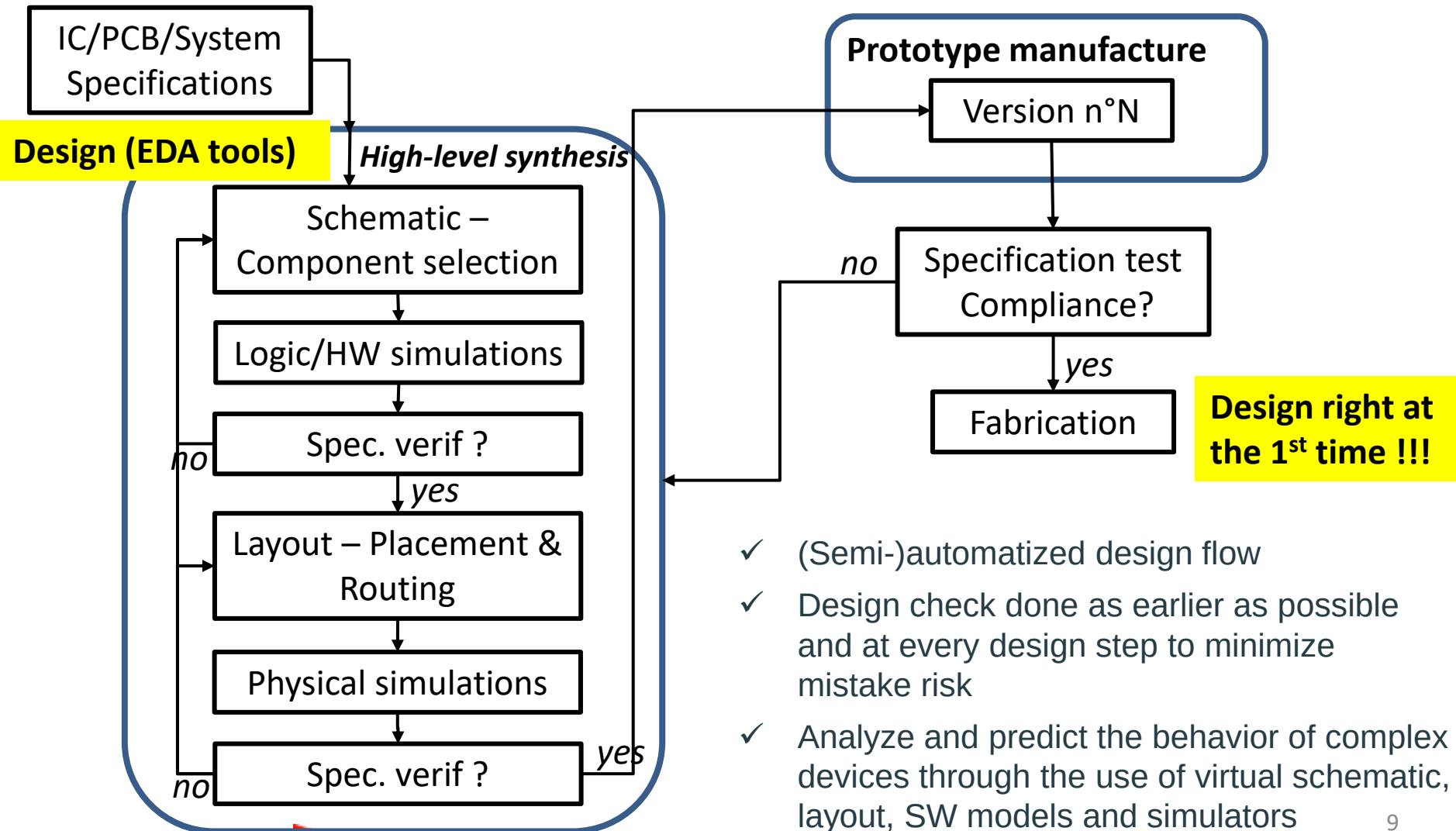


Evolution of cost of design changes during the design phase:



Overview of EDA Tools and Usage

► Electronic Design Flow – EDA-based design flow



Overview of EDA Tools and Usage

► Simulation and models for electronic design

Technological data (confid.)

Design variables

Carrier drift-diffusion equation $J = q(n\mu E + D\nabla n)$

Kirchoff's laws $\sum_k I_k = 0$ $\sum_i E_i - \sum_n Z_n I_n = 0$
⋮

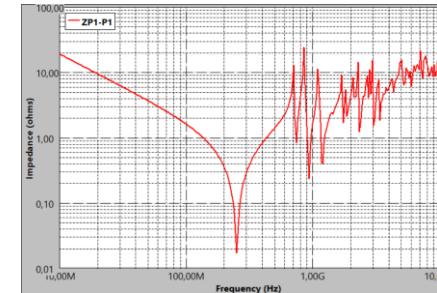
**Modelling and
simulations**

Maxwell's equations $\nabla \times E = -\frac{\partial B}{\partial t}$

Thermal diffusion equation $\rho C_P \frac{\partial T}{\partial t} + \nabla \cdot j = S$
⋮

Response
prediction

Functional ? Manufacturable ?
Robust ? Reliable ?



- ✓ Design validation/optimization of electronic devices/systems requires physical modeling and simulation when the complexity increases.
- ✓ Main issues:
 - ✓ Non-linear and multiphysical models → numerical solvers (computationally intensive)
 - ✓ Response related to proprietary technological data
 - ✓ Prediction hardly based on simplified model or generalized response.

Overview of EDA Tools and Usage

► EDA – Electronic Design Automation

EDA = software and specialized hardware tools for the design, verification, manufacturing and testing of electronic systems (ICs, PCBs, FPGAs, ...)

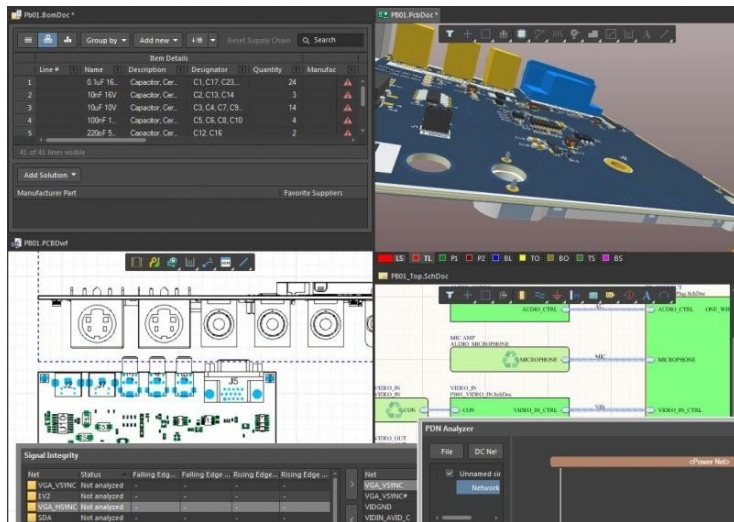
- ✓ Developed since the 1980's (initially for IC design)
- ✓ Essential when electronic devices are too complex to design and responses unpredictable without the help of computers
- ✓ EDA tools integrate several CAD interfaces and numerical simulation tools to predict, optimize, verify and analyze the performances and the manufacturability.

Overview of EDA Tools and Usage

EDA vs. CAD (Computer-Aided Design) tools

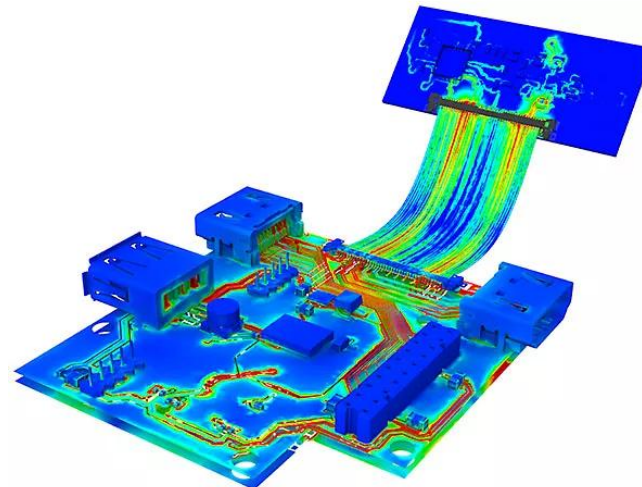
- ✓ CAD: 2D/3D modeling and visualization. Possible link with simulations
- ✓ EDA: SW dedicated for the different steps of electronic system design (schematic, simulation layout design, verification, ...)

CAD tool



(Altium Designer)

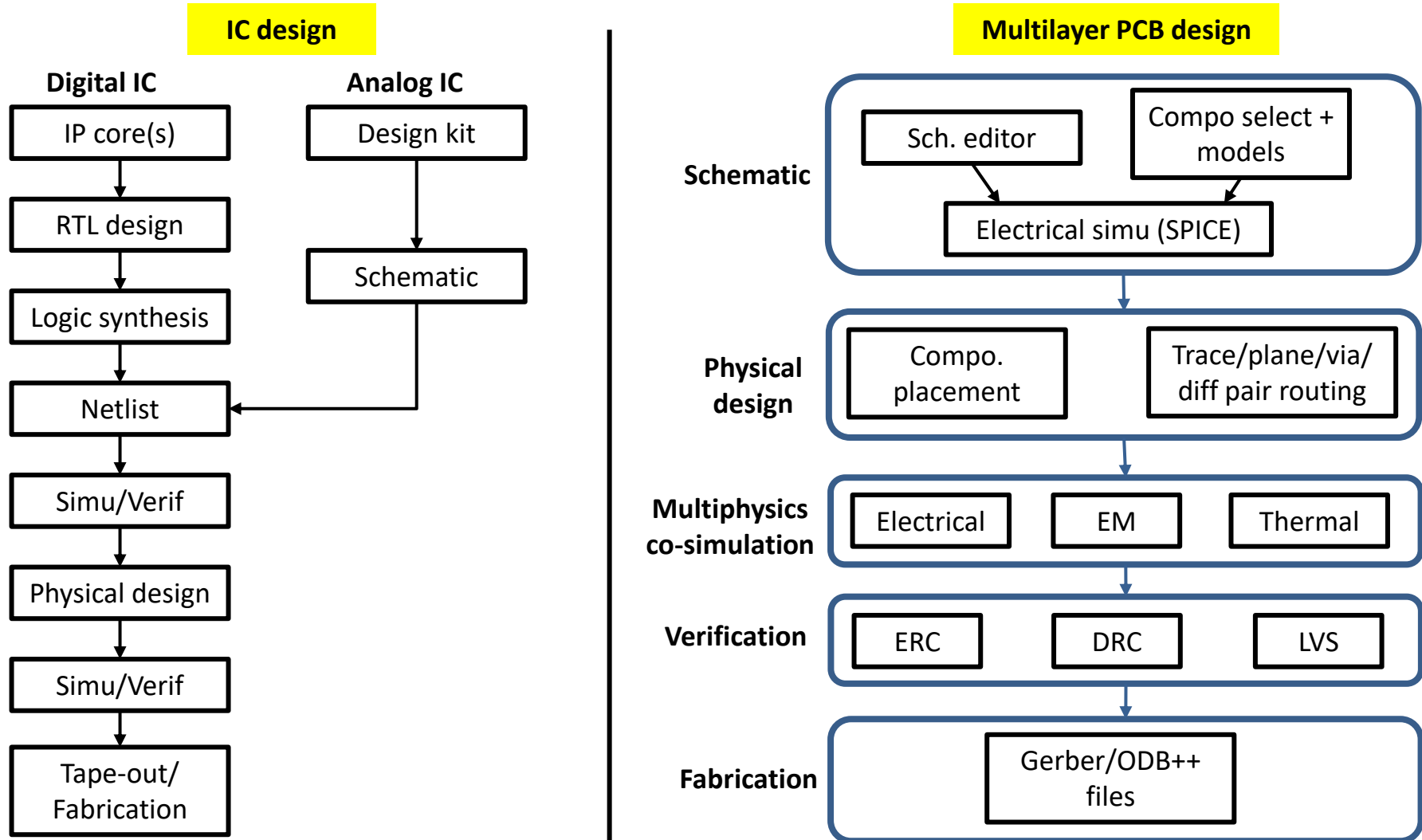
EDA tool



(Ansys HFSS: Electromagnetic simulation of PCBs)

Overview of EDA Tools and Usage

EDA in IC and PCB design – Typical flow

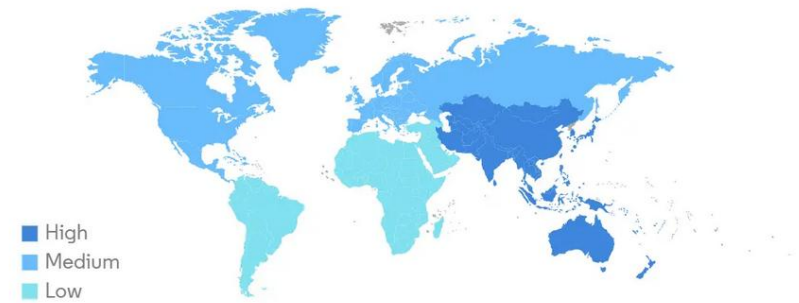


Overview of EDA Tools and Usage

EDA market

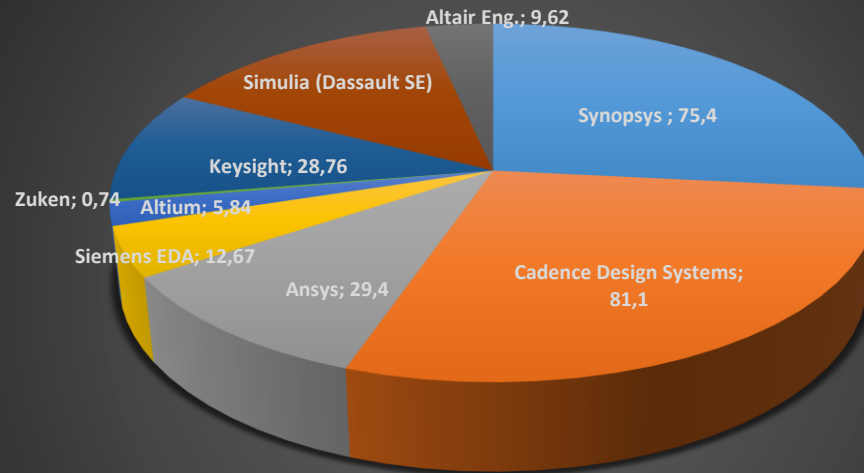
- ✓ Market of 19.22 Billions of Dollars in 2025
- ✓ Growth rate of +8.5 % (2025-2030)
- ✓ Main EDA vendors and market capitalization (Billions of \$ in 2025):

Global Electronic Design Automation Tools (EDA) Market CAGR (%), Growth Rate by Region, 2025 - 2030



Source: Mordor Intelligence

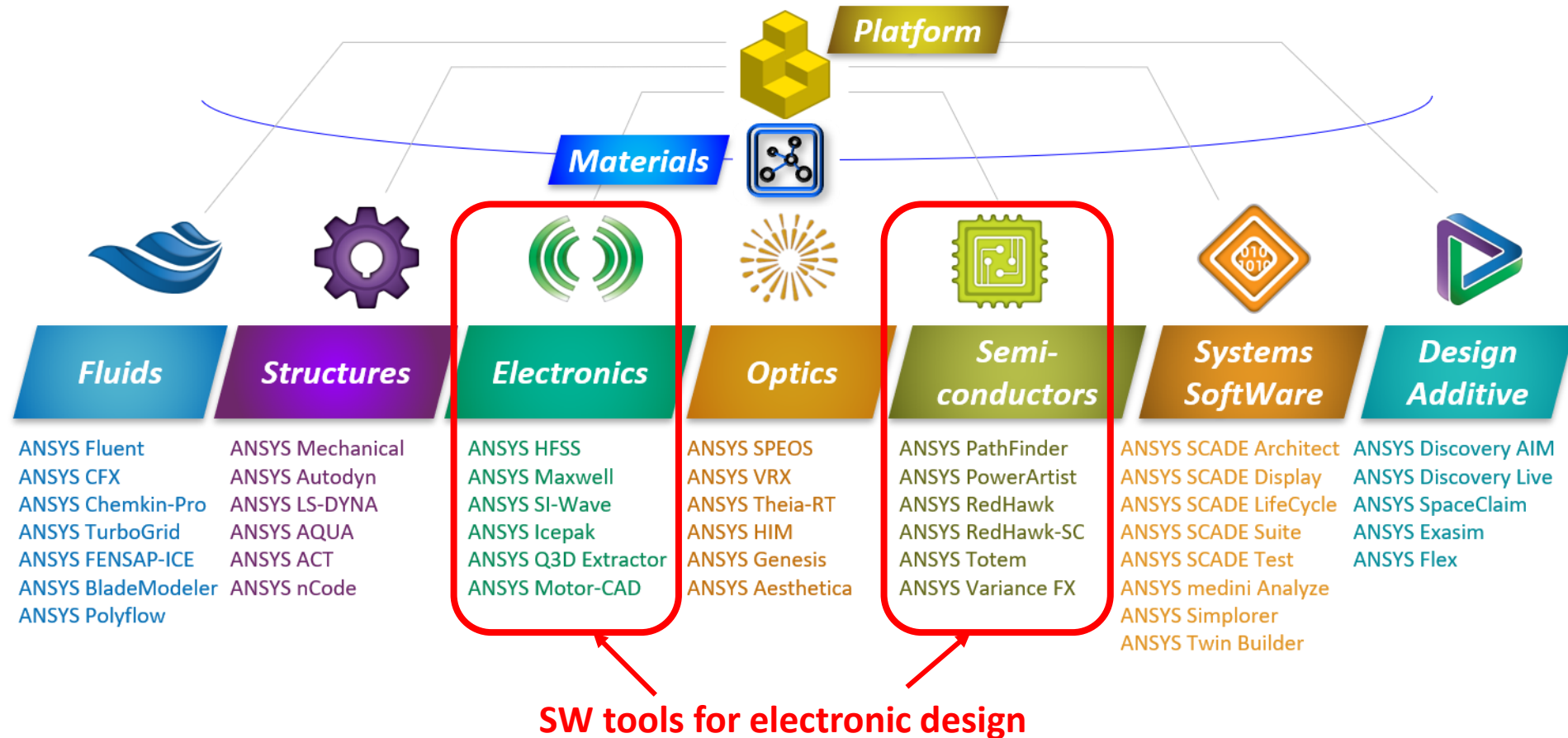
EDA market capitalization in 2025 - Main EDA vendors



Synopsys Cadence Design Systems Ansys
 Siemens EDA Altium Zuken
 Keysight Simulia (Dassault SE) Altair Eng.

Overview of EDA Tools and Usage

EDA software: an example - ANSYS



2. Current challenges of EDA – Benefits of AI/ML for EDA

Current challenges of EDA – Benefits of AI/ML for EDA

► Current challenges for EDA

- ✓ Computational time and resources for some simulations (e.g. SPICE simu. with transistor-based models, 3D EM/thermal simulations, ...)
- ✓ Simulation data are usually sparse
- ✓ Multi-objective design optimization needs to adjust tens of design variables → limit of human intervention in the optimization process
- ✓ Problem of confidentiality when IP cores and models are shared

- ✓ AI/ML → new methodologies and tools for electronic designers to solve more sophisticated problems
- ✓ Main purpose of AI/ML in EDA: automatic processing of simulation data for:
 - Surrogate modeling (data-driven model) → accelerate the simulation and IP preservation
 - Fully automatized design under optimization constraints, exploration of new design solutions
 - Sensitivity analysis and uncertainty propagation

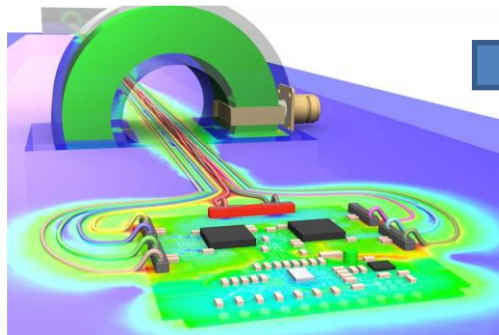
Current challenges of EDA – Benefits of AI/ML for EDA

► Traditional physics modeling vs. Surrogate modeling

- ✓ Surrogate modeling (or metamodeling) = generation of a simplified and computationally-efficient approximation of a more complex, high-order model.
- ✓ Example: EM/SI/PI analysis of a PCB:

Traditional numerical method

Numerical solving of
diff. equations (e.g.
FEM, FDTD, MoM,
TLM, ...)

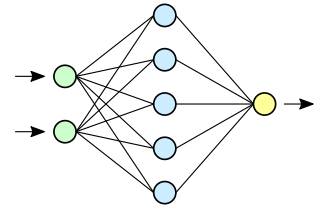


(Ansys – simulation of BCI on
automotive harness)

Hours/days of simulation to predict
PCB responses for different
combinations of design variables

Surrogate modeling

Training model to
approximate physical
behavior of all system or a
part of the system



Seconds/ minutes of simulation to
predict PCB responses for different
combinations of design variables

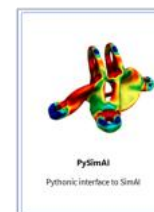
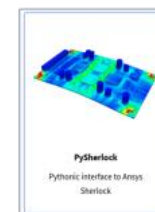
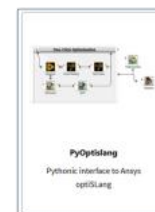
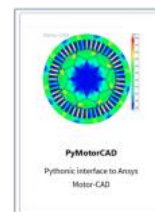
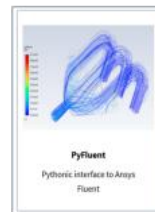
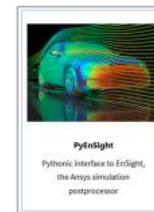
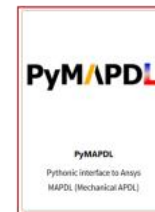
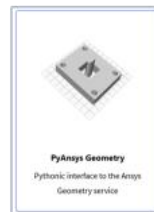
Current challenges of EDA – Benefits of AI/ML for EDA

► Python automation for AI/ML

- ✓ Scripts are used for a long time to control and interface with EDA tools
- ✓ Problem: proprietary scripting language
- ✓ Things change for 10 years with the general development of Python interfacing and automatizing for most EDA tools.
- ✓ Advantages:
 - Popular language and many open source code and libraries (especially for data processing, AI and ML)
 - Flexible and automatic deployment of design process, without human input
 - Launch automatic process of data generation and ML models



Python + ANSYS = Py/Ansys



« An Introduction to PyAnsys:
Revolutionizing Engineering
Simulations », Mar 2025, Ansys.

3. Overview of AI/ML methods

Overview of AI/ML methods

► Overview

Artificial Intelligence

Machine Learning

Natural
Language
Processing (NLP)

Reasoning

Planning

Knowledge
Representation
and Reasoning

Vision

Motion and
Manipulation

Multiagent
Systems

General
Intelligence

Supervised
Learning

Semi-supervised
Learning

Unsupervised
Learning

Reinforcement
Learning

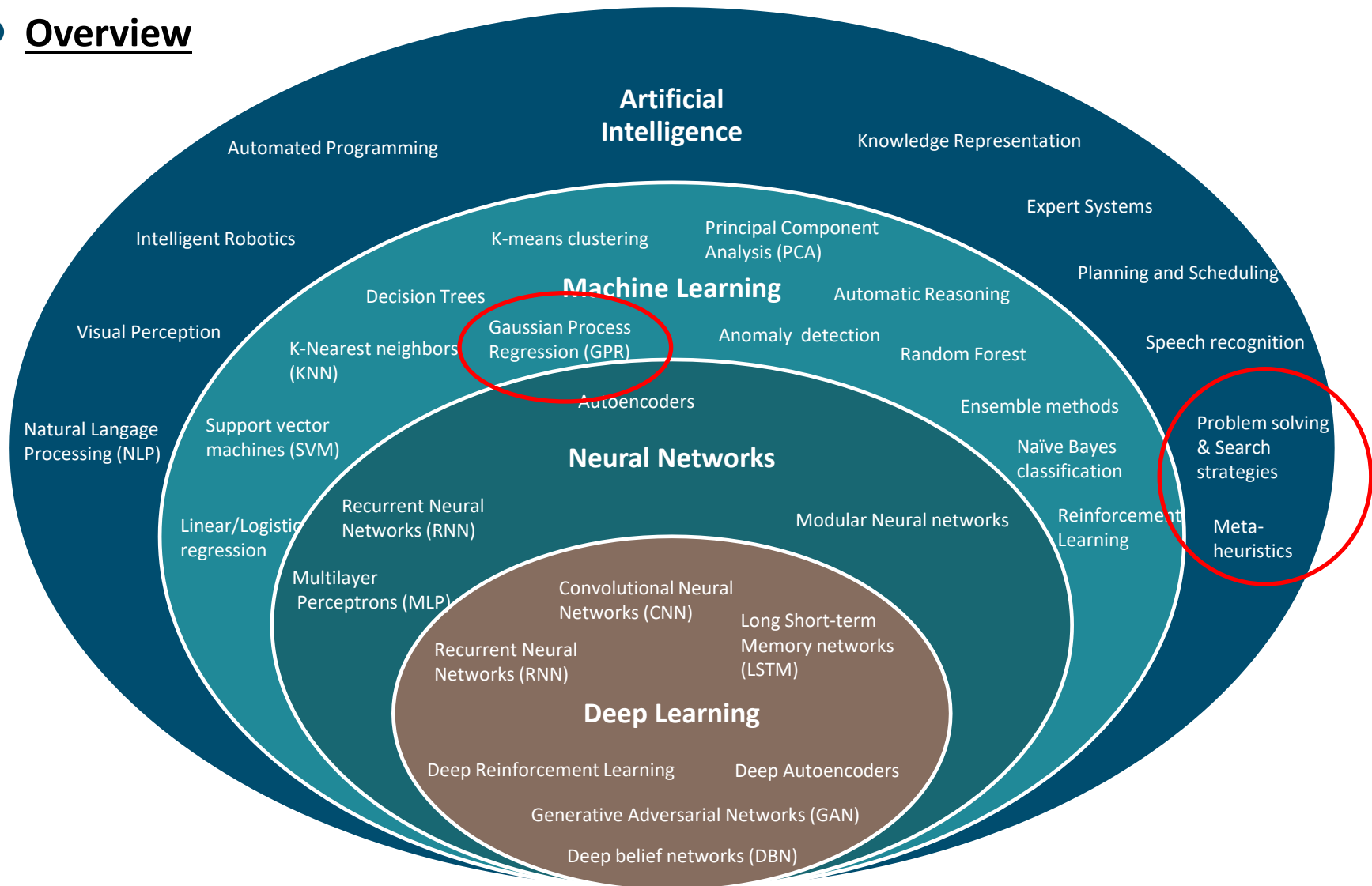
Transfer
Learning

Deep Learning

Course I4AEML21 – Machine Learning (A. Bit-Monnot, E. Chanthery, M-J. Huguet, P. Leleux, M. Siala)

Overview of AI/ML methods

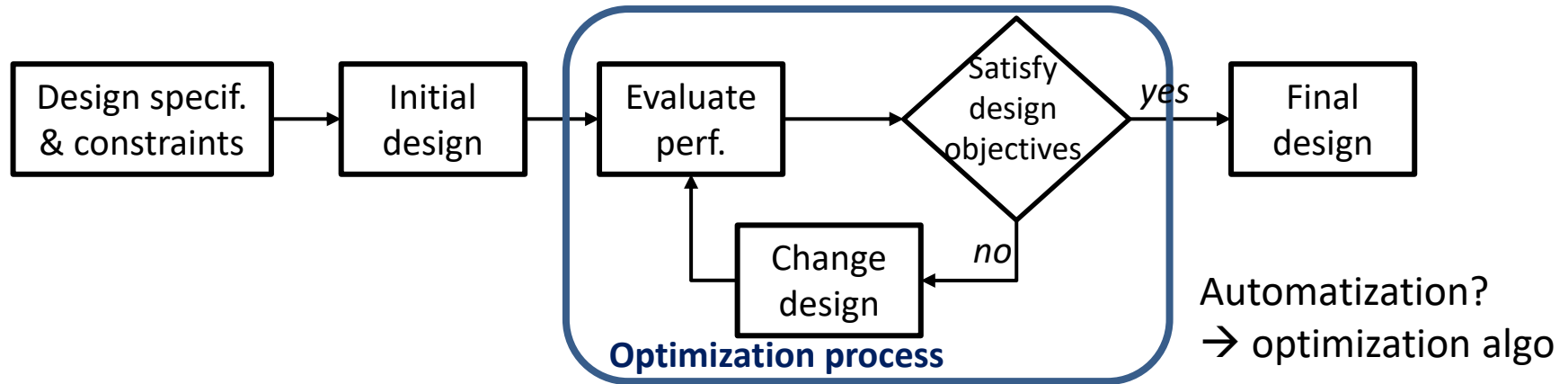
► Overview



4. Metaheuristics for electronic design optimization

Metaheuristics for electronic design optimization

► Typical engineering optimization process



✓ Advantages:

- Reduce the risk of human errors, avoid misleading intuition
- Accelerate design process when design variables increase (humans manage 2-3 variables)
- Systematic procedure

✓ Drawbacks

- High computational resources and time
- Depend on problem specified by designer (algo may find inadequate solutions)
- Optimal design may be counterintuitive and difficult to interpret
- Most of optimization algo do not guarantee to find the optimal design (local vs. global minimum)

Metaheuristics for electronic design optimization

► Optimization problem

Objective or cost function
(explicitly known or
black-box)

Design point

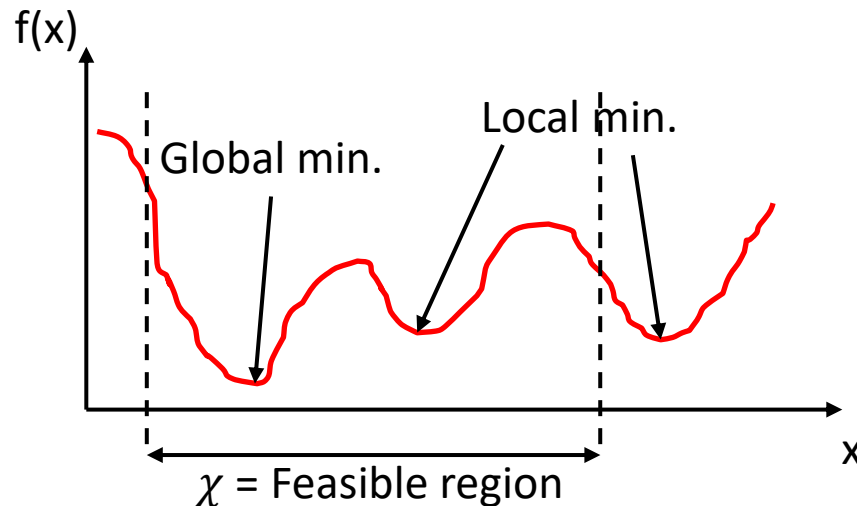
N design variables (real
or integer values)

$$\arg \min_x f(x), x = [x_0; x_1; \dots; x_N]$$

subject to $x \in \chi$

Constraints (linear
and/or non-linear)

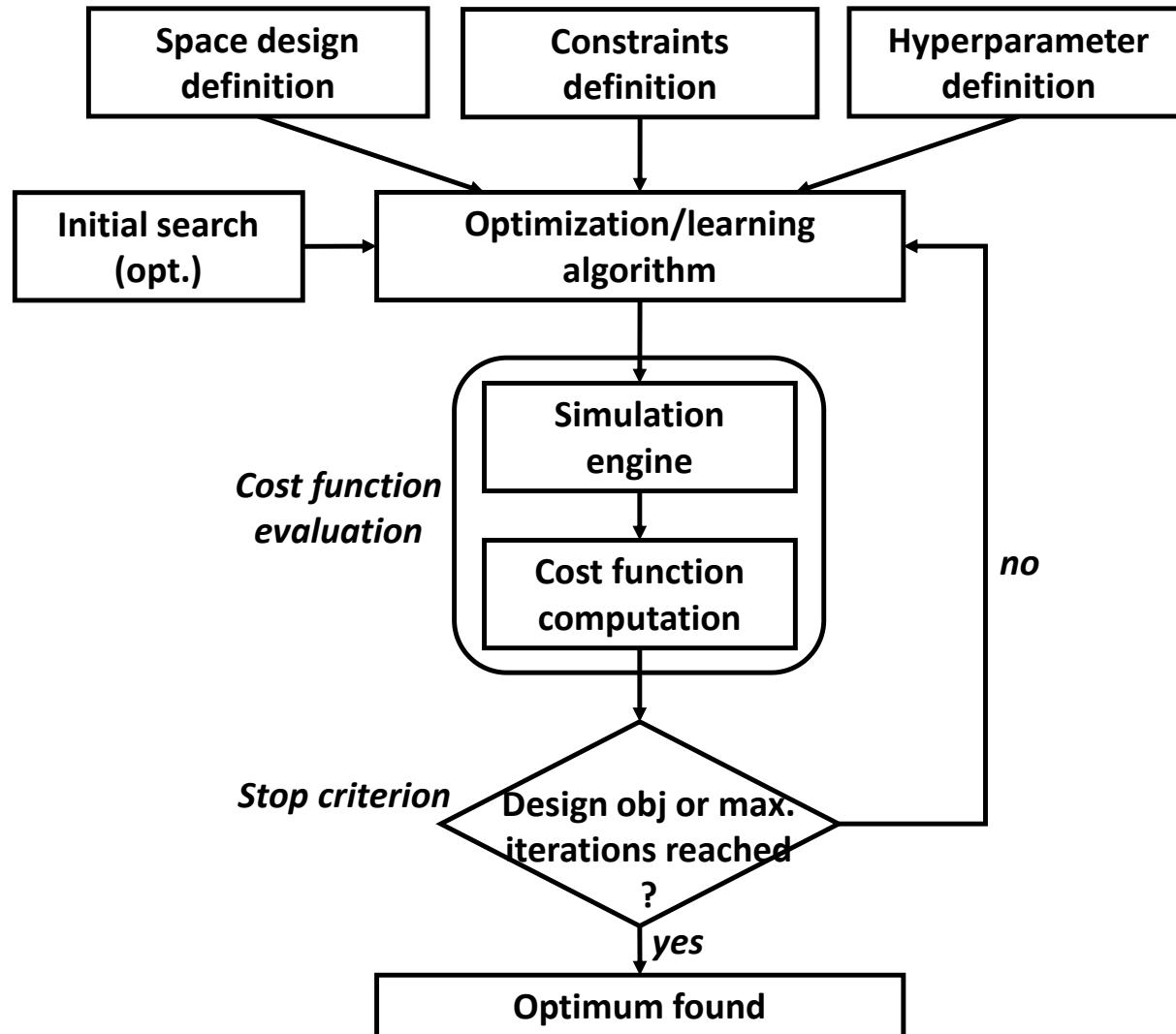
✓ 1D problem illustration:



Purpose: if it exists, find the global minimum in the feasible region defined by the constraints

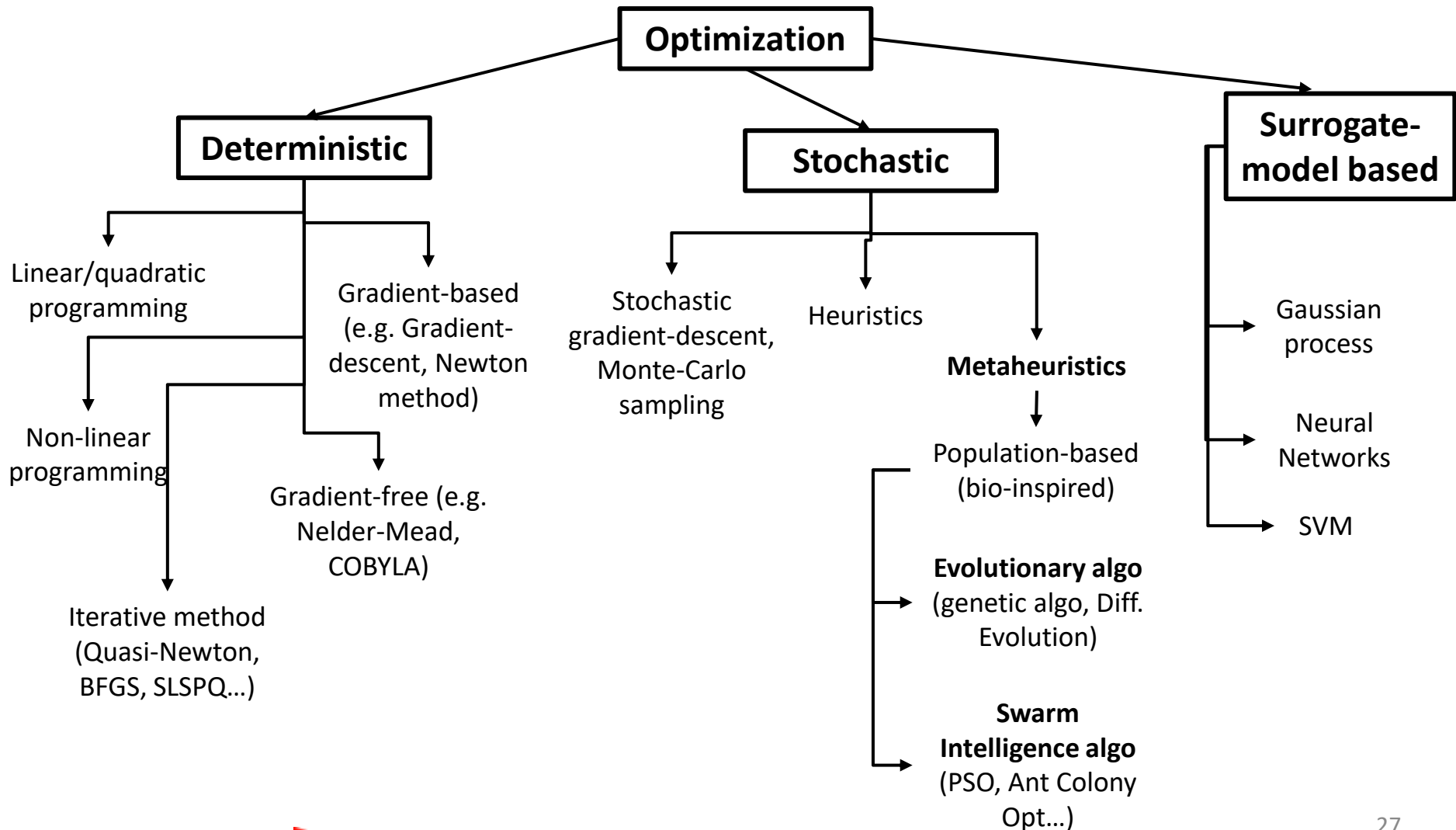
Metaheuristics for electronic design optimization

► Typical optimization flow in electronic design



Metaheuristics for electronic design optimization

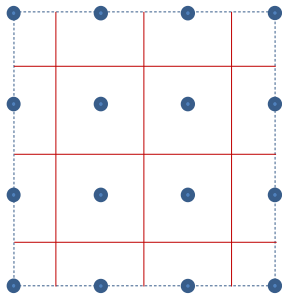
► Overview of optimization algorithms (not exhaustive list)



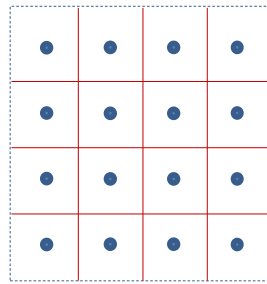
Metaheuristics for electronic design optimization

► Design space sampling

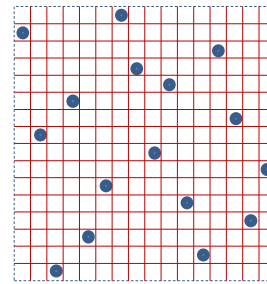
- ✓ Initial search requires an initial design space sampling.



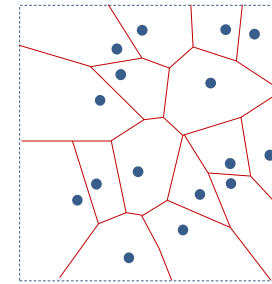
Regular grid



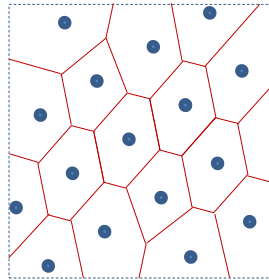
Sukharev grid



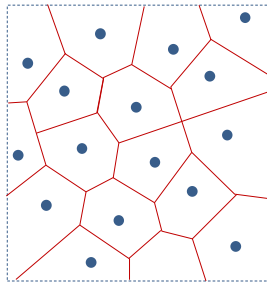
Latin Hypercube sampling



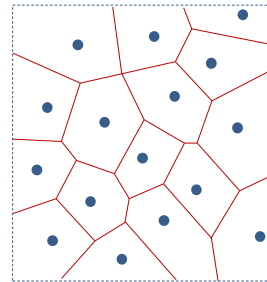
pseudo-random sampling



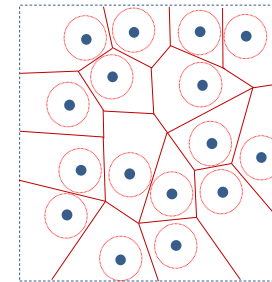
Lattice sampling



Hammersley sampling



Halton sampling

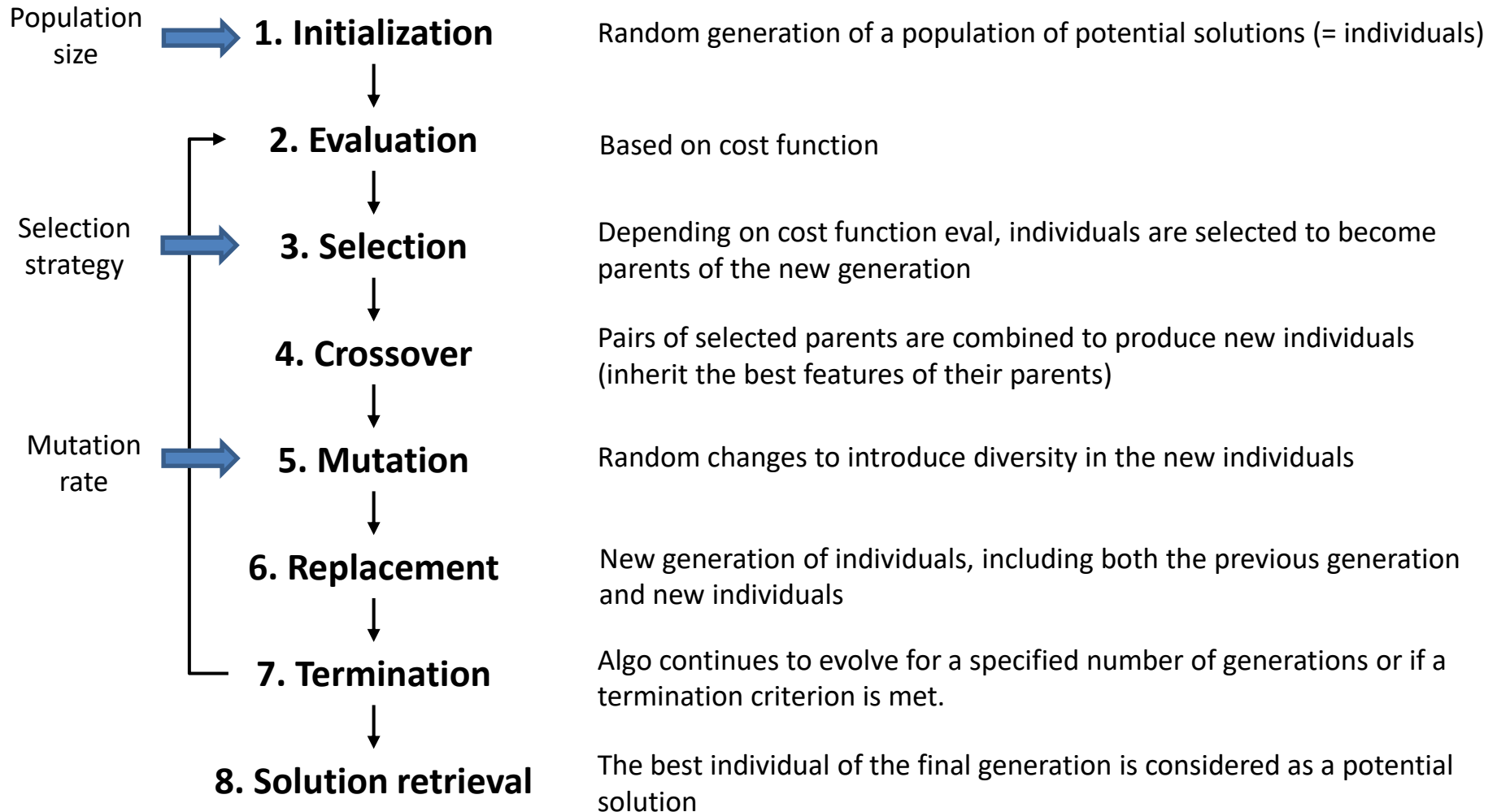


Poisson disk sampling

S. Serpaud et al., "Efficiency of Sequential Spatial Adaptive Sampling Algorithm to Accelerate Multifrequency Near-Field Scanning Measurement", IEEE Trans. on EMC, 2022.

Metaheuristics for electronic design optimization

► Genetic algorithm



Metaheuristics for electronic design optimization

► Differential evolution

- ✓ Simplified approach of genetic algorithm, usually more performant. Heavily parallelized.

Population size N → **1. Initialization**

Random generation of N individuals: individual $x = [x_0; x_1; \dots; x_n]$

↓
2. Evaluation

Based on cost function f

Impact of N, F and CR ?

Evolution strategy, F →

↓
3. Evolution

$X' = x_{\text{best}} + F \times (x_{\text{rand0}} - x_{\text{rand1}})$, F = mutation coefficient $\in [0; 2]$

Crossover proba CR →

↓
4. Crossover

Random selection of a number $r_i \in [0; 1]$

For each entry i of an individual :
$$x_{\text{new}}[i] = \begin{cases} x'[i] & \text{if } r_i < CR \\ x_{\text{old}}[i] & \text{if } r_i > CR \end{cases}$$

↓
5. Evaluation

Cost function evaluation of new individuals.

If $f(x_{\text{new}}) < f(x_{\text{old}})$, x_{new} replaces x_{old}

↓
7. Termination

Algo continues to evolve for a specified number of generations or if a termination criterion is met.

↓
8. Solution retrieval

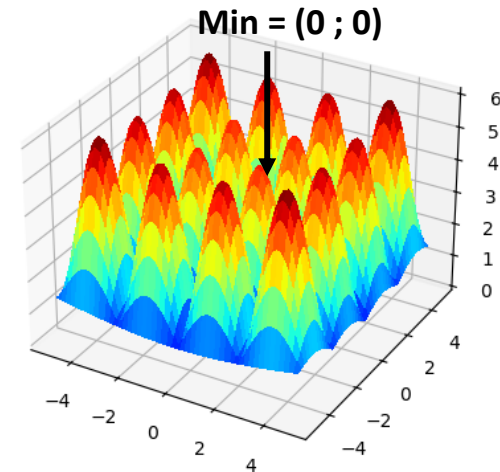
The best individual of the final generation is considered as a potential solution

Metaheuristics for electronic design optimization

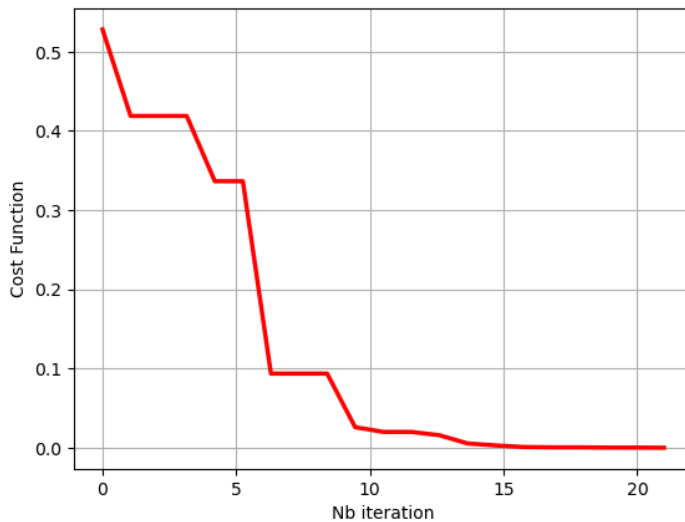
► Differential evolution

✓ Example: minimize $f(x, y) = \sqrt{x^2 + y^2} + \sin\left(\frac{2\pi}{5}x\right)\sin\left(\frac{2\pi}{5}y\right)$.

- Bounds = $[-5;5]$, $[-5;5]$
- Population size = 40
- Nb of iterations = 20
- Proba mutation = 0.7
- Coef mutation = 0.5



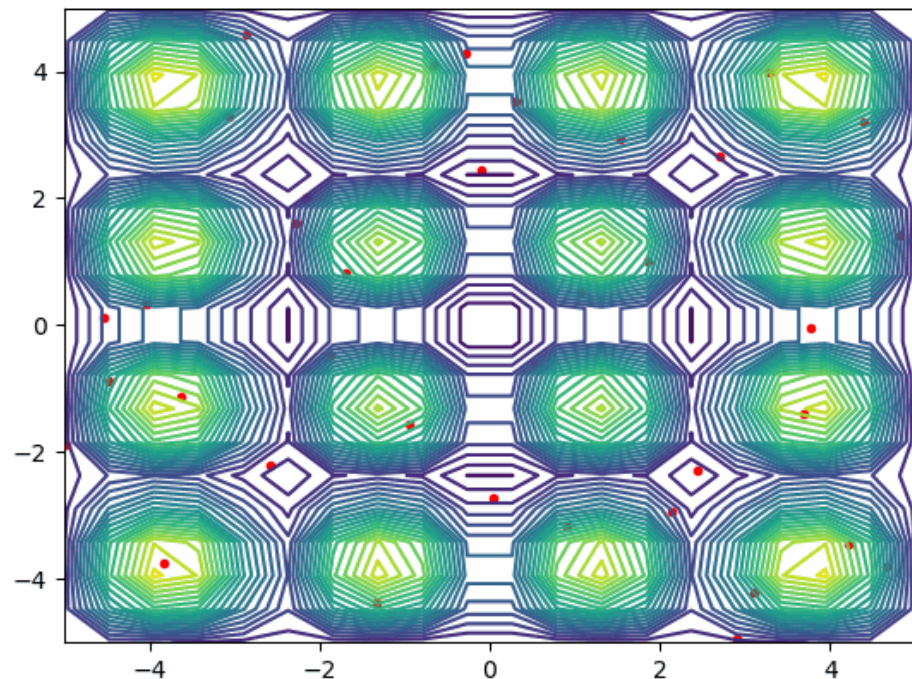
Evolution of best solution after each iteration



Convergence

Solution found = (4.440e-04 ; -2.441e-04)

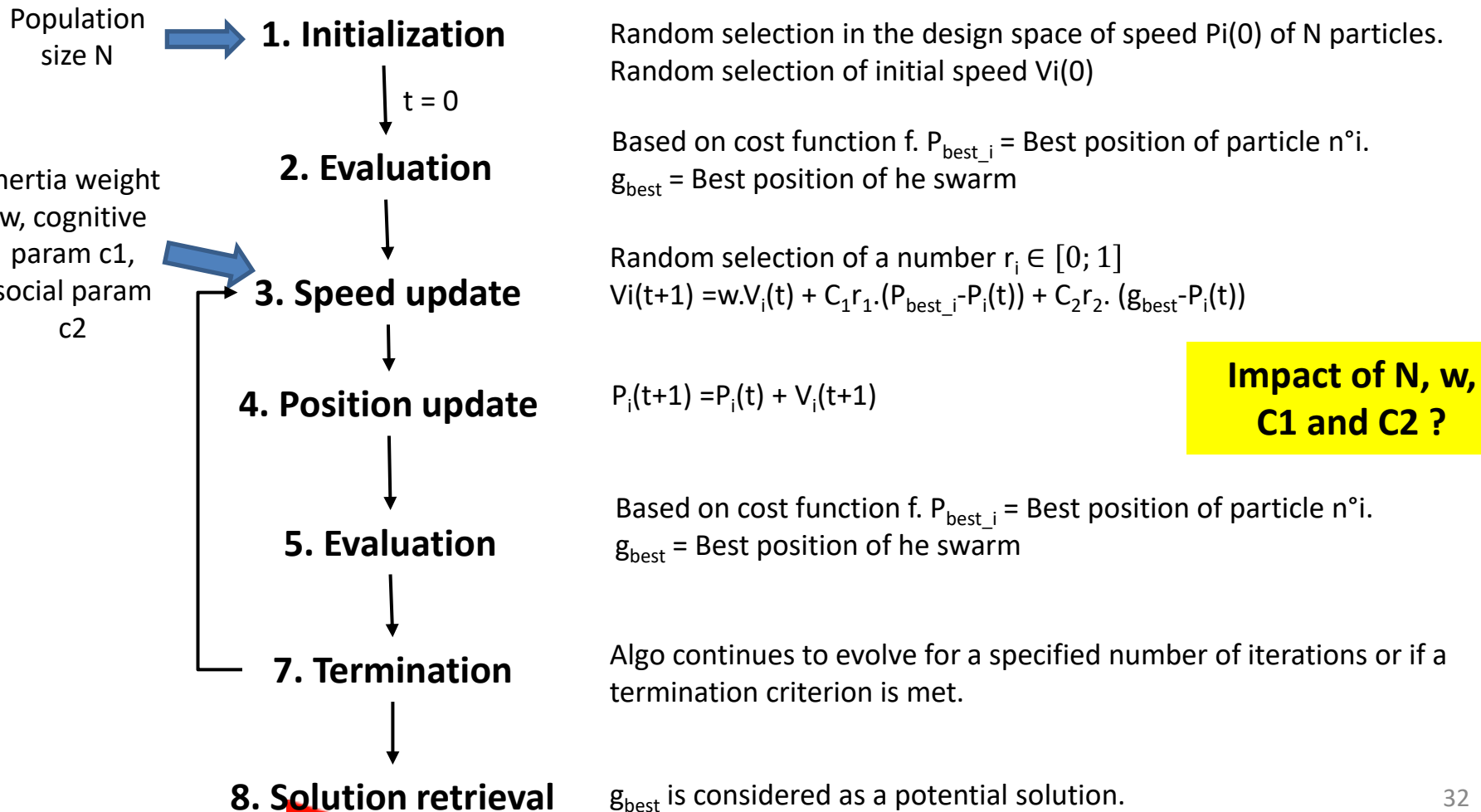
iter = 0



Metaheuristics for electronic design optimization

► Particle Swarm Optimization

- ✓ Based on the movement of individual particles. Heavily parallelized.

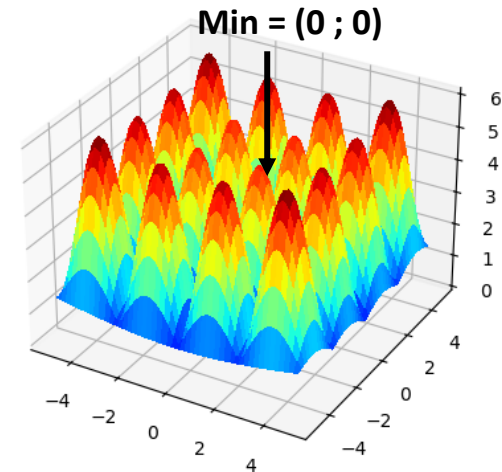


Metaheuristics for electronic design optimization

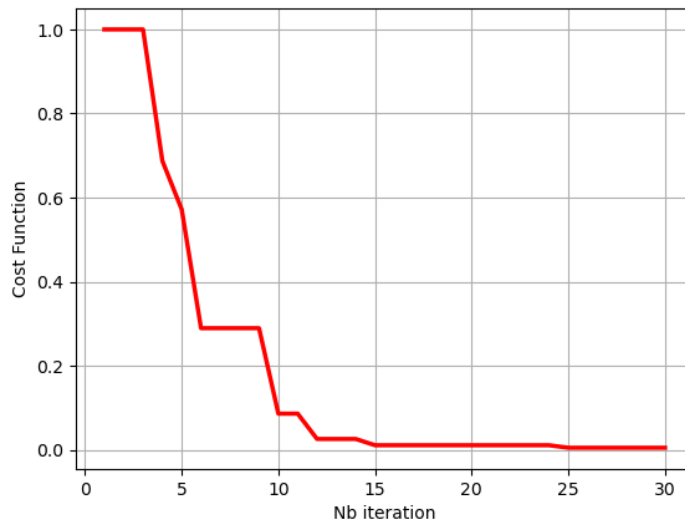
► Particle Swarm Optimization

✓ Example: minimize $f(x, y) = \sqrt{x^2 + y^2} + \sin\left(\frac{2\pi}{5}x\right) \sin\left(\frac{2\pi}{5}y\right)$.

- Bounds = $([-5;5], [-5;5])$
- Population size = 30
- Nb of iterations = 30
- $W = 0.8$
- $C1 = C2 = 0.5$



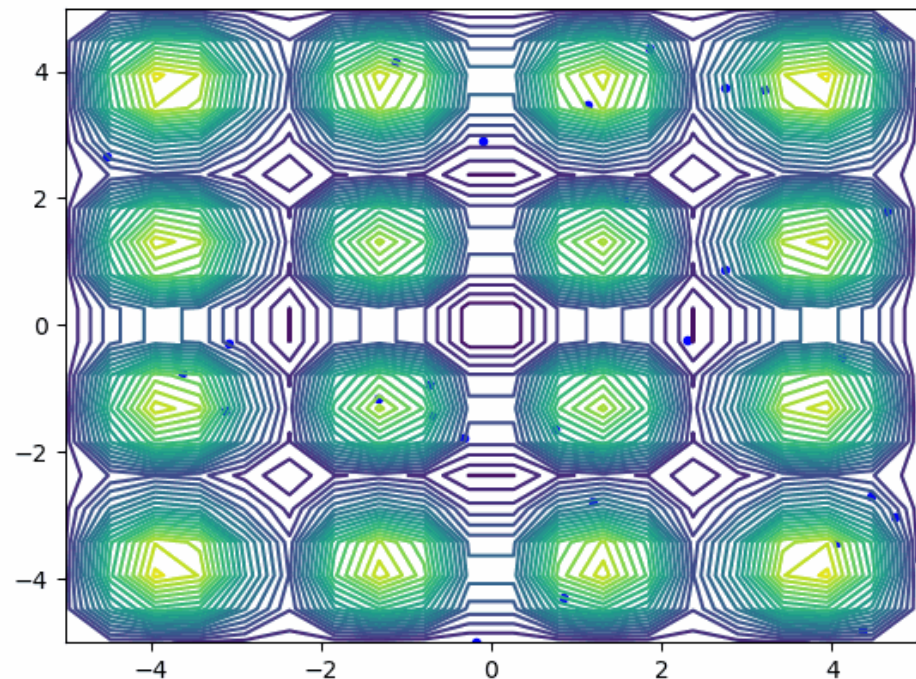
Evolution of best solution after each iteration



Convergence

Solution found = (0.01429838 ; -0.01161504)

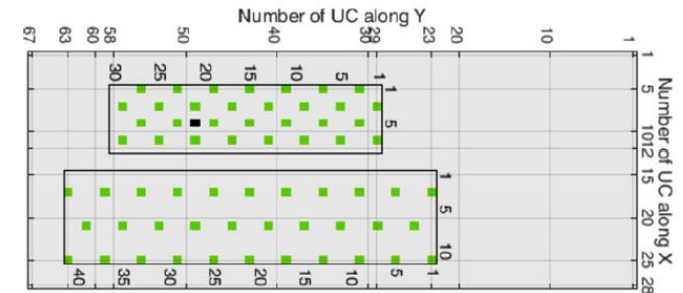
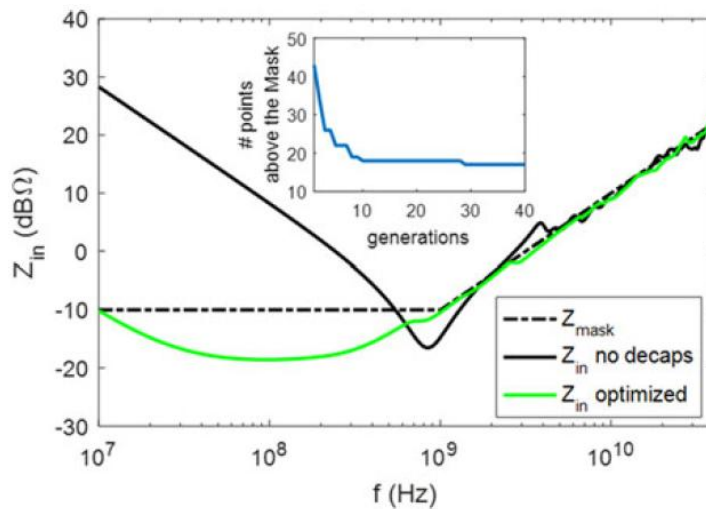
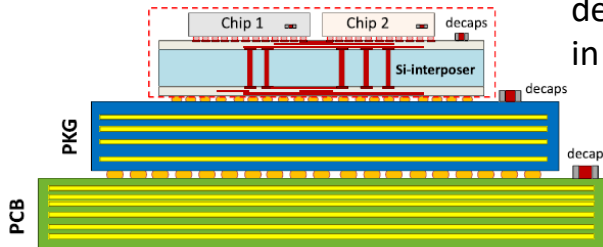
iter = 0



Metaheuristics for electronic design optimization

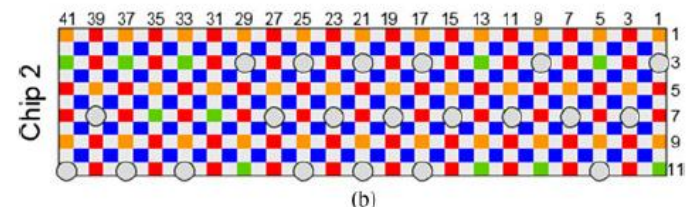
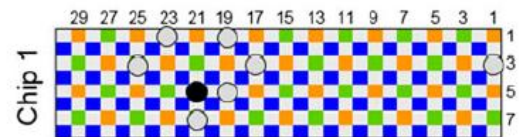
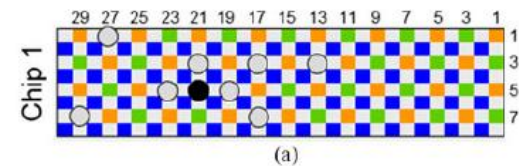
Genetic Algorithm/Particle Swarm Optimization - Example for electronic design

Optimal values/placement of decoupling capacitors (decaps) in multichip module ?



Possible positions of decaps on chips 1 and 2

Nature Inspired Algo.
(belong to GA)



Final positions of decaps on chips 1 and 2

S. Piersanti, F. de Paulis, C. Olivieri, A. Orlandi, "Decoupling Capacitors Placement for a Multichip PDN by a Nature-Inspired Algorithm", IEEE Trans. On EMC, vol. 60, no. 6, Dec. 2018.

Metaheuristics for electronic design optimization

► Multidisciplinary Optimization (MDO)

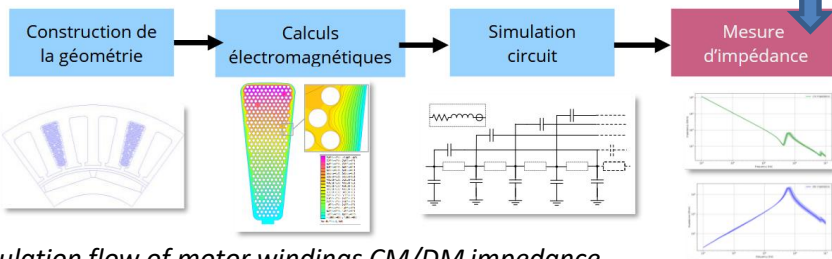
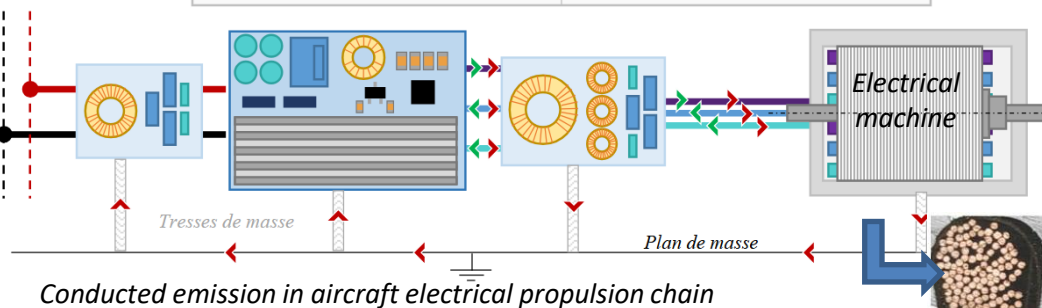
- ✓ Usage of optimization methods to solve design problems incorporating several disciplines simultaneously
- ✓ Examples of applications: in aircraft design: aero-structural optimization, electrical systems optimization
- ✓ Example: open source library for MDO:

GEMSEO

<https://gemseo.readthedocs.io>



► *Courant de Mode Commun* ► *Courant de Mode Différentiel*



Optimize geometry (conductor numbers, placement, insulator thickness...) to reduce EM emission, volume and weight

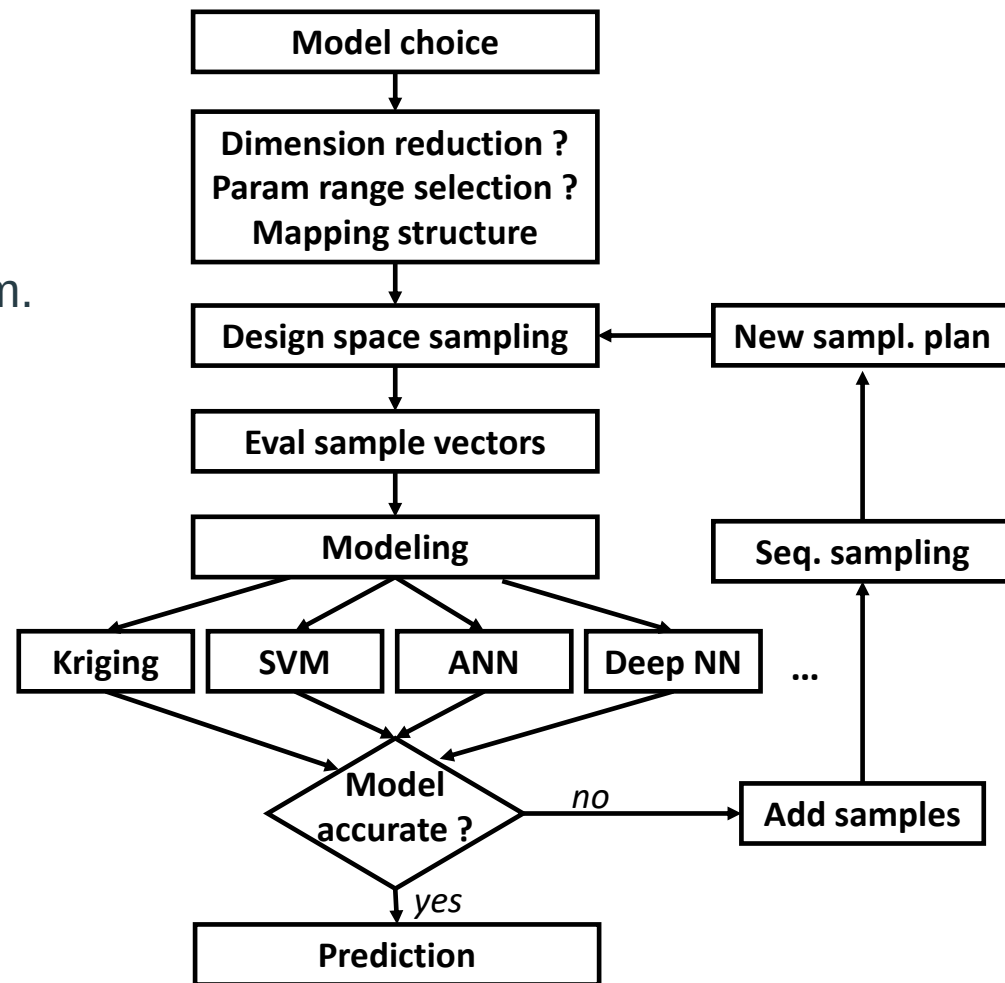
A. Piat, "Contribution à la prise en compte de paramètres incertains dans la modélisation prédictive des impédances de Mode Commun et Différentiel d'une machine électrique à grand nombre de spires", PhD Thesis, Univ. Paris-Saclay, 2024.

5. Supervised learning methods for surrogate modeling

Supervised learning methods for surrogate modeling

Supervised learning

- ✓ Construct surrogate black box or metamodels, i.e. an alternative mathematical model to simplify the representation of a complex system.
- ✓ Use given input data and corresponding labelled output data to train a model → accurate output predictions for new inputs.
- ✓ Advantages:
 - simplified representation
 - acceleration of simulation
 - protect IP
- ✓ Drawbacks:
 - no physics inside
 - no design details !!!

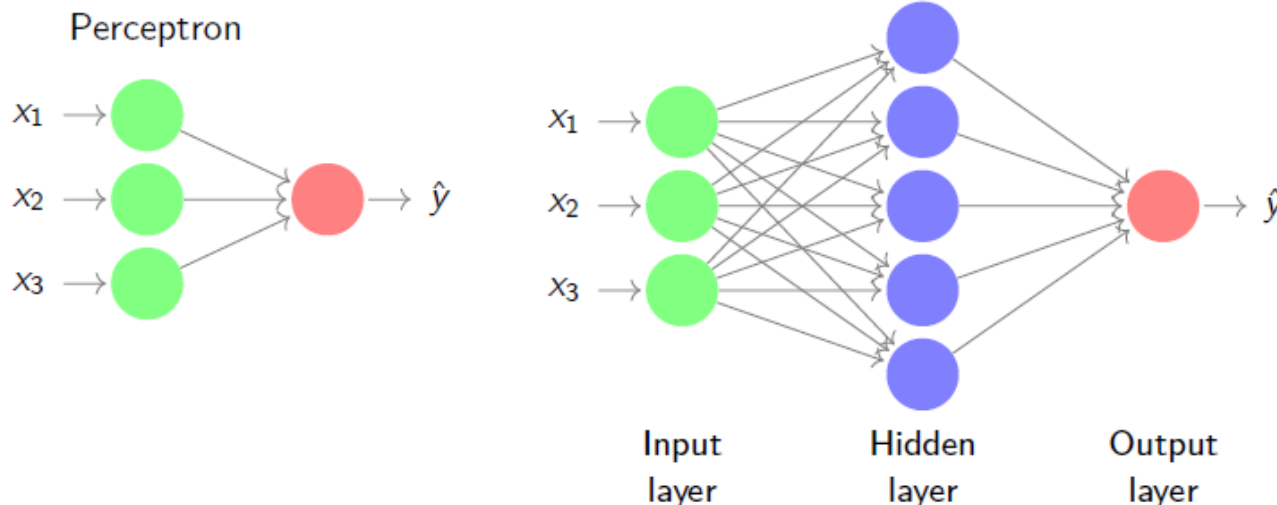


Generic flow for surrogate model construction

Supervised learning methods for surrogate modeling

► Artificial Neural Networks – Feedforward network

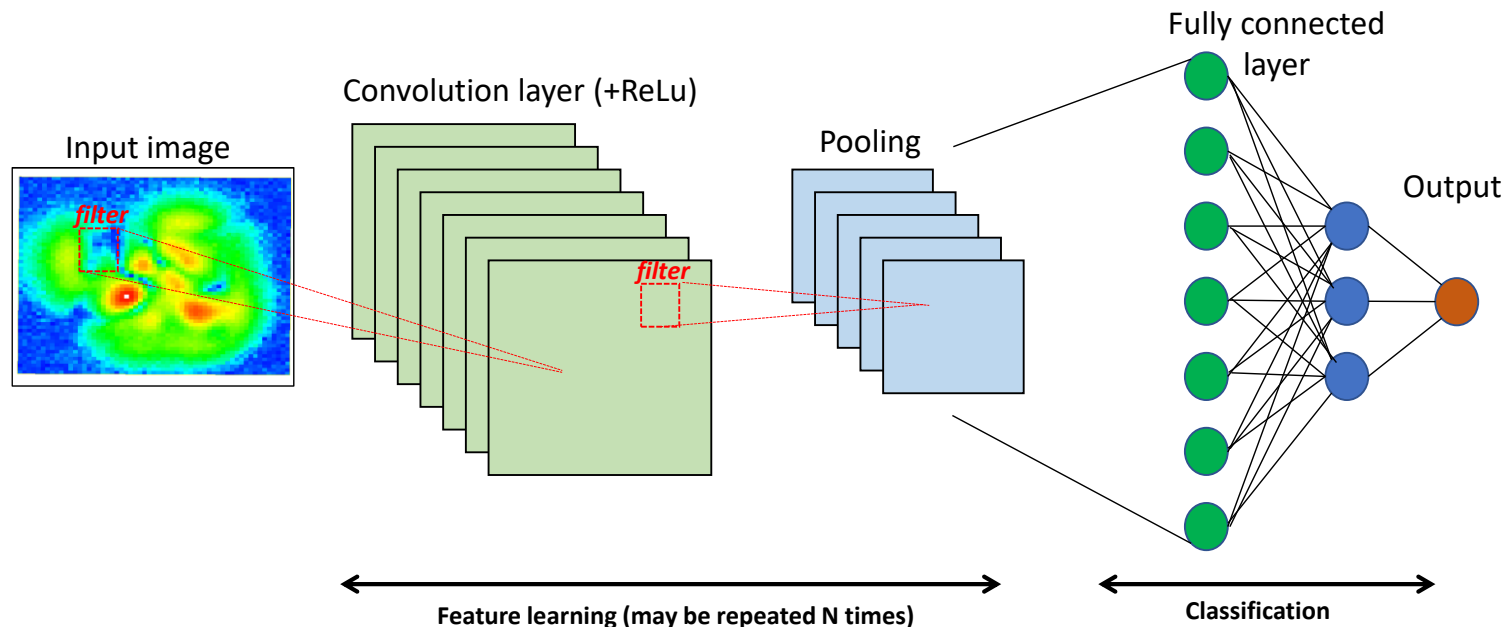
- ✓ Possibility to mimic any complex continuous function (Universal Approximation Theorem)
- ✓ Heterogeneous inputs can be considered
- ✓ High dimensions $\rightarrow$ multilayer perceptron
- ✓ Several layers = Deep neural network
- ✓ Risks: determination of the optimal hidden parameters (number of layer, weight, bias), risk of overfitting.



Supervised learning methods for surrogate modeling

► Artificial Neural Networks – FCNN, CNN and RNN

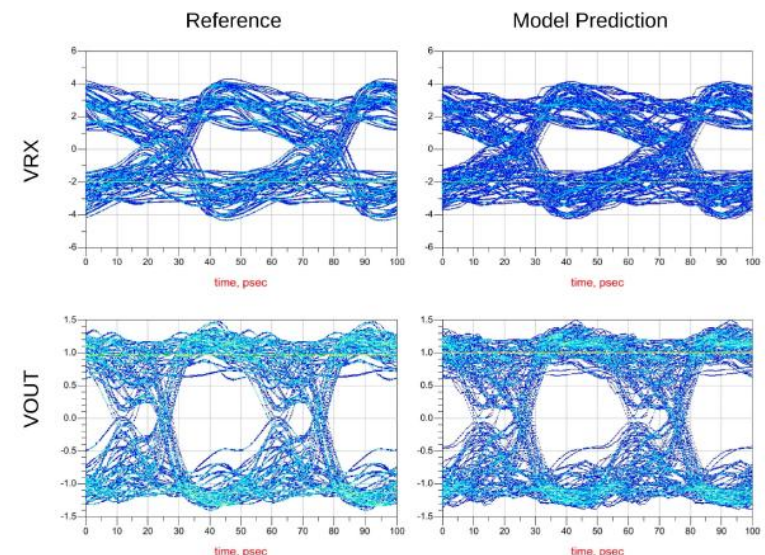
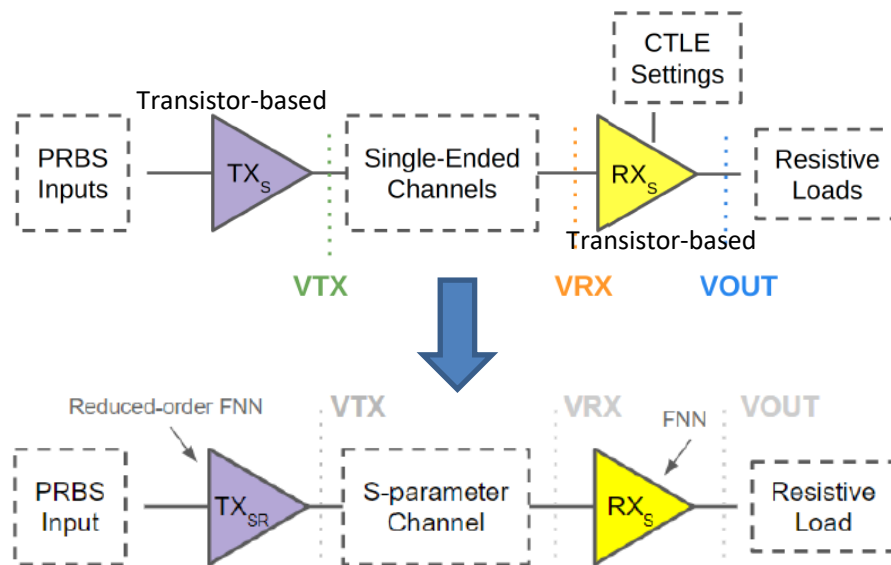
- ✓ Fully Connected Neural Network (FCNN): each input is connected to all the neurons. Unnecessary connections in high dimensions problems.
- ✓ Convolutional Neural Network (CNN): use of a filter (convolutional layer) to connect the previous to the next neuron layer → connection of each neuron to a local area of the input layer
- ✓ Recurrent Neural Network (RNN): connection of neurons of the same layer → capture dynamical effects of the modeled system (e.g. sequential circuits).



Supervised learning methods for surrogate modeling

▶ Artificial Neural Networks – Example for electronic design

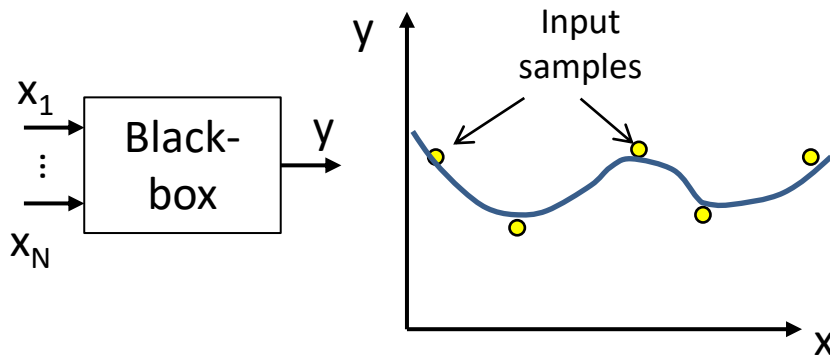
- Modular behavioral model of TX/RX buffers for rapid and accurate prediction of transient waveform in high-speed link (Signal Integrity prediction)
- Based on a 3 hidden layers feedforward neural network
- Trained using collection of discretized reference waveforms obtained under the typical operating conditions of the transceivers → extraction of kernel matrices independent of the channel.
- Alternative to IBIS file (table-based current sources and passive devices)
- 20x faster than conventional SPICE-like circuits.



Supervised learning methods for surrogate modeling

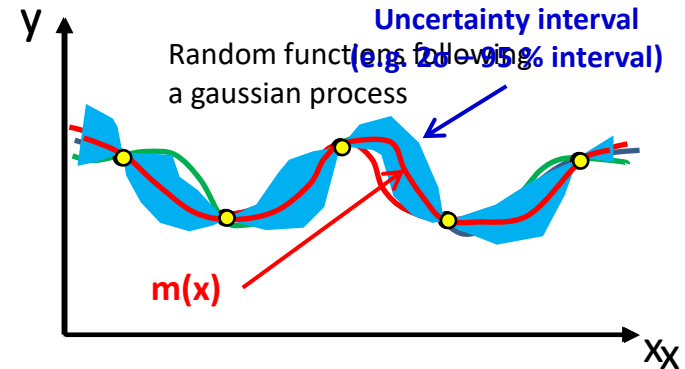
► Gaussian Process Regression (GPR) or kriging

Typical regression model



- ✓ Train one regression function from input data to minimize RMS error

GPR model



- ✓ Probabilistic supervised ML model
- ✓ Train a set of random functions (based on a given kernel k) following a multivariate gaussian distribution
- ✓ Evaluate the mean (estimated function) and variance (uncertainty) of the model:

$$f(x) = GP(m(x), k(x, x'))$$

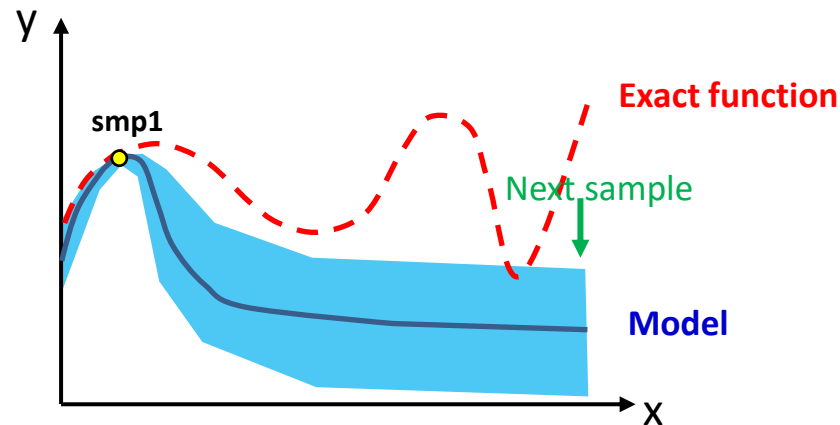
with

$$\begin{cases} m(x) = \mathbb{E}[f(x)] \\ k(x, x') = \mathbb{E}[(f(x) - m(x))(f(x') - m(x')))] \end{cases}$$

Supervised learning methods for surrogate modeling

► Bayesian optimization principle

Using GPR to sample iteratively and adaptively the design space in order to find the optimum value efficiently (e.g. in a case where the evaluation of the cost function is expensive).



- ✓ Exploration: increase the accuracy of the model → add points where the predicted uncertainty is maximum.
- ✓ To find the optimum → balance between exploration and exploitation.
- Require an **acquisition function** to determine the next sample (where it is minimized)

Example of acquisition function: lower confidence bound:

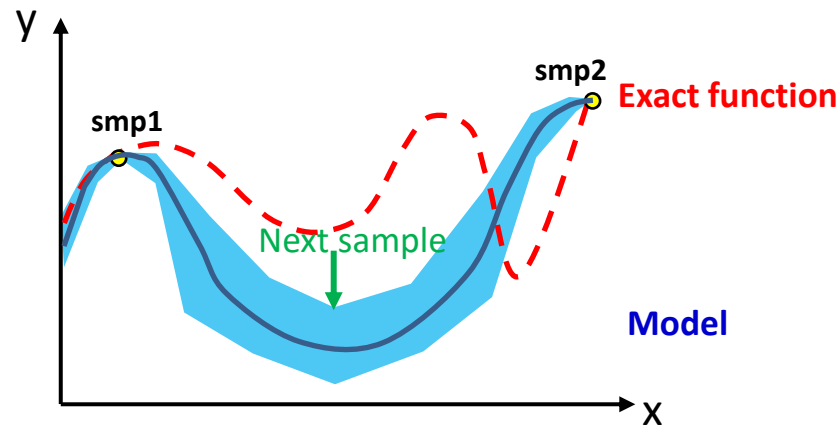
$$a(x; \lambda) = m(x) - \lambda \cdot \sigma(x)$$

↑
Increase λ improves exploration

Supervised learning methods for surrogate modeling

► Bayesian optimization principle

Using GPR to sample iteratively and adaptively the design space in order to find the optimum value efficiently (e.g. in a case where the evaluation of the cost function is expensive).



- ✓ Exploration: increase the accuracy of the model → add points where the predicted uncertainty is maximum.
- ✓ To find the optimum → balance between exploration and exploitation.
- Require an **acquisition function** to determine the next sample (where it is minimized)

Example of acquisition function: lower confidence bound:

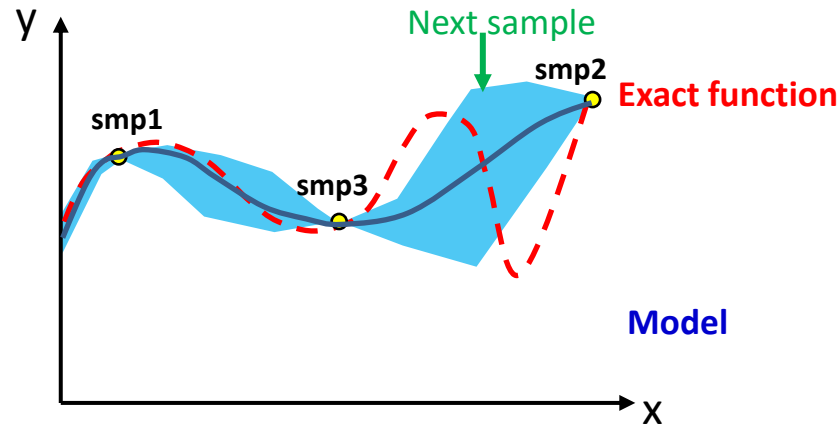
$$a(x; \lambda) = m(x) - \lambda \cdot \sigma(x)$$


 Increase λ improves exploration

Supervised learning methods for surrogate modeling

► Bayesian optimization principle

Using GPR to sample iteratively and adaptively the design space in order to find the optimum value efficiently (e.g. in a case where the evaluation of the cost function is expensive).



- ✓ Exploration: increase the accuracy of the model → add points where the predicted uncertainty is maximum.
- ✓ To find the optimum → balance between exploration and exploitation.
- Require an **acquisition function** to determine the next sample (where it is minimized)

Example of acquisition function: lower confidence bound:

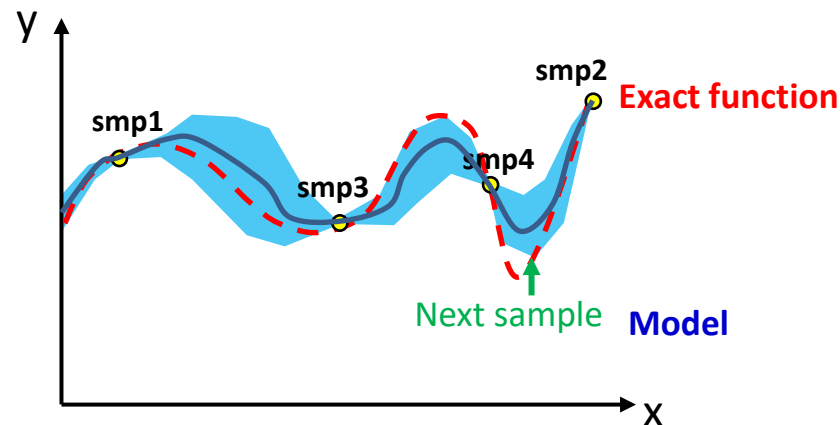
$$a(x; \lambda) = m(x) - \lambda \cdot \sigma(x)$$


 Increase λ improves exploration

Supervised learning methods for surrogate modeling

► Bayesian optimization principle

Using GPR to sample iteratively and adaptively the design space in order to find the optimum value efficiently (e.g. in a case where the evaluation of the cost function is expensive).



- ✓ Exploration: increase the accuracy of the model → add points where the predicted uncertainty is maximum.
- ✓ To find the optimum → balance between exploration and exploitation.
- Require an **acquisition function** to determine the next sample (where it is minimized)

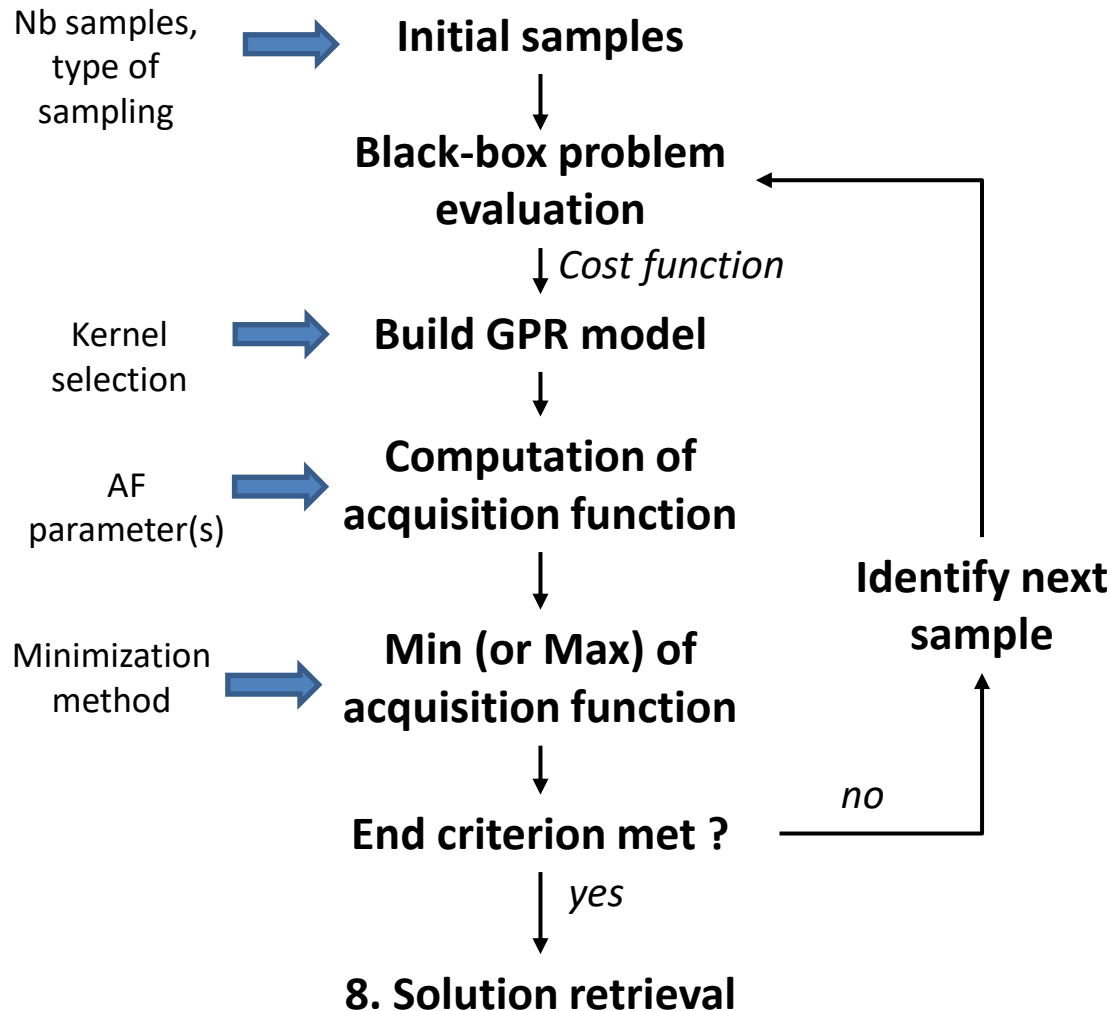
Example of acquisition function: lower confidence bound:

$$a(x; \lambda) = m(x) - \lambda \cdot \sigma(x)$$

↑
Increase λ improves exploration

Supervised learning methods for surrogate modeling

► Bayesian optimization – typical flow



✓ Advantages:

- Need few sample points to determine optimum
- → interesting when sample evaluation is costly
- Adaptive sampling
- Improvement of model at each iteration

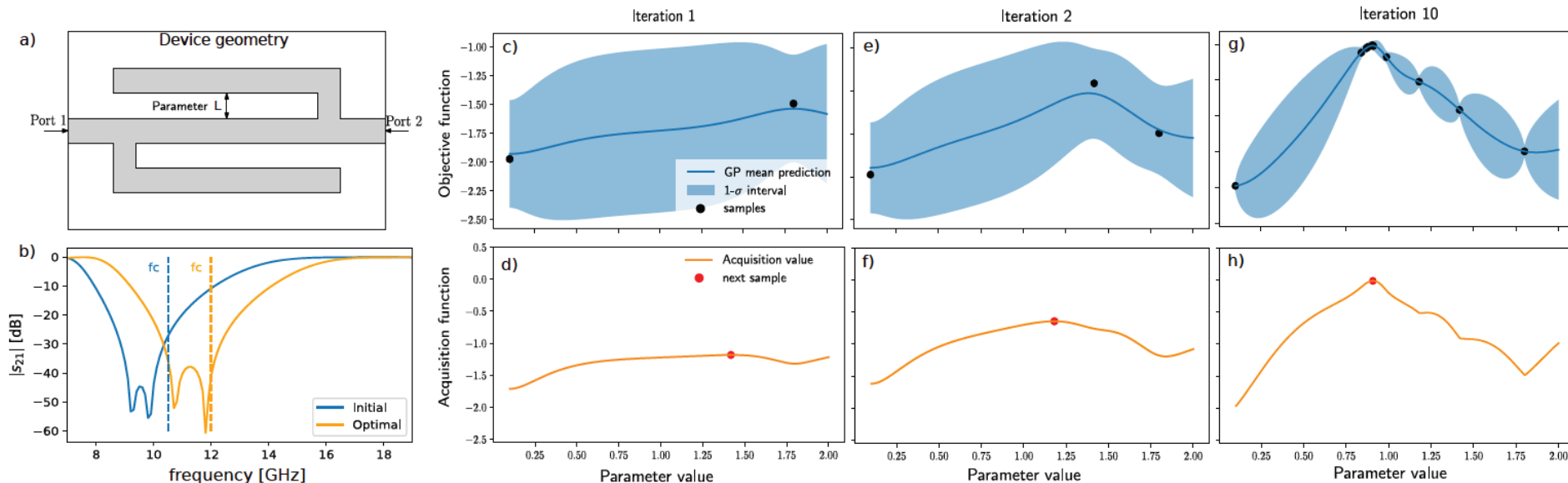
✓ Drawbacks:

- Dimensionality curse

Supervised learning methods for surrogate modeling

Bayesian optimization – Example for electronic design

- ✓ Optimization of a microstrip-based bandpass filter (folded stub filter)
- ✓ Find the best value for parameters L , S , h , ϵ to optimize the attenuation over the expected bandwidth (7 GHz) around the central frequency (12 GHz)
- ✓ BO algorithm controls a 3D full-wave EM simulator
- ✓ Around 30 samples are required



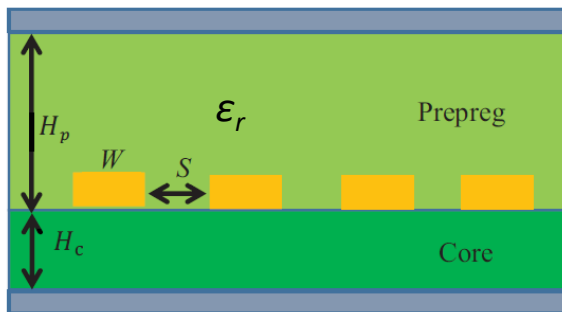
F. Garbuglia, J. Qing, N. Knudde, D. Spina, I. Couckuyt, D. Deschrijver, and T. Dhaene, "Bayesian active learning for multi-objective feasible region identification in microwave devices," *Electronics Letters*, vol. 57, no. 10, pp. 400–403, 2021

Supervised learning methods for surrogate modeling

► Bayesian optimization – Example for electronic design

- ✓ Coupling with ANN to accelerate the evaluation of the cost function
- ✓ Example: PCB/package stackup optimization:
 - Step 1: training of an ANN based on 2D EM simulation (352 samples)
 - Step 2 : Bayesian Optimization to minimize insertion loss and impedance mismatch (1000 iterations required).

Cost function: $f(\mathbf{x}) = |Z_c - Z_{target}| - a * Loss$



5 parameters to tune

Method	Time (minutes)
Genetic algo without NN	276
BO without NN	210
Genetic algo with NN	12
BO with NN	3.7

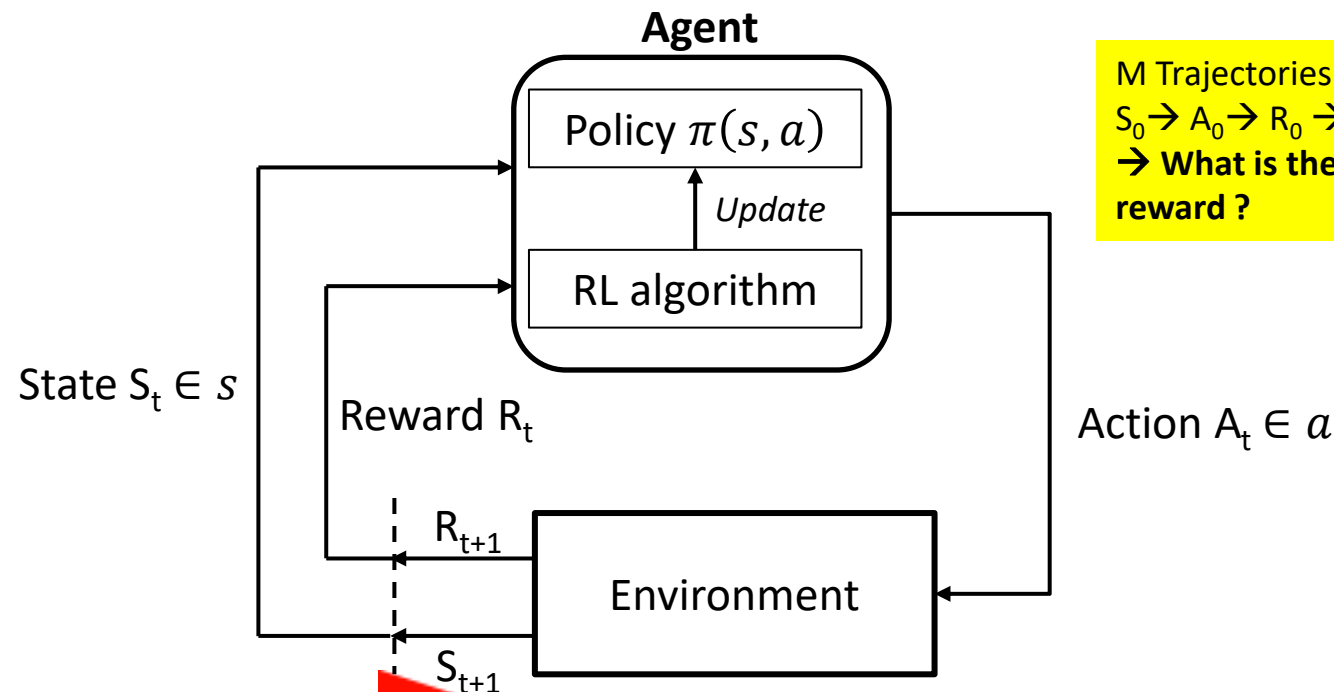
Z. Kiguradze, J. He, B. Mutnury, A. Chada, J. Drewniak, "Bayesian Optimization for Stack-up Design" IEEE Symp. on EMC, Signal and Power Integrity, Jul. 2019.

6. Reinforcement learning for electronic design optimization

Reinforcement learning for electronic design optimization

► Reinforcement learning (RL)

- ✓ Based on a Markov decision process (sequential decisions in an uncertain environment).
- ✓ RL trains an autonomous agent that iteratively learns from environment reaction with « no human in the loop » → Learning based on a Trial and error process
- ✓ The agent continuously interacts with the environment by observing the state, taking an action, obtaining a reward, observing the next state, and so on.
- ✓ Objective: Explore/exploit actions to maximize the gain (sum of rewards).



M Trajectories:

$S_0 \rightarrow A_0 \rightarrow R_0 \rightarrow S_1 \rightarrow A_1 \rightarrow R_1 \rightarrow \dots \rightarrow S_N \rightarrow A_N \rightarrow R_N$
→ **What is the policy that maximizes the total reward ?**

Reinforcement learning for electronic design optimization

► Example of RL algorithm: Q-learning

Initialization matrix $q(s,a) = 0$



Repeat M episodes



S_0 = initial state



While time step $t < N$



Choose action A_t from S_t according to policy (e.g. ϵ -greedy)



Apply A_t



Get S_{t+1} and R_{t+1} from environment



Update Q-function:

$$q(s,a) = (1 - \alpha)q(s,a) + \alpha \left(r + \gamma \max_{a'} q(s', a') \right)$$



End episodes ?

Increment t

r: reward to the agent
 α : learning factor [0;1]
 γ : actualisation factor [0;1]

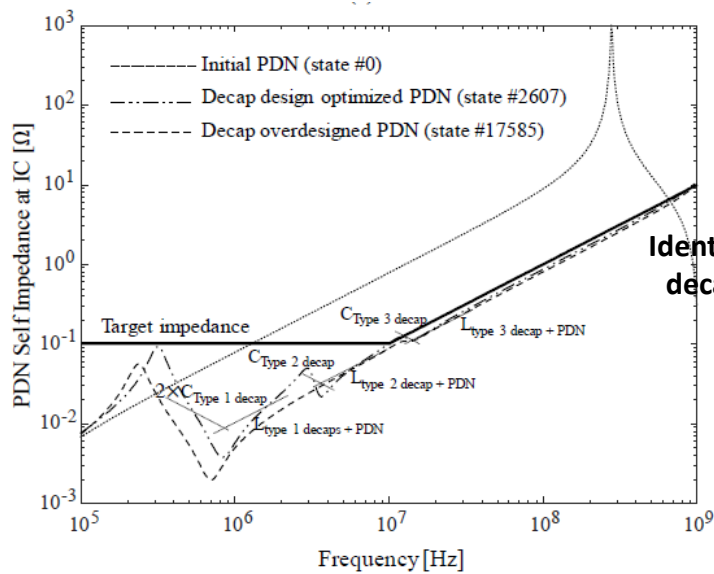
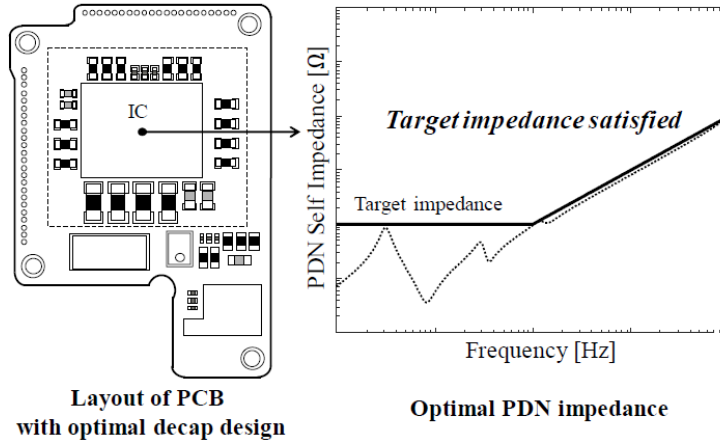
Best policy: $\pi^*(s,a) = \begin{cases} 1 & \text{if } a = \underset{a}{\operatorname{argmax}} q_\pi(a / s) \\ 0 & \end{cases}$



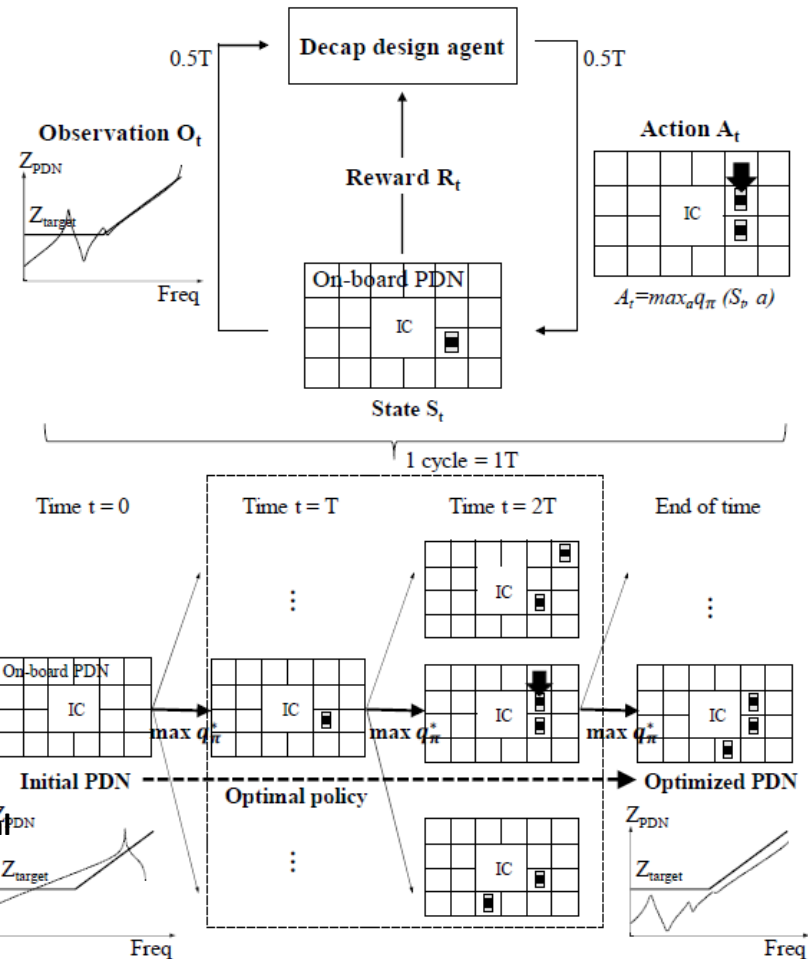
Reinforcement learning for electronic design optimization

Reinforcement Learning – Example for electronic design

Use RL to determine the optimal decoupling capacitors (decap) for power integrity purpose



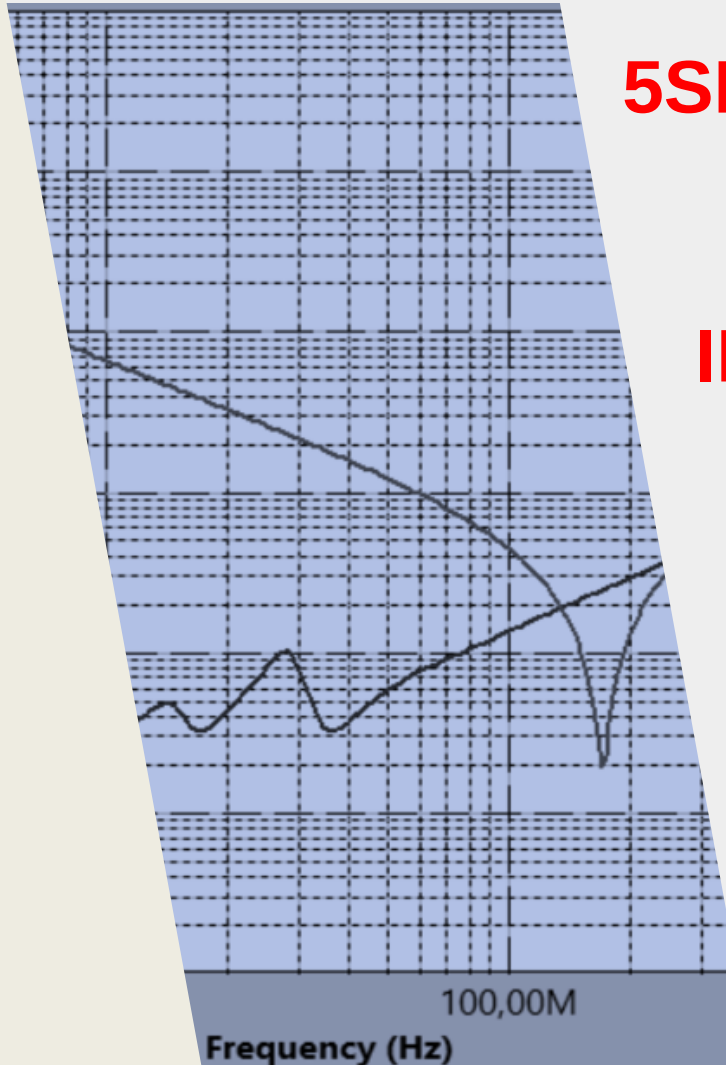
Identification of 37 optimal decap designs (4 decaps)



H. Park, J. Park, S. Kim, D. Lho, S. Park, G. Park, K. Cho, J. Kim, "Reinforcement Learning-based Optimal On-board Decoupling Capacitor Design Method," 27th Conf. On Electrical Perf. Of Electronic Packaging and Systems, Oct. 2018.

Some references

- ✓ INSA course I4AEML21 – Machine Learning, <https://moodle.insa-toulouse.fr/course/view.php?id=1790>
- ✓ H. Ren, J. Hu, « Machine Learning Applications in Electronic Design Automation », Springer, 2022, ISBN 978-3-031-13073-1.
- ✓ M. J. Kochenderfer, T. A. Wheeler « Algorithms for Optimization – 2nd Edition », MIT Press, 2025.
- ✓ L. Jiang, « Machine Learning for EMC/SI/PI – Blackbox, Physics Recovery, and Decision Making », IEEE EMC Magazine, vol. 12, Quarter 4, 2023.
- ✓ F. Garbuglia, D. Deschrijver, T. Dhaene, « On the Role of Bayesian Learning for Electronic Design Automation: A Survey », IEEE EMC Magazine, vol. 12, Quarter 4, 2023.
- ✓ M. Swaminathan, H. M. Torun, H. Yu, J. A. Hejase, W. D. Becker, «Demystifying Machine Learning for Signal and Power Integrity Problems in Packaging», IEEE Trans. On Components, Packaging and Manuf. Techno, vol. 10, no. 8, Aug. 2020.
- ✓ B. Shahriari, K. Swersky, Z. Wang, R. P. Adams, N. de Freitas, “Taking the Human Out of the Loop: A Review of Bayesian Optimization”, Proc. of the IEEE, vol. 104, No. 1, Jan. 2016.



5SEE – MACHINE LEARNING FOR ELECTRONIC DESIGN

INTRODUCTION TO THE LAB: POWER INTEGRITY ISSUES

Alexandre Boyer

alexandre.boyer@insa-toulouse.fr

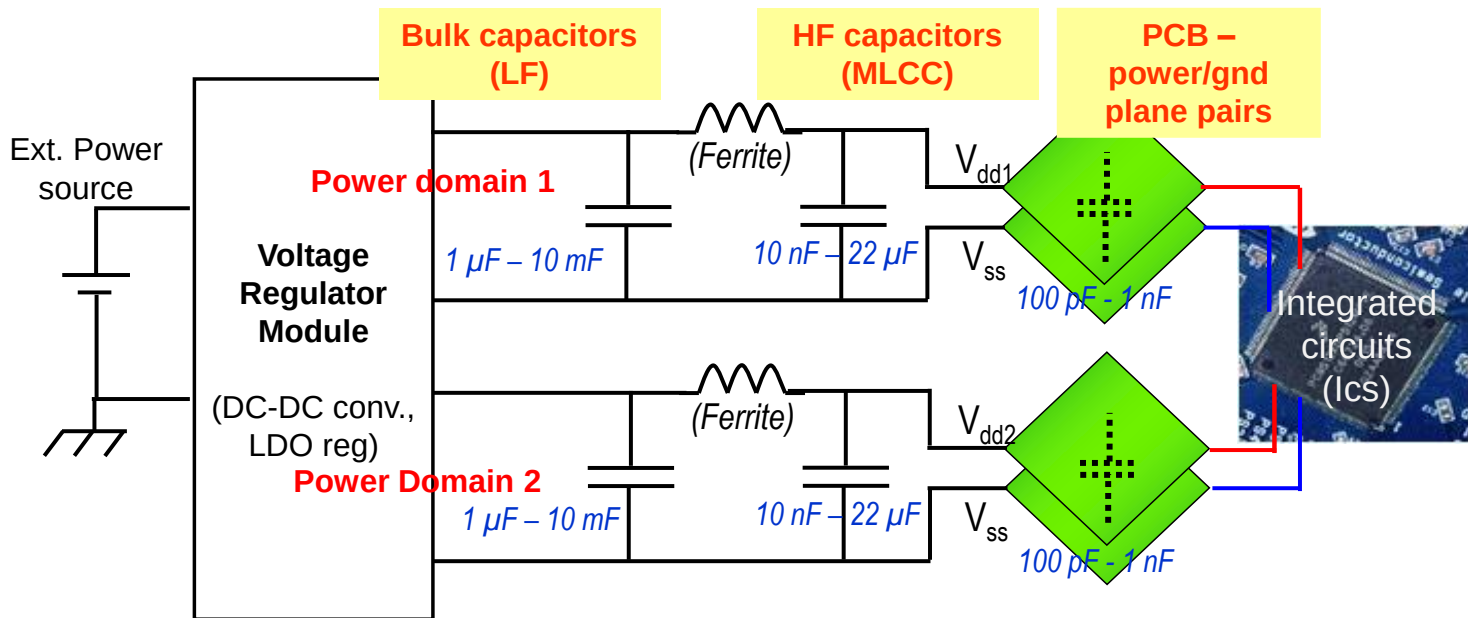
November 2025

moodle.insa-toulouse.fr

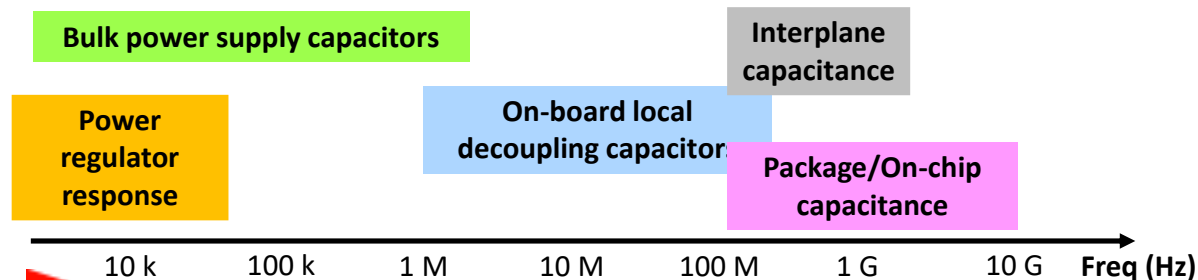
Power Distribution Network of PCB

Typical structure of multilayer PCB Power Distribution Network (PDN)

- ✓ Purpose: deliver stable power and ground references of digital ICs mounted on a PCB.



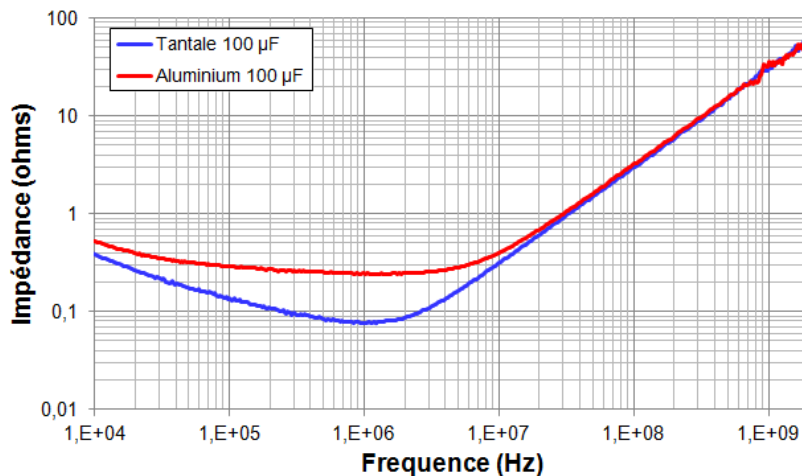
- ✓ Typical frequency range of influence of the typical elements of a multilayer PCB PDN :



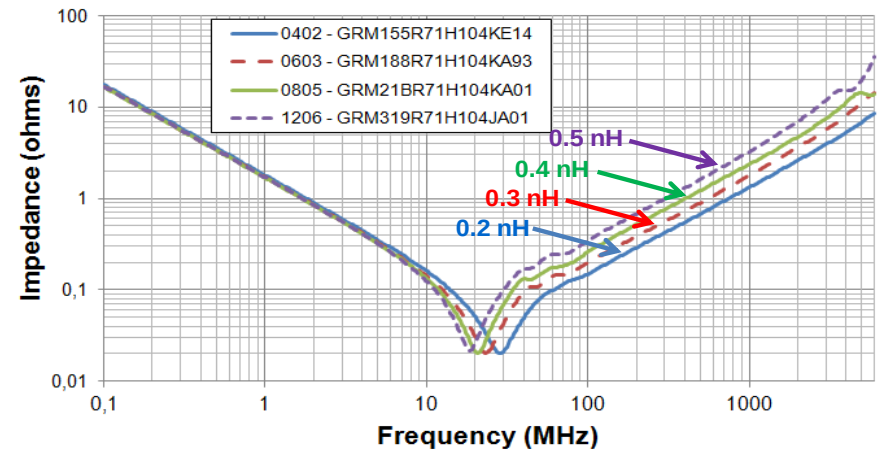
Power Distribution Network of PCB

► Role of decoupling capacitors

- ✓ Local charge tank to respond rapidly to current peak produced by ICs.
- ✓ According to the targeted frequency range, different technologies/packages are used.
- ✓ The complex impedance $Z(f)$ is the most important indicator → $Z(f)$ small = efficient filtering !



Typical $Z(f)$ for bulk capacitors (100 µF)

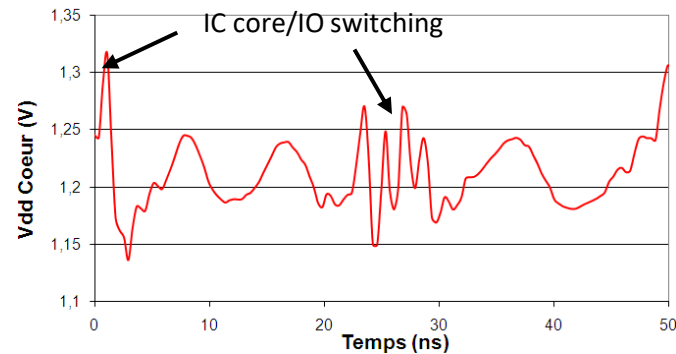
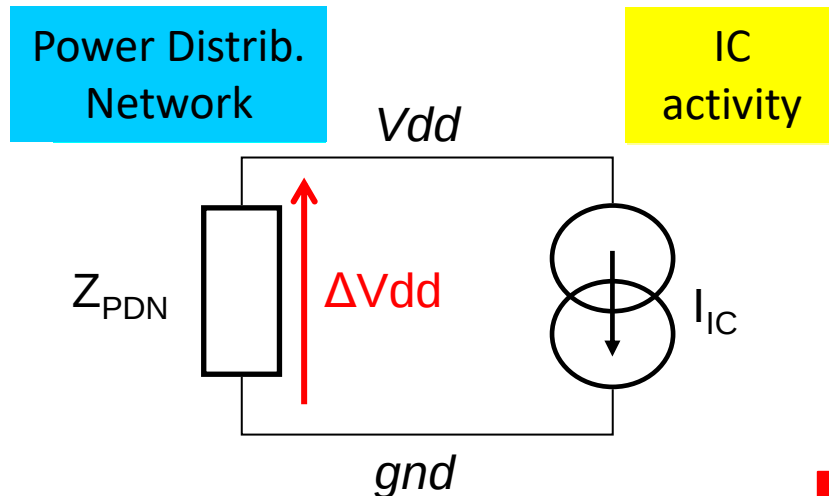


Typical $Z(f)$ for MLCC (100 nF) mounted in different SMD cases

Power Integrity

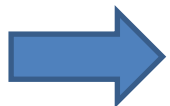
► Power integrity (PI) issues

- ✓ Small signal HF equivalent circuit of a digital IC mounted on a multilayer PCB – simulation of power supply voltage fluctuations due to IC transient current.



$$\Delta V_{dd}(f) = (Z_{PDN}(f)) \times I_{IC}(f)$$

Ensuring power integrity = keep power supply/ground voltage ΔV_{dd} stable whatever the IC activity



To ensure PI, the PCB designer must guarantee a sufficiently low PDN impedance over the frequency range covered by IC transient activity: VRM module (LF), Bulk capacitors (LF), MLCC (MF, HF) Power/ground planes design (HF).

Power Integrity

► Concept of target impedance Z_{Target}

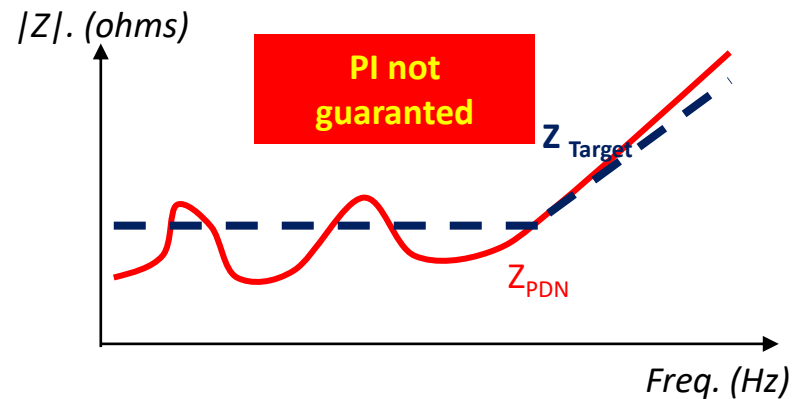
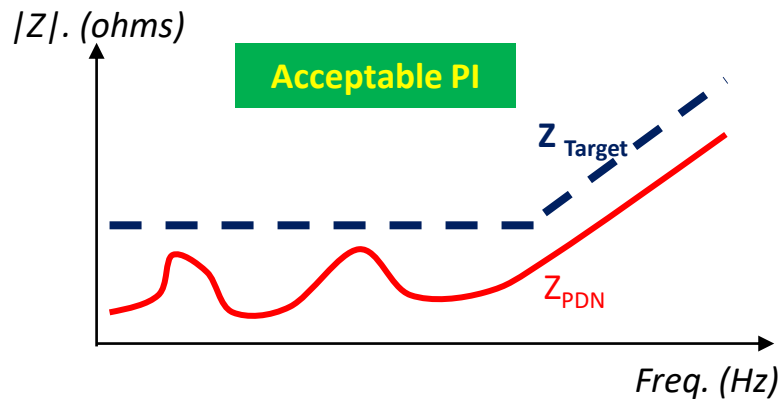
- ✓ Efficient figure of merit to ensure if a PDN design will ensure a sufficient PI.

$$Z_{Target} = \frac{\Delta V_{dd_{max}}}{I_{avg}}$$

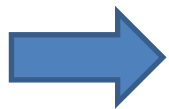
Max. allowed power supply voltage fluctuation (e.g. 5/10 % of Vdd)

Average current consumption (may be frequency-dependent)

- ✓ Design target:



$$Z_{PDN}(f) < Z_{Target}(f) \quad \forall f \in [f_{min} ; f_{max}]$$

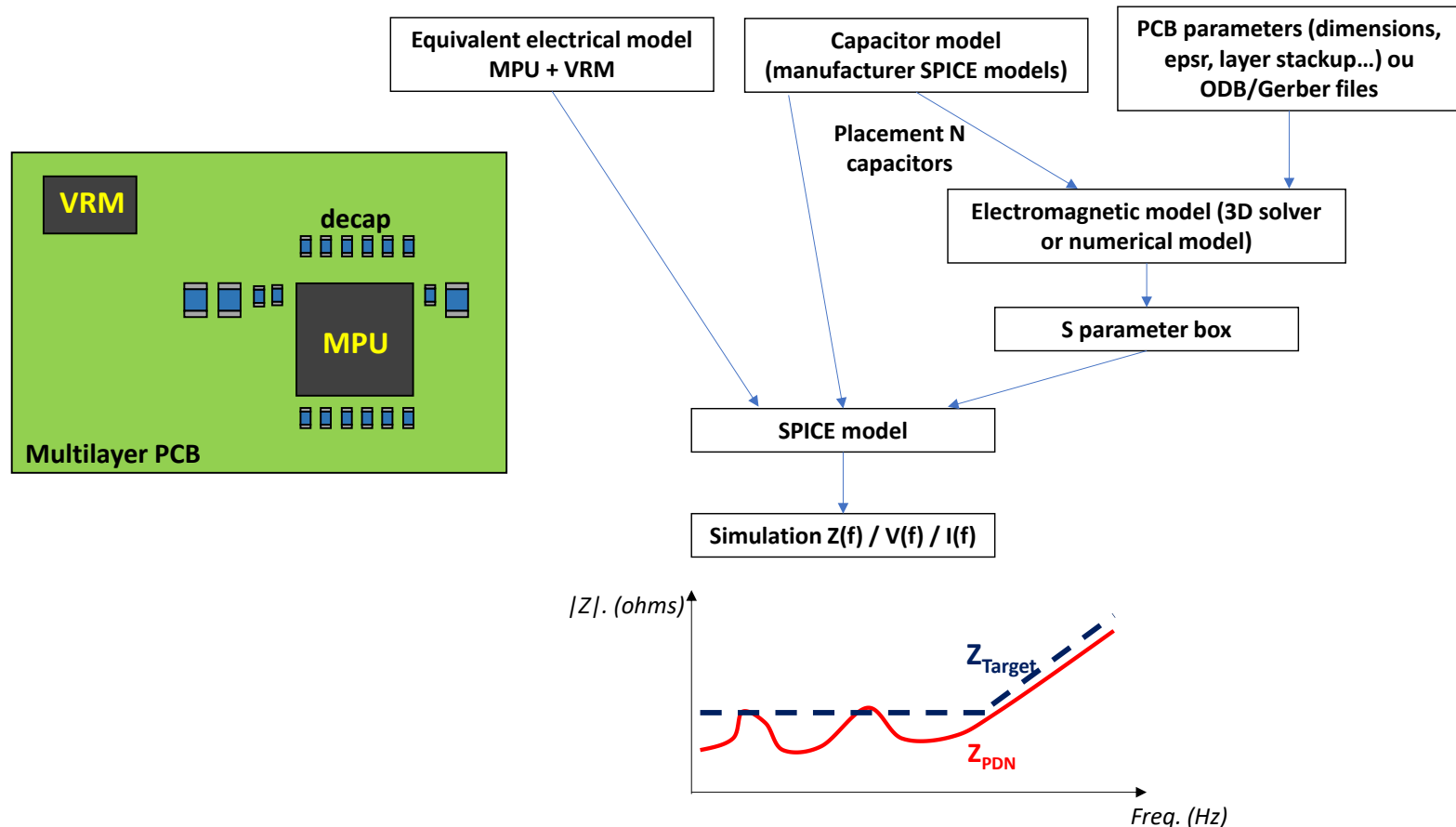


Select capacitors and placement + power/gnd plane pairs to ensure $Z_{PDN} < Z_{target}$.

Power Integrity

Typical PI simulation flow

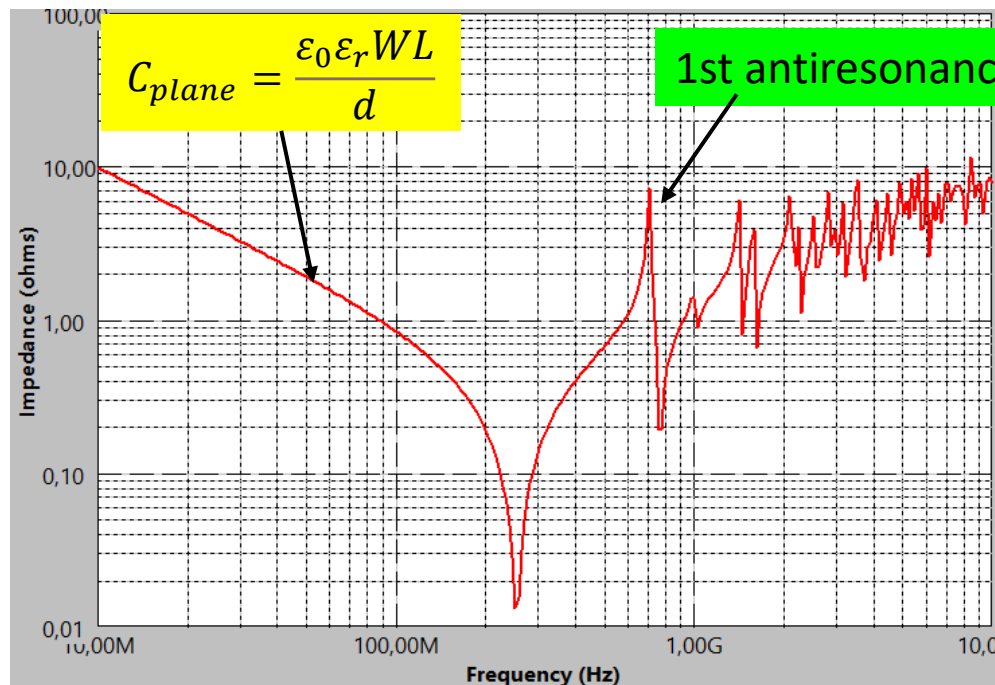
- ✓ For complex multilayer PCB with huge digital ICs (large transient signal), PI must be predicted to validate PDN design before fabrication.



Power-ground plane response

► Rectangular power-ground plane pair model

- ✓ A $W \times L$ rectangular power-ground plane pair separated by a short distance d forms a rectangular resonant cavity.
- ✓ If electrically short, equivalent to a plane capacitor.
- ✓ If electrically long, equivalent to a 2D transmission line terminated by open ends
- ✓ Response characterized by numerous resonant modes.

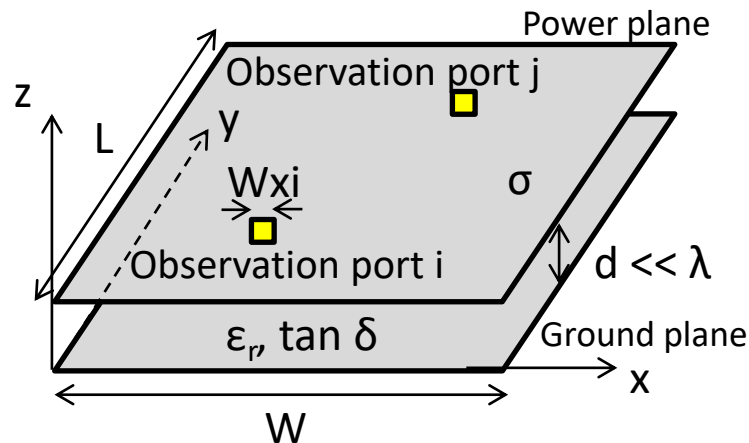


Simulated $Z(f)$ of a power-ground plane pair ($W=L=100$ mm, $d = 0.25$ mm, $\epsilon_r = 4.5$)

Power-ground plane response

► Rectangular cavity model

- ✓ For rectangular plane pairs without any aperture, $Z(f)$ can be determined analytically from Maxwell's equation resolution:



$$Z_{PGij}(\omega) = \sum_{n=0}^{\infty} \sum_{m=0}^{\infty} \frac{N'_{mni} N'_{mni}}{\frac{1}{j\omega L_{mn}} + j\omega C_{mn} + G_{mn}}$$

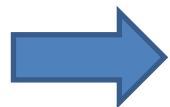
With:

$$N_{mni} = \cos\left(\frac{m\pi x_i}{W}\right) \cos\left(\frac{n\pi y_i}{L}\right) \sin c\left(\frac{m\pi W_{xi}}{2W}\right) \sin c\left(\frac{n\pi W_{yi}}{2L}\right)$$

$$L_{mn} = \frac{d}{2\pi F_{Cmn} W L \epsilon_0 \epsilon_r} \quad C_{mn} = \frac{W L \epsilon_0 \epsilon_r}{d} \quad N'_{mni} = C_m C_n N_{mni}$$

$$G_{mn} = \frac{W L \epsilon_0 \epsilon_r}{d} 2\pi F_{Cmn} \left(\tan \delta + \frac{1}{d} \frac{1}{\sqrt{\pi F_{Cmn} \mu \sigma}} \right)$$

$$C_m, C_n = \begin{cases} 1 & \text{if } m, n = 0 \\ \sqrt{2} & \text{otherwise} \end{cases}$$



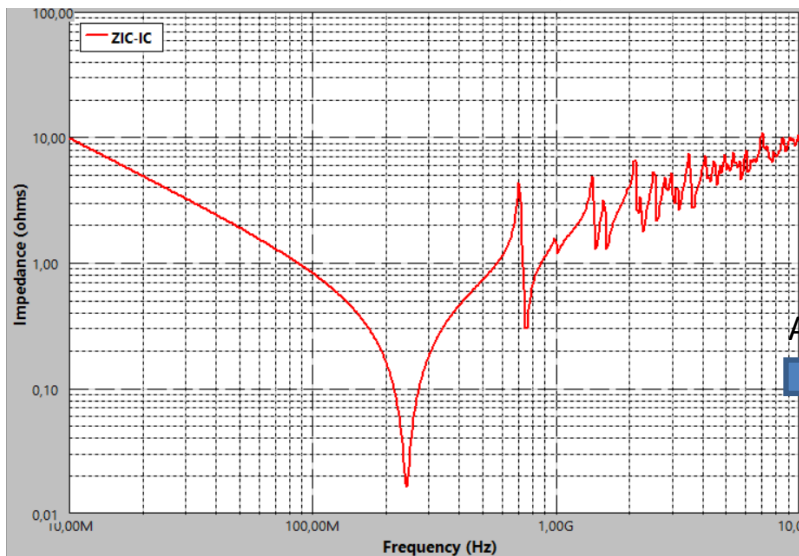
Can be estimated numerically by summing a finite number of resonant modes.

Power-ground plane response

► Rectangular cavity model

✓ Example: $W=L=100$ mm, $d = 0.25$ mm, $\epsilon_r = 4.5$

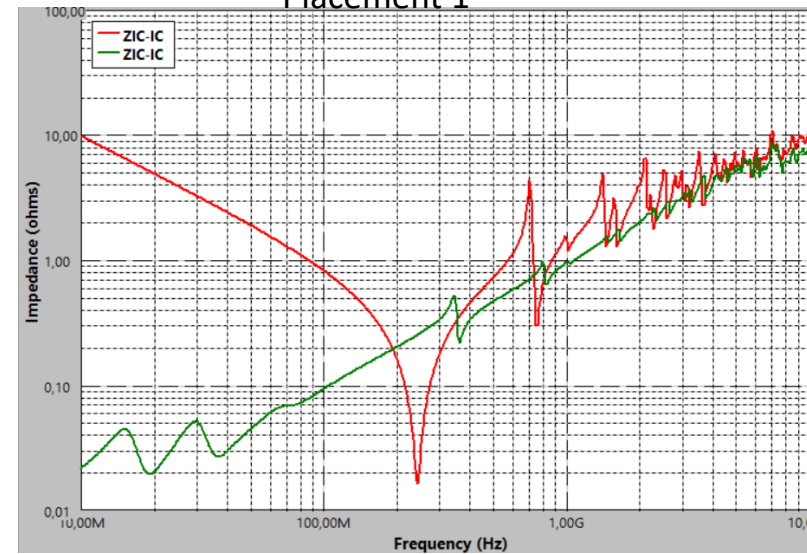
Plane pair + IC + VRM (no decap)



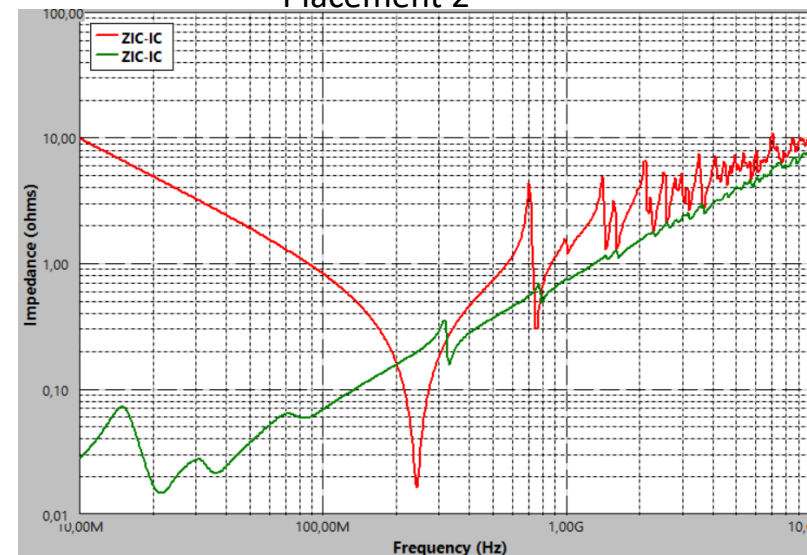
Add 8 capacitors



Placement 1



Placement 2



Enoncé du TP – Optimisation du réseau de condensateurs de découplage d'un circuit imprimé intégrant un SoC

I. Contexte de l'étude et cahier des charges

Ce sujet de TP s'inscrit dans le processus de conception d'un circuit imprimé intégrant un circuit intégré complexe (un System-on-Chip multicoeur et un FPGA) et dédié à une application critique. La maîtrise de l'intégrité d'alimentation (*power integrity*), de l'intégrité du signal et de l'émission rayonnée est une contrainte majeure pour cette application. Pour garantir ces différentes contraintes, une liste ou budget de condensateurs de découplage conséquent est préconisé, basé sur des recommandations du constructeur et sur l'application d'une marge de sécurité. Cependant, le nombre élevé de condensateurs rend le placement-routage très difficile. De plus, de nombreux experts ont jugé ce budget de découplage excessif et estime qu'il peut être réduit.

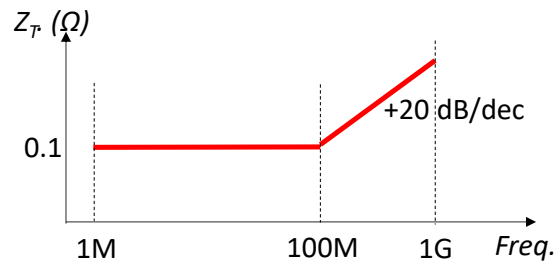
Afin d'éviter un cycle long et coûteux d'essai-erreur basé sur la réalisation de plusieurs prototypes, la mise en place d'une méthode basée sur la modélisation et la simulation électrique-électromagnétique est envisagée. Celle-ci doit fournir une approche objective pour optimiser le budget, qui pourra être réutilisée pour de futures conceptions.

Vous êtes donc en charge d'une étude visant à mettre au point cette méthode, qui permettra de valider la simplification du budget de découplage et proposer un placement adéquat des condensateurs. Dans cette étude, nous focaliserons uniquement sur le découplage prévu pour l'alimentation du cœur digital du circuit (d'autres condensateurs de découplage participent au découplage des GPIO, de la PLL interne, ...). Le tableau ci-dessous donne quelques caractéristiques du circuit digital (boîtier, tension et courant consommé par le cœur digital).

Boîtier du circuit intégré	BGA 784 broches
Taille du boîtier	23 × 23 mm
Pitch boîtier	0.8 mm
Nombre GPIO	460
Nombre paires de broches Vdd-Vss cœur	23
Tension d'alimentation du cœur	1.05 V
Courant moyen du cœur	550 mA

Caractéristiques du circuit intégré étudié

Pour garantir une stabilité de la tension de cœur délivré au circuit (une fluctuation inférieure à 5 % de la tension d'alimentation), l'impédance cible Z_T décrite sur la figure ci-dessous a été définie. Elle est spécifiée uniquement entre 1 MHz et 1 GHz, bande de fréquence sur laquelle le cœur digital produit un bruit de commutation significatif. L'impédance présentée au circuit entre les plans d'alimentation Vdd et de masse Vss du circuit imprimé doit donc respecter la limite définie par Z_T . Afin de tenir compte des incertitudes sur les composants et leur placement, sur le circuit imprimé, on analysera les résultats en intégrant une marge de 10 % sur l'impédance calculée.

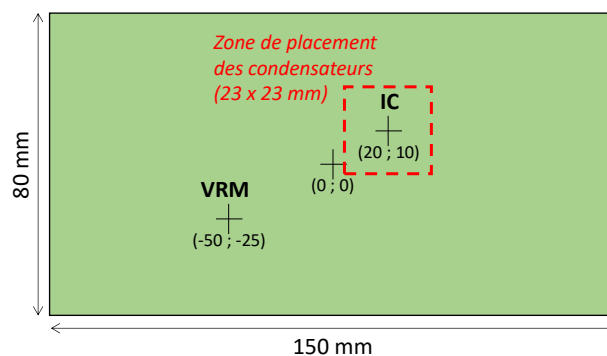


Le circuit imprimé prévu est de forme rectangulaire, dont les caractéristiques sont données dans le tableau ci-dessous. Il s'agit d'une carte multicouche, pour lequel plusieurs couches sont prévues pour le routage des références d'alimentation et des masses. Dans cette étude, nous considérerons qu'une couche complète est attribuée à Vdd et Vss, formant ainsi des plans complets.

Dimensions du circuit imprimé	150 × 80 mm
Espacement entre les plans Vdd et Vss	0.4 mm
Substrat	Epoxy FR4
Permittivité électrique relative ϵ_r	4.5
Pertes substrat ($\tan \delta$)	0.02

Caractéristiques du circuit imprimé étudié

L'alimentation Vdd est fournie par un régulateur monté sur la carte, appelée VRM. Les positions sur la carte du régulateur et du circuit intégré sont précisées sur le schéma ci-dessous. Ces deux composants seront montés en top. Dans l'étude, on supposera que la position de ces deux composants est assimilable à deux points, précisés sur le schéma. Les condensateurs de découplage seront montés en bottom, dans l'aire délimitée sur la figure ci-dessous.



Dimensions de la carte et position des composants

Le tableau ci-dessous décrit le nouveau budget de condensateurs de découplage proposé et validé par un expert.

Noms	Valeur C (F)	Boitier et dimensions	ESR (mΩ)	ESL (nH)
C1	47 μ	1206 - 3.2 × 1.6 mm	3	0.6
C2, C3	4.7 μ	0805 - 2.0 × 1.2 mm	6	0.35
C4, C5, C6	100 n	0402 - 1.0 × 0.5 mm	20	0.25
C7, C8	10 n	0201 - 0.6 × 0.3 mm	60	0.2

Budget de découplage prévu

Votre mission consiste donc à proposer un placement adéquat pour ces 8 condensateurs. En outre, votre étude doit permettre de vérifier si ce budget peut être simplifié. Pour cela, vous devez proposer une méthodologie d'optimisation du placement des condensateurs. Celle-ci s'appuiera sur l'outil de simulation « PG Plane Model » du logiciel IC-EMC (voir document [Prise_en_main_PG_plane_tool.pdf](#)), piloté par un programme Python que vous développerez. Les liens pour télécharger l'outil et les librairies sont donnés dans la partie V de ce rapport.

Trois méthodes d'optimisation prometteuses ont été identifiées (décrites succinctement dans le document [Presentation_methodes_optimisation_TP_ML.pdf](#) :

1. Evolution différentielle (librairie Scipy)
2. Optimisation par essais particuliers ou PSO (librairie Scikit_opt)
3. Optimisation bayésienne basée sur du krigeage (librairie Scikit-Optimize)

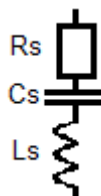
Vous en testerez au minimum deux (préférentiellement les méthodes 1 et 3 ou 2 et 3) et vous comparerez les résultats obtenus (ils ne donneront sans doute pas les mêmes résultats, mais des tendances proches pourront ressortir) et les performances des méthodes d'optimisation. Si vous identifiez que le budget de découplage peut être simplifié, vous indiquerez quels condensateurs pourraient être supprimés.

Question bonus : si le temps vous le permet, l'étude pourrait être relancée dans le cas d'une carte présentant plus de couches, permettant de réduire la distance de séparation entre les plans Vdd et Vss à 0.2 mm. Dans cette nouvelle configuration, le budget de découplage peut-il être simplifié ?

II. Hypothèses de travail

La modélisation électromagnétique et électrique complète du circuit imprimé et des composants qui y seront montés s'avère longue et complexe. Afin de conserver des temps de simulation acceptable, un modèle suffisamment réaliste et faciliter le développement de l'algorithme d'optimisation, les hypothèses suivantes sont faites :

- Les plans d'alimentation et de masse sont supposés complets, sans trous et occupent toute la couche sur laquelle ils sont routés.
- Bien que le circuit intégré présente une quarantaine de broches Vdd et Vss réparties sur toute la surface du boîtier, on suppose qu'elles sont localisées en un point situé au centre du circuit intégré.
- Les condensateurs seront supposés carrés. L'interdistance minimale entre les condensateurs ne prendra en compte que les dimensions des condensateurs, pas d'espacement minimal.
- Des modèles haute-fréquence (HF) simplifiés sont considérés pour les condensateurs, décrit par l'image ci-dessous.



- Le modèle du VRM est simplifié. Seul l'effet du condensateur bulk (de 330 μF) placé à sa sortie est considéré. Il est modélisé par un modèle RLC série, avec $R = 75 \text{ m}\Omega$, $L = 1.5 \text{ nH}$, $C = 330 \text{ }\mu\text{F}$.
- Le modèle HF du circuit intégré est simplifié. On considère que cœur digital est équivalent à un modèle RLC série avec $R = 40 \text{ m}\Omega$, $L = 0.7 \text{ nH}$, $C = 30 \text{ nF}$.
- L'optimisation ne portera que sur le placement des condensateurs. Leur nombre et les valeurs (modèles) utilisés seront choisis par l'utilisateur et resteront fixes.
- La position des condensateurs peut prendre n'importe quelle position sous le circuit intégré, en tenant compte des contraintes de non chevauchement entre les condensateurs.

Paramètres conseillés pour l'outil « PG Plane Model » :

- Pendant les phases de recherche d'optimum, vous pouvez réduire le nombre de fréquence et l'ordre.
- La simulation de la réponse de la paire de plan Vdd-Vss et des condensateurs de découplage sera limité à la bande de fréquence 1 MHz – 1 GHz. Vous utiliserez un balayage logarithmique de 30 points par décade. Ce nombre pourra être augmenté une fois un placement identifié, pour accroître la précision du résultat.
- La simulation du modèle de cavité se fera avec un ordre de 100, pour limiter le temps de calcul. Ce nombre pourra être augmenté une fois un placement identifié, pour accroître la précision du résultat.

Le fichier de configuration **CasTest_8decap.pgconf** vous est proposé. En l'important dans l'outil « PG Plane Model », la géométrie, les paramètres des circuits et des condensateurs ainsi que les paramètres du simulateur seront automatiquement configurés.

III. Méthodologie suggérée

Le problème traité s'avère complexe et sa résolution nécessite un grand nombre d'itérations de simulations potentiellement longue. Le temps de simulation peut atteindre plusieurs dizaines de minutes (selon les paramètres de la recherche et la machine employée). Il est donc indispensable de valider au préalable le bon fonctionnement de l'algorithme de recherche d'optimum avant de l'appliquer au problème complet. Il est donc conseillé de :

- Construire et coder l'algorithme
- Valider l'algorithme sur un problème plus simple (par exemple en limitant le nombre de variables à optimiser (e nombre de condensateurs), en limitant le nombre de fréquence et l'ordre) et en réduisant le nombre d'itérations. A ce stade, le but n'est pas de trouver le résultat, mais de vérifier que l'algorithme et la méthode d'optimisation fonctionne bien.
- Une fois celui-ci vérifié, vous pouvez progressivement augmenter la complexité du problème ainsi que le nombre d'itérations afin de voir comment évolue le temps de calcul. Par exemple, le fichier de configuration **CasTest_4decap.pgconf** est proposé. Celui-ci configurera le simulateur pour le placement de 4 condensateurs de découplage, ce qui réduira le temps de calcul.
- Un fois votre algorithme de recherche prêt et les paramètres de recherche correctement identifiés, vous pouvez lancer l'optimisation sur le problème complet. N'hésitez pas à le

lancer à un moment où vous n'avez pas besoin d'utiliser votre machine, par exemple entre les séances de TP, le soir ... pour ne pas gaspiller les séances de TP à attendre l'issue du calcul.

IV. Attendus du rapport

Votre travail sera évalué sur la base d'un rapport. Le rapport devra, de manière synthétique (10 pages max.) :

- Expliquer le contexte et l'objectif de l'étude
- Rappeler brièvement le principe de fonctionnement des méthodes d'optimisation employées et décrire le fonctionnement de votre algorithme
- Préciser la ou les fonctions de coût utilisée(s)
- Comparer les résultats des différentes méthodes (notamment, leur convergence, temps de calcul, les meilleures impédances cibles, les placements proposés ...)
- Les hyperparamètres des algorithmes devront être proposés (pour garantir qu'un utilisateur futur pourra reproduire vos résultats)
- Proposer un nombre de condensateurs de découplage optimal et un placement adéquat.

Il convient de privilégier une présentation synthétique (schéma bloc pour les algorithmes, tableaux comparatifs) et graphique des résultats.

V. Installation des logiciels de simulations et des librairies

Le logiciel de saisie de schématique et d'analyse des résultats, IC-EMC v2.9, est téléchargeable à l'adresse suivante : www.ic-emc.org/download/IC-EMC-2v9.zip.

Dézipper les deux dossiers dans un emplacement connu. Le lancement du logiciel IC-EMC se fait en lançant l'exécutable `ic_emc.exe` . L'outil PG Plane Model se lance à partir du menu Tools > PG Plane Model. La version console de l'outil PG Plane Model (`PGPlane_Calculator.exe`), qui sera pilotée par vos scripts Python, est téléchargeable sur la page Moodle de l'enseignement Machine learning pour la conception électronique (dossier zippé `PGPlane_Calculator_Fichiers_config.zip`).

Remarques :

- Ces deux outils ne fonctionnant que sous Windows, vous devrez exécuter vos scripts sur Windows.
- Eviter d'installer ces outils dans un répertoire avec un chemin d'accès contenant des espaces ou des caractères spéciaux.

Vous ferez tourner vos scripts sur vos machines personnels (fonctionnant sur Windows). Vous aurez besoin de réaliser les installations suivantes :

- la distribution recommandée pour ce TP est Anaconda (www.anaconda.com/download). Une fois installé, vous pourrez vérifier les packages installés à l'aide d'Anaconda Prompt en tapant « `conda list` ».

- l'éditeur Python recommandée est Spyder (il est fourni directement dans la distribution Anaconda)
- les méthodes d'optimisation testées sont fournies dans les librairies suivantes :
 - Scipy (pour la méthode d'évolution différentielle) → présent dans la distribution Anaconda
 - Scikit-opt (sko) (pour la méthode PSO) → installer le package avec la commande `pip install scikit-opt` depuis Spyder.
 - Scikit-optimize (skopt) (pour la méthode d'optimisation bayésienne) → installer le package avec la commande `pip install scikit-optimize` depuis Spyder.

INSTITUT NATIONAL DES SCIENCES
APPLIQUEES DE TOULOUSE

5^{ème} Année SEE

**TECHNIQUES POUR
L'OPTIMISATION DE LA COMMANDE
ELECTRONIQUE**

**MACHINE LEARNING POUR LA
CONCEPTION ELECTRONIQUE**

**PRESENTATION DES METHODES
D'OPTIMISATION UTILISEES DURANT
LE TP**

Alexandre Boyer
alexandre.boyer@insa-toulouse.fr
www.alexandre-boyer.fr

Septembre 2025

Le TP de Machine learning pour la conception électronique vise à mettre en place un flot typique de simulation utilisant des méthodes de machine learning pour optimiser un problème de conception électronique complexe. Il s'appuie sur un cas d'étude, dédié à l'optimisation du placement de condensateurs de découplage sur un circuit imprimé afin de garantir une intégrité de puissance acceptable. Le but de cet enseignement n'a pas vocation à tester et comparer l'ensemble des méthodes de machine learning existante, mais plutôt d'en introduire plusieurs, qui sont plutôt populaires et adaptées au problème traité (ici, un problème d'optimisation). Dans ce TP, nous avons sélectionné trois approches différentes :

- Evolution différentielle
- Optimisation par essaims particuliers (*Particle Swarm Optimization*)
- Optimisation Bayésienne basée sur une régression par processus gaussien (krigeage)

Il est important de noter que les deux premières ne sont pas vraiment classées comme des méthodes de machine learning, mais sont plutôt des métaheuristiques d'optimisation. La dernière est une méthode d'optimisation basée sur une méthode de machine learning supervisé. Elles appartiennent cependant au domaine de l'intelligence artificielle et ont en commun de reposer sur des algorithmes dédiés à la résolution de problèmes complexes conçus sans instructions explicites sur les actions à mener et fonctionnant de manière automatique et sans intervention humaine.

Ce document vise à :

- donner la vue d'ensemble du flot d'optimisation utilisé pendant ce TP
- décrire succinctement le principe des méthodes (métaheuristiques et machine learning) utilisées durant ce TP, ainsi que leur paramétrage
- donner des exemples d'utilisation simples basés sur des bibliothèques Python open-source (les codes Python sont disponibles sur Moodle).

I. Flot d'optimisation utilisé

Le problème traité durant ce TP s'apparente à un problème d'optimisation non-linéaire contraint. En raison du nombre élevé de variables à optimiser (plusieurs dizaines) et du grand nombre d'optimum locaux, les algorithmes d'optimisation traditionnels reposant sur une descente de gradient ne sont pas adaptés. De plus, le problème traité nécessite l'utilisation d'un outil de simulation numérique dédié, potentiellement gourmand en temps de calcul. Des méta-heuristiques ou des méthodes d'optimisation basées sur l'apprentissage d'un métamodèle sont donc requis. Les méthodes mises en œuvre dans ce TP sont décrites dans la prochaine partie.

Quelles que soient la méthode utilisée, le flot d'optimisation restera le même. Son principe est illustré sur la Figure 1. Au préalable, l'**espace des solutions** (plage des valeurs pour les différentes variables à optimiser) et les **contraintes du problème** doivent être définis. Une fois l'algorithme d'optimisation ou d'apprentissage choisi, ses hyperparamètres doivent être réglés. Ces derniers vont influencer sur la convergence et la rapidité de la convergence. L'algorithme d'optimisation/d'apprentissage va chercher une combinaison de variables d'entrée à l'intérieur de l'espace des solutions correspondant au minimum d'une **fonction dite de cout**, qui correspond à la fonction à minimiser. Celle-ci doit être choisie judicieusement. La manière dont se fait cette recherche est pilotée par l'algorithme d'optimisation/d'apprentissage. Durant ce TP, nous utiliserons des fonctions existantes et fournies par des bibliothèques Python open-source.

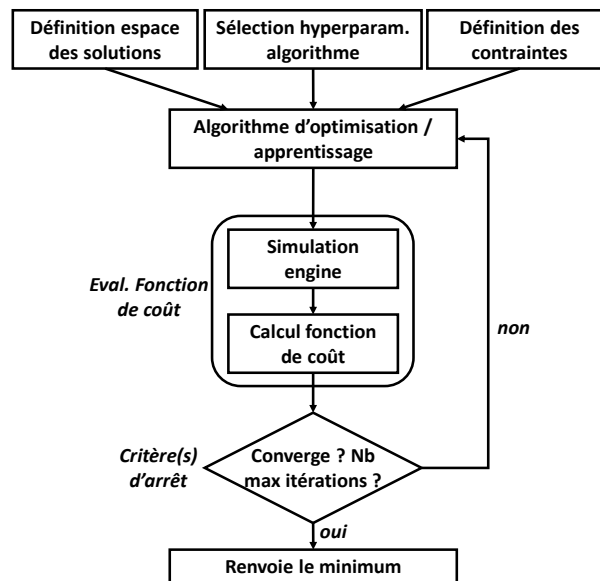


Figure 1 – Principe du flot d'optimisation mis en œuvre durant ce TP

La recherche de l'optimum est itérative. L'algorithme arrête la recherche selon un critère d'arrêt (lié à la convergence de la recherche et/ou un nombre maximal d'itérations). A chaque itération, l'algorithme fait appel à une autre fonction qui évalue la fonction de coût. A l'issue de cette recherche, la solution identifiée sera la combinaison de variables d'entrée qui minimise la fonction de coût tout en satisfaisant aux contraintes du problème.

La fonction de coût peut être simple à évaluer et directement calculée par le programme Python qui gère l'algorithme de recherche d'optimum. Dans ce TP, nous considérons un problème de conception électronique s'appuyant sur des outils de simulation numérique complexes et propriétaires. Pour l'utilisateur, il s'agit d'une « boîte noire » dont le fonctionnement interne est inaccessible. L'évaluation de la fonction de coût passe donc par un appel à cet outil de simulation externe, que nous appellerons **simulation engine**. L'interfaçage avec cet outil se fait via des fichiers d'entrée et de sortie, voire des fonctions dans le cas où l'outil de simulation propose une interface de pilotage.

Il est important de noter que le lancement du *simulation engine* engendre un temps d'exécution parfois non négligeable. Une manière de réduire ce temps d'exécution est l'utilisation d'un mode dit de **vectorisation**. Plutôt que d'envoyer une combinaison de variables d'entrée au *simulation engine*, ce mode consiste à envoyer N combinaisons de variables d'entrée et de collecter les résultats pour ces différentes combinaisons. Ce mode permet donc de diviser par N le temps lié au lancement du *simulation engine*. Cependant, tous les algorithmes d'optimisation/d'apprentissage ne s'y prêtent pas forcément. Les métaheuristiques évolutionnaires et PSO supportent ce mode, mais pas les méthodes de régression par krigeage.

Remarque : rien ne garantit à l'issue de l'exécution de ce flot que l'algorithme à identifier le minimum global du problème. Dans tous les cas, seuls l'utilisateur sera capable d'apprécier la pertinence de(s) la solution identifiée par l'algorithme.

II. Description succincte des méthodes utilisées durant le TP

1. Evolution différentielle (DE)

a. Principes

L'évolution différentielle est un algorithme évolutionnaire, dérivée de l'algorithme génétique, qui s'inspire de la sélection naturelle et de l'évolution pour identifier des solutions aux problèmes d'optimisation, plutôt que se baser sur le gradient de la fonction à optimiser. Ces méthodes sont aussi appelées métaheuristiques car elles ne font aucune hypothèse sur le problème à optimiser (considéré comme une boîte noire) et sont capables de faire une recherche de solution sur un large espace de solutions. Comparé à l'algorithme génétique, l'algorithme DE propose une approche simplifiée, faisant moins appel à des analogies avec la génétique, ne s'applique qu'à des valeurs réelles et s'avère généralement plus performante.

Le DE optimise un problème de manière stochastique, en conservant une population de solutions candidates et en créant de nouvelles en combinant celles qui existent déjà, puis en conservant la solution candidate qui minimise la fonction de coût. Différentes stratégies existent pour créer les nouveaux candidats : celles que nous utiliserons durant ce TP est basée sur la construction du nouveau candidat à partir du meilleur candidat de la génération précédente.

Pour formaliser la présentation de l'algorithme, soit $x \in \mathbb{R}^n$ une solution candidate appartenant à une population de candidats. Un candidat est donc un vecteur formé de n valeurs, où n est le nombre de variables à optimiser. Nous cherchons la solution qui minimise la fonction à valeurs réelles $f(x)$, la fonction de coût. L'algorithme est défini par 3 hyperparamètres :

- NP : la taille de la population ou nombre d'individus (par exemple, $10 \times n$)
- CR : la probabilité de mutation ou crossover probability ou recombination constant, compris entre 0 et 1.
- F : le coefficient de mutation ou differential weight, compris entre 0 et 2.

Initialisation : les NP individus sont initialisés aléatoirement sur l'espace des solutions. Leur fonction de coût est évaluée.

Boucle : tant que le critère d'arrêt n'est pas atteint :

1. Chaque individu x_{old} de la population va évoluer de la manière suivante :

$$x' = x_{best} + F \times (x_{rand0} - x_{rand1})$$

Où le vecteur x' contient les prochaines valeurs probables de l'individu, x_{best} le meilleur candidat de la dernière génération (celui qui minimise la fonction de coût), x_{rand0} et x_{rand1} sont deux candidats de la dernière génération sélectionnés aléatoirement.

2. Pour chaque entrée $n^o i$ du vecteur x ($i \in [1; n]$), on tire un nombre aléatoire r_i compris entre 0 et 1. Si $r_i < CR$, alors l'entrée $n^o i$ du prochain candidat x_{new} prend l'entrée $n^o i$ du vecteur x' . Sinon, il conserve l'ancienne valeur (l'entrée $n^o i$ du candidat x_{old}).

3. On calcule la fonction de coût des nouveaux individus x_{new} de la population.

4. Si $f(x_{new}) < f(x_{old})$, alors x_{new} remplace x_{old} .

Fin : l'algorithme renvoie l'individu présentant la fonction de coût la plus faible.

Il n'y a pas de choix à priori optimaux pour les hyperparamètres, mais il est possible d'analyser leur effet. Augmenter la taille de la population permet d'accroître les chances de trouver l'optimum global, au prix d'un temps de calcul plus long. Augmenter le coefficient de mutation F accroît le rayon de la recherche mais au prix d'une convergence plus lente. Même chose si on augmente la probabilité de mutation CR . De plus, de très fortes valeurs de CR et F peuvent induire une instabilité dans la

recherche de solutions. Jouer sur ces deux paramètres revient à favoriser l'exploration ou l'exploitation de l'espace des solutions.

b. Fonction `Scipy.Differential_evolution`

Durant ce TP, nous utiliserons la fonction `differential_evolution` de la librairie Scipy pour mettre en œuvre l'algorithme de DE. Cette librairie est directement intégrée dans la distribution Anaconda. Une description détaillée de la fonction peut être trouvée ici : https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.differential_evolution.html

Voici une description succincte des principaux arguments d'entrée de cette fonction :

`differential_evolution(func, bounds, args=(), strategy='best1bin', maxiter=1000, popsize=15, tol=0.01, mutation=(0.5, 1), recombination=0.7, rng=None, callback=None, disp=False, polish=True, init='latinhypercube', atol=0, updating='deferred', workers=1, constraints=(), x0=None, *, integrality=None, vectorized=False, seed=None)`

- `func` : la fonction appelée pour lancer l'évaluation de la fonction de coût.
- `bounds` : vecteur donnant les bornes de l'espace des solutions.
- `strategy` : la stratégie d'évolution différentielle mise en œuvre. On pourra conserver la stratégie 'best1bin'.
- `maxiter` : le nombre maximal d'itérations ou de générations. Si le critère de tolérance n'est pas atteint, l'algorithme sera exécuté `Maxiter+1` fois.
- `popsize` : définit la taille de la population (NP) = `popsize` × nombre de variables à optimiser.
- `tol` : le critère d'arrêt basé sur la tolérance relative pour la convergence
- `atol` : le critère d'arrêt basé sur la tolérance absolue pour la convergence
- `mutation` : les valeurs min et max du coefficient de mutation F. La fonction peut changer aléatoirement cette valeur.
- `recombination` : la probabilité de mutation CR.
- `polish` : à l'issue des `maxiter+1` itérations, une optimisation supplémentaire peut être lancée pour affiner le résultat trouvé. Ce n'est pas forcément conseillé dans notre TP, car le temps de calcul peut être ralenti.
- `Init` : définit la manière dont les valeurs initiales des candidats sont échantillonnées. L'échantillonnage de type hypercube latin est un choix pertinent, puisqu'il cherche à maximiser la couverture de l'espace des solutions.
- `constraints` : définition des contraintes linéaires et non-linéaires du problème (voir exemple dans la partie III.1)
- `vectorized` : pour activer le mode de vectorisation.

La fonction renvoie un objet de type `OptimizeResult`, dont les détails sont donnés ici : <https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.OptimizeResult.html#scipy.optimize.OptimizeResult>

2. Optimisation par essais particulaires ou Particle Swarm Optimization (PSO)

a. Principes

L'algorithme d'optimisation par essais particulaires (PSO) est une métaheuristique d'optimisation bio-inspirée, à l'instar de l'algorithme DE. Bien qu'utilisant une terminologie et un algorithme différents, leurs principes restent assez proches. La méthode PSO consiste en une population (ou essaim) de solutions candidates (appelées particules) dont les coordonnées dans un espace à n

dimensions correspondent aux valeurs prises par les n variables à optimiser. A chaque itération, les particules sont déplacées en modifiant leurs positions et leurs vecteurs vitesse (vélocité) selon un loi mathématique simple. Le mouvement de chaque particule est non seulement influencé par la meilleure position qu'elle a pu rencontrer (celle qui minimise la fonction de coût), mais aussi par les meilleures positions rencontrées par les autres particules de l'essaim. Cela garantit que les particules convergent toutes vers un optimum (qui est espéré global, même si rien ne le garantit). Cette méthode s'inspire ainsi de vols d'oiseaux ou de bancs de poissons pour localiser la nourriture, où chaque individu exploite ce qu'il a appris via son mouvement local (via un paramètre cognitif) et ce qu'il perçoit des autres individus (via un paramètre social). Dès qu'un individu localise une source de nourriture, les autres vont chercher à reproduire son comportement. Ainsi, en jouant sur ces paramètres cognitif et social, les individus vont pouvoir chercher efficacement la solution dans l'espace de recherche.

Pour formaliser la présentation de l'algorithme, soit $x \in \mathbb{R}^n$ une solution candidate appartenant à une population de particules. Une particule est donc un vecteur formé de n valeurs, où n est le nombre de variables à optimiser. Nous cherchons la solution qui minimise la fonction à valeurs réelles $f(x)$, la fonction de coût. L'algorithme est défini par 4 hyperparamètres :

- NP : la taille de l'essaim ou nombre de particules (par exemple, $10 \times n$)
- w : est le facteur d'inertie (généralement compris entre 0.8 et 1.2)
- C1: le paramètre cognitif.
- C2 : le paramètre social. Il est préférable d'avoir $C1+C2 < 4$.

Chaque particule, repéré par un indice i , conserve la mémoire de sa meilleure position P_{best_i} .

Initialisation : les positions $P_i(0)$ des NP particules sont initialisées aléatoirement sur l'espace des solutions, ainsi que leurs vélocités initiales $V_i(0)$. La meilleure position de chaque particule, P_{best_i} , est initialisée avec $P_i(0)$. On évalue la meilleure position prise de l'essaim g_{best} en évaluant la fonction de coût de chaque particule.

Boucle : tant que le critère d'arrêt n'est pas atteint, on itère ($t \leftarrow t+1$) en modifiant la position et la vélocité de chaque particule :

1. Pour chaque vecteur particule et pour chaque entrée de ce vecteur :
 - On tire aléatoirement entre 0 et 1 deux valeurs notées r_1 et r_2 .
 - On met à jour la vélocité de chaque particule d'indice i selon l'équation suivante :

$$V_i(t+1) = w \times V_i(t) + C_1 r_1 (P_{best_i} - P_i(t)) + C_2 r_2 (g_{best} - P_i(t))$$
 - On met à jour la position de chaque particule d'indice i :

$$P_i(t+1) = P_i(t) + V_i(t+1)$$

2. On met à jour la meilleure position prise par chaque particule d'indice i :

$$\text{Si } f(P_i(t+1)) < f(P_{best_i}) \rightarrow P_{best_i} = P_i(t+1)$$

3. On met à jour la meilleure position prise par l'essaim :

$$\text{Si } f(P_{best_i}) < f(g_{best}) \rightarrow g_{best} = P_{best_i}$$

Fin : l'algorithme renvoie la meilleure position de l'essaim g_{best} .

Là encore, le choix des hyperparamètres influe sur la convergence de la méthode. Le facteur d'inertie définit la capacité d'exploration de chaque particule. Une grande valeur du facteur d'inertie induit une grande amplitude de mouvement et donc une plus grande exploration globale. Une faible valeur induira une exploration plus locale. Une valeur trop importante (> 1.2) risque de compromettre la convergence de l'algorithme. Augmenter $C1$ tend à accroître l'instinct de conservation de chaque particule, tandis qu'augmenter $C2$ tend à inciter les particules à suivre les particules voisines. Ainsi,

augmenter C1 et diminuer C2 tend à favoriser l'exploration de l'espace. Inversement, augmenter C2 et diminuer C2 favorise l'exploitation.

b. Fonction skopt.PSO

Durant ce TP, nous utiliserons la fonction PSO de la librairie Scikit_opt (<https://scikit-opt.github.io/>) pour mettre en œuvre l'algorithme PSO. Cette librairie n'est pas nativement présente dans la distribution Anaconda. Vous pouvez l'installer depuis un IDE Python (comme Spyder) à l'aide de la commande suivante : `pip install scikit-opt`.

Une description de la fonction peut être trouvée ici : <https://scikit-opt.github.io/scikit-opt/#/en/README?id=3-psoparticle-swarm-optimization>

Voici une description succincte des principaux arguments d'entrée de cette fonction :

`PSO(func, n_dim, pop=50, max_iter=200, lb, ub, w=0.8, c1=0.5, c2=0.5, constraint_ueq)`

- `func` : la fonction appelée pour lancer l'évaluation de la fonction de coût.
- `n_dim` : la dimension du problème, c'est-à-dire le nombre de variables à optimiser
- `pop` : définit la taille de la population ou de l'essaim
- `maxiter` : le nombre maximal d'itérations ou de générations.
- `lb, ub` : vecteurs donnant les limites basses et hautes de l'espace de recherche
- `w` : le facteur d'inertie.
- `c1` et `c2` : les paramètres cognitif et social.
- `Constraint_ueq` : définition des contraintes non-linéaires du problème (voir exemple dans la partie III.2)

La fonction propose aussi un mode de vectorisation. Pour cela, appelez la fonction suivante :

```
mode = 'vectorization'  
set_run_mode(func, mode)
```

3. Bayesian optimization using Gaussian Processes Regression (Kriging)

a. Principes

L'algorithme d'optimisation bayésienne (BO) est une méthode efficace dès qu'il s'agit d'explorer un problème complexe et coûteux (en temps de calcul). Cet algorithme permet un échantillonnage adaptatif de l'espace des solutions, en positionnant les futurs points là où ils seront le plus susceptibles de réduire l'incertitude du modèle. L'algorithme proposé utilise l'approche de régression par processus gaussien (krigeage) pour établir un métamodèle de la fonction de coût et identifier le prochain point d'échantillonnage selon une fonction d'acquisition. Selon les hyperparamètres sélectionnés, la fonction d'acquisition favorise soit l'exploration (en ajoutant des points où l'incertitude estimée est maximale) ou l'exploitation (en ajoutant des points là où la moyenne estimée est minimale). Le flot typique de l'optimisation bayésienne est décrit sur la figure ci-dessous.

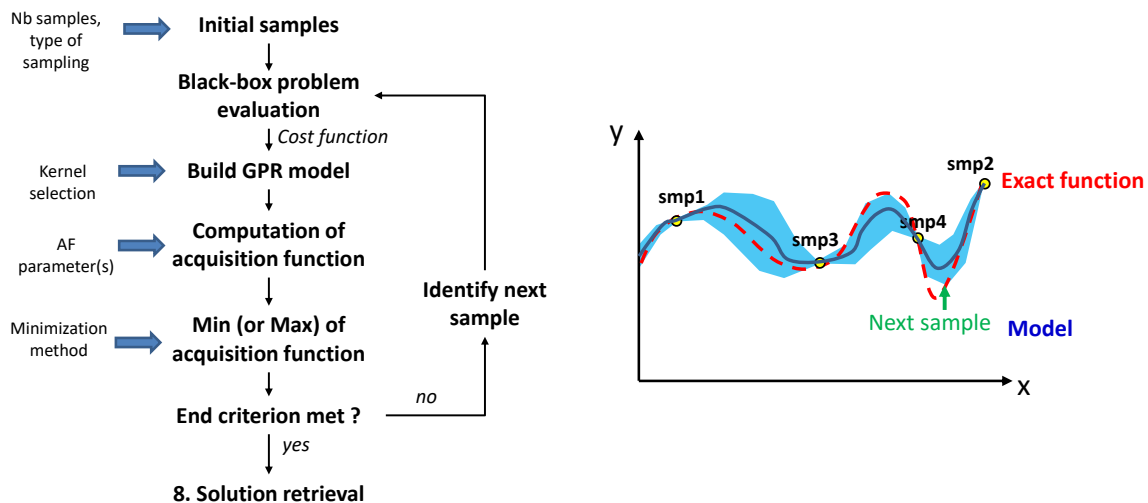


Figure 2 – Flot typique de l’optimisation bayésienne (à gauche) et principe de l’échantillonnage adaptatif (à droite)

Le prochain point d’échantillonnage est sélectionné là où la fonction d’acquisition $a(x)$ est minimale. Plusieurs fonctions d’acquisition courantes existent :

- Lower confidence bound (LCB) : $a(x;\lambda) = m(x) - \lambda \cdot \sigma(x)$, où $m(x)$ et $\sigma(x)$ sont les espérances et les écart-types prédits par le modèle GPR et λ un paramètre choisi par l’utilisateur. Cette fonction d’acquisition est une simple combinaison linéaire entre l’espérance et la variance, pondérée par le terme λ . Un fort λ favorise l’exploration, tandis qu’une faible valeur favorise l’exploitation.
- Probability of Improvement (PI) : comme l’estimateur est un processus gaussien, la probabilité que la valeur en un point x soit plus faible que la valeur aux précédents meilleurs points, il est possible d’établir en tout point la probabilité de trouver un meilleur point. C’est pourquoi on parle d’amélioration. Le prochain point d’échantillonnage est sélectionné là où cette probabilité est maximale. Un hyperparamètre $\zeta > 0$ est introduit pour indiquer quelle amélioration de la PI est attendu. Une forte valeur de ζ favorise l’exploitation.
- Expected Improvement (EI) : elle est similaire à la précédente, hormis que l’espérance mathématique de l’amélioration qu’apporterait un échantillonnage en un point est estimé. Le prochain point d’échantillonnage est sélectionné là où cette espérance est maximale. De même, un hyperparamètre $\zeta > 0$ est introduit pour indiquer quelle amélioration de l’EI est attendu. Une forte valeur de ζ favorise l’exploitation.

b. Fonction `gp_minimize`

Durant ce TP, nous utiliserons la fonction `gp_minimize` de la librairie `Scikit_optimize` (https://scikit-optimize.github.io/stable/modules/generated/skopt.gp_minimize.html) pour mettre en œuvre l’algorithme BO. Cette librairie n’est pas nativement présente dans la distribution Anaconda. Vous pouvez l’installer depuis un IDE Python (comme Spyder) à l’aide de la commande suivante : `pip install scikit-optimize`.

Voici une description succincte des principaux arguments d’entrée de cette fonction :

`gp_minimize(func, bounds, acq_function, kappa, xi, n_calls, n_initial_points, initial_point_generator, acq_optimizer, n_restarts_optimizer, random_state)`

- `func` : la fonction appelée pour lancer l’évaluation de la fonction de coût.
- `bounds` : vecteur donnant les bornes de l’espace des solutions.

- `acq_function` : définit la fonction d'acquisition ("LCB", "EI", "PI""gp_hedge","Elps", "PIps")
- `kappa` et `xi` : les hyperparamètres pour les fonctions d'acquisition LCB et EI/PI respectivement
- `n_calls` : le nombre maximal d'itérations de la fonction `func`
- `n_initial_points` : le nombre d'évaluation initiale de la fonction `func` avant que l'estimation par GPR ne soit lancée
- `initial_point_generator` : définit le type de générateurs pour les points d'évaluation initiale (`n_initial_points`) (« random","sobol","halton","hammersly","lhs")
- `acq_optimizer` : définit la méthode employée pour chercher le minimum de la fonction d'acquisition ("sampling" or "lbfgs")
- `n_restarts_optimizer` : dans le cas où la minimisation de la fonction d'acquisition est effectuée par la méthode L-BFGS, ce paramètre définit le nombre de redémarrage de la méthode. Attention, augmenter ce paramètre ralentit le processus d'optimisation.
- `random_state` : définit la séquence de génération pseudo-aléatoire pour le tirage des points initiaux. En spécifiant une valeur, on garantit un résultat reproductible.

Par rapport aux deux autres méthodes, on remarque qu'on ne peut pas définir de contraintes de type inégalités. En outre, l'approche BO étant basée sur un échantillonnage adaptatif, aucune vectorisation n'est possible.

III. Exemples de mise en œuvre

Afin de prendre en main les fonctions d'optimisation, plusieurs programmes simples sont fournis. Le problème traité est la recherche du minimum de la fonction à 2 dimensions :

$$f(x; y) = \sqrt{\sqrt{(x - x_0)^2 + (y - y_0)^2}}$$

où x_0 et y_0 sont deux paramètres réglables par l'utilisateur et correspondent au minimum de la fonction. Dans les exemples fournis, x_0 et y_0 sont réglés à +1 et -1 respectivement. La Figure 3 présente la surface de réponse de cette fonction.

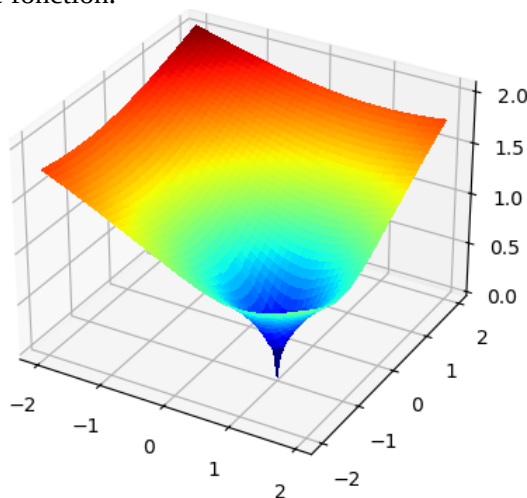


Figure 3 – Fonction à optimiser : le minimum se situe en ($x = +1$; $y = -1$)

Dans ces exemples, le critère d'arrêt est uniquement défini par un nombre maximal d'itérations. L'espace des solutions est borné à $[-2 ; +2]$ pour les 2 variables à optimiser. Une contrainte non-linéaire est ajoutée : $0 < \sqrt{x^2 + y^2} < 2$.

Malgré la simplicité de la fonction à évaluer (qui pourrait être directement évaluée par le programme Python), celle-ci est évaluée par une application .exe externe, afin d'illustrer la manière de piloter le *simulation engine* depuis le programme Python. Pour cela, nous utilisons l'application Calcul_Fonction_Obj.exe, fourni dans le dossier zippé script_prise_en_main.zip. Celle-ci attend les 4 arguments lors de son appel :

Calcul_Fonction_Obj.exe NbCalcul Fichier_individus.txt Fichier_resultats.txt Valeur_x0 Valeur_y0

- Le premier argument (NbCalcul) est un entier qui indique le nombre d'individus passé à l'application, donc le nombre de fois où la fonction devra être évaluée. Si NbCalcul > 1, alors le mode vectorisation est activé. Dans ce cas, NbCalcul = NP, la taille de la population.
- Le second argument (Fichier_individus.txt) passe les coordonnées (x ;y) des NP individus d'une génération. Le fichier est structuré sous le format d'une matrice avec NP lignes et 2 colonnes (x y), les 2 colonnes étant séparées par un espace.
- Le troisième argument (Fichier_resultats.txt) contient les résultats de l'évaluation de la fonction par l'application. Les NP résultats apparaissent sous la forme d'une matrice colonne.
- Les deux derniers arguments donnent les valeurs des termes x_0 et y_0 de la fonction, c'est-à-dire les coordonnées du minimum attendu.

1. Evolution différentielle (Scipy.Differential_evolution)

Le programme exemple est : DE_Fonction_ext_Run_Vectorized.py. Il est configuré avec :

- Une population de $20 \times 2 = 40$ individus
- 20 itérations maximales
- Probabilité de mutation = 0.7
- Coefficient de mutation = 0.5

La contrainte non-linéaire est passée de la manière suivante : définition d'une fonction qui évalue la fonction non-linéaire associée, d'une borne basse et d'une borne haute, qui sont passées à un objet de type NonLinearConstraint (plus de détails ici : <https://docs.scipy.org/doc/scipy-1.15.2/reference/generated/scipy.optimize.NonlinearConstraint.html>).

```
#définition de contraintes nonlinéaires : par ex.  $0 \leq \sqrt{x^2+y^2} \leq 2$ 
NbContraintesNonLin = 1
lowerBoundNonLinearConstraints = 0*np.ones(NbContraintesNonLin)
upperBoundNonLinearConstraints = 2*np.ones(NbContraintesNonLin)
```

```
#déf de la fonction de contraintes non-linéaires.
```

```
def MyNonlinearConstraint(p):
    x, y = p
    if np.size(x) == 1 :
        res = np.sqrt(x**2+y**2)
    else:
        taille_x = np.shape(x)
        #print("Taille x = "+str(taille_x[0]))
        Nb_calcul = taille_x[0]
        res = np.zeros((1,Nb_calcul))
        for ii in range(0,Nb_calcul,1):
            res[0,ii] = np.sqrt(x[ii]**2+y[ii]**2)
    return res
```

```
nonlinearConstraints = NonlinearConstraint(MyNonlinearConstraint, lowerBoundNonLinearConstraints,
upperBoundNonLinearConstraints )
```

Au bout de 9.5 s, l'algorithme identifie la solution suivante, qui est très proche du résultat attendu : [1.000e+00 ; -1.000e+00], avec une fonction de coût minimale de 0.00494807450928616. La figure ci-dessous présente la convergence de l'algorithme et les positions prises par le meilleur individu à chaque itération. On vérifie que l'algorithme converge progressivement vers la bonne solution. Les premiers minima trouvés sont relativement éloignés de la solution, mais se rapproche rapidement de celle-ci.

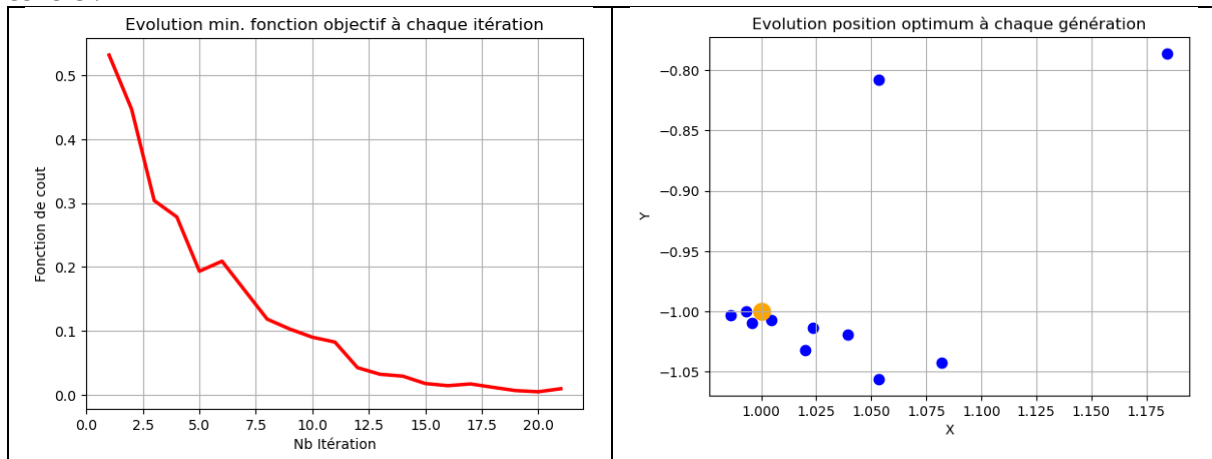


Figure 4 – Résultat algorithme DE : convergence (à gauche) et positions prises par le meilleur individu à chaque itération (en orange, la solution) (à droite).

On remarque aussi que lors des premières itérations, la taille de la population n'est pas exactement de 40, mais peut être comprise entre 20 et 30. Ce n'est plus le cas lors des dernières itérations. Cela vient de la contrainte utilisée ($x^2 + y^2 < 2$). L'algorithme DE de Scipy élimine les points qui ne satisfont pas à ce critère. Quand l'algorithme converge vers la bonne solution, les candidats qu'il génère satisfont de mieux en mieux cette contrainte.

2. Particle Swarm Optimization (skopt.PSO)

Le programme exemple est : PSO_Fonction_ext_Run_Vectorized.py. Il est configuré avec :

- Une population de 30 particules
- 25 itérations maximales
- Un facteur d'inertie = 0.8
- Des paramètres cognitif et social = 0.5

La contrainte non-linéaire est passée de la manière suivante :

```
#définition d'une contrainte d'inégalité :
# sqrt(x²+y²) <= R
#Ca se traduit sous la forme : sqrt(x²+y²) - R <= 0
constraint_ueq = [
    lambda x: np.sqrt(x[0]**2 + x[1]**2)-2
]
```

Au bout de 11 s, l'algorithme identifie la solution suivante, qui est très proche du résultat attendu : [1.00981394 ; -0.9967156], avec une fonction de coût minimale de 0.10172976171989. La figure ci-dessous présente la convergence de l'algorithme et les positions prises par le meilleur individu à chaque itération. On vérifie que l'algorithme converge progressivement vers la bonne solution. Les premiers minima trouvés sont relativement éloignés de la solution, mais se rapproche rapidement de celle-ci.

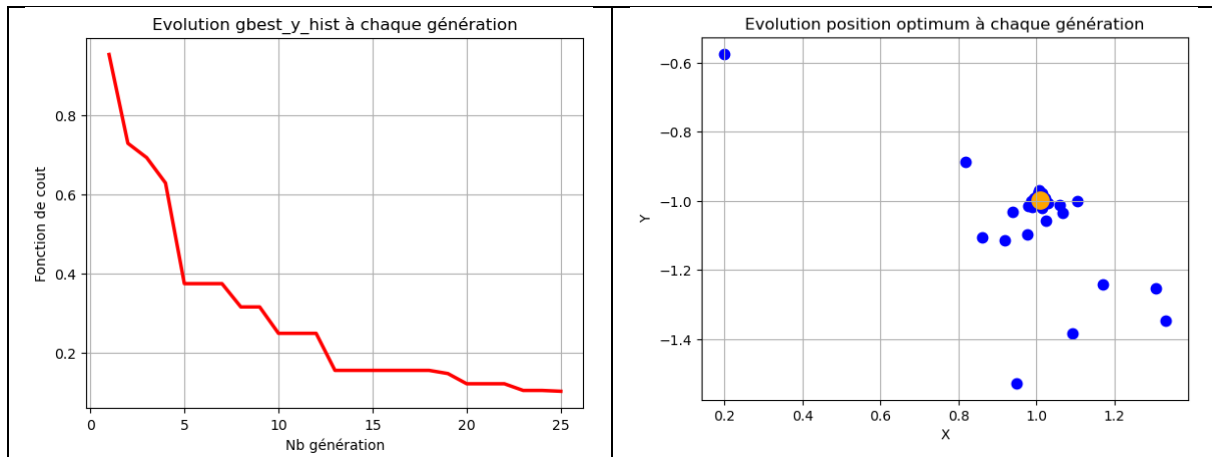


Figure 5 – Résultat algorithme PSO : convergence (à gauche) et positions prises par le meilleur individu à chaque itération (en orange, la solution) (à droite).

3. Optimisation bayésienne (Scikit-optimize.gp_minimize)

Le programme exemple est : BO_Fonction_ext_Run.py. Il est configuré avec :

- Un nombre d'itérations max de 80
- 20 itérations initiales, sélectionnées par un générateur de type LHS
- Une fonction d'acquisition de type LCB avec un paramètre kappa = 3
- La méthode L-BFGS pour identifier le minimum de la fonction d'acquisition (4 redémarrage de la recherche)

La contrainte non-linéaire est passée sous la forme d'une pénalité (= 4) ajoutée à la fonction de coût :

```
#ajout de la containte : on pénalise la fonction objectif si la contrainte
#n'est pas respectée. Pour cela, on fixe arbitrairement une valeur max
#(plus grande que le minimum recherché) quand la contrainte n'est pas respectée.
distance = np.sqrt((x-X0)**2+(y-Y0)**2)
if distance > Rmax: #violation de la contrainte
    Fonction_cout = penalite #attention à ne pas mettre une valeur trop grande, qui dégrade le
    #modèle GPR extrait.
```

Au bout de 51 s (le temps de calcul est plus long que les deux autres approches car on ne profite pas de la vectorisation, mais le nombre d'itérations de l'outil de simulation est nettement plus faibles), l'algorithme identifie la solution suivante, qui est très proche du résultat attendu : [0.9997521595025813 ; -0.9768736010002019], avec une fonction de coût minimale de 0.152078029272925. La figure ci-dessous présente la convergence de l'algorithme et les positions prises par le meilleur individu à chaque itération. On vérifie que l'algorithme converge progressivement vers la bonne solution. Au cours de l'exécution du programme, la fonction de coût affiche parfois 4 : il s'agit de la valeur de pénalité, qui a été sélectionné à une valeur suffisamment grande pour ne pas être considérée comme un minimum intéressant.

La figure présente aussi le tracé en 2D de la fonction de coût estimée par le GPR : celle-ci présente un minimum unique centré sur le point (1 ; -1). Les points d'échantillonnage sont aussi ajoutés à la figure. Ceux-ci sont plutôt regroupés autour du minimum, indiquant qu'au fur et à mesure du calcul, l'algorithme a ajouté des points d'échantillonnage autour du minimum (phase d'exploitation). Il en a aussi ajouté sur les bords, sans doute pendant la phase initiale et pour les besoins de l'exploration de l'espace des solutions).

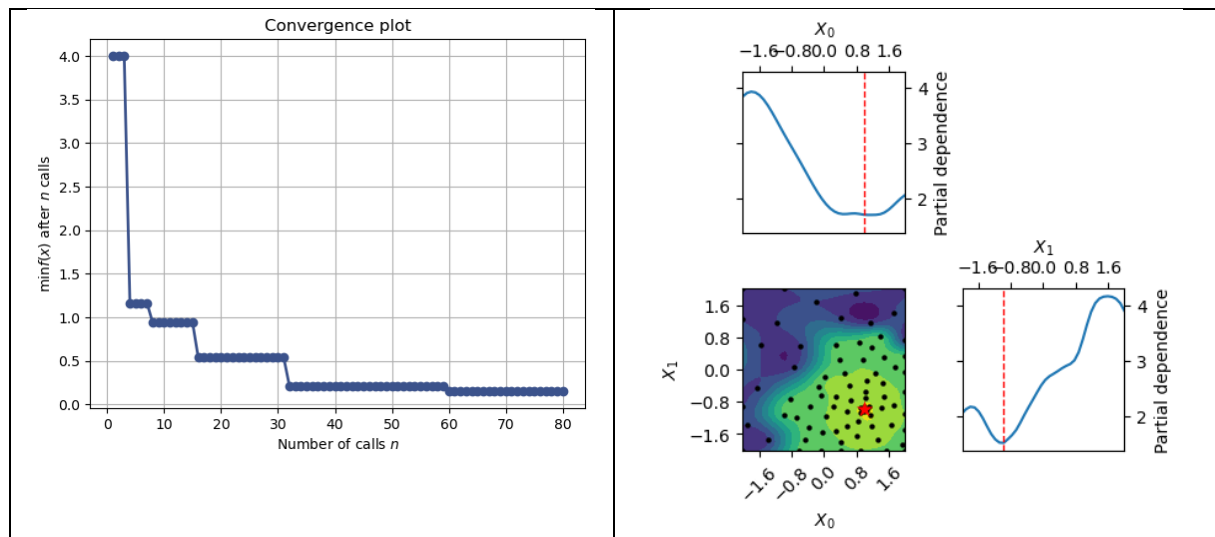


Figure 6 – Résultat algorithme BO : convergence (à gauche) et affichage de la fonction estimée en 2D (à droite).

INSTITUT NATIONAL DES SCIENCES
APPLIQUEES DE TOULOUSE

5^{ème} Année SEE

**TECHNIQUES POUR
L'OPTIMISATION DE LA COMMANDE
ELECTRONIQUE**

**MACHINE LEARNING POUR LA
CONCEPTION ELECTRONIQUE**

**GUIDE DE PRISE EN MAIN DE
L'OUTIL PCB POWER-GROUND PAIR
RECTANGULAR PLANE MODEL**

Alexandre Boyer
alexandre.boyer@insa-toulouse.fr
www.alexandre-boyer.fr

Septembre 2025

L'outil "PCB Power-Ground Pair Rectangular Plane Model" est dédié au calcul de la réponse haute fréquence (sous la forme d'une matrice de paramètres Z) d'une paire de plans d'alimentation (Vdd)-masse (Vss) d'un circuit imprimé multicouche, notamment pour mettre en évidence ces multiples modes de résonance. L'outil est limité (actuellement) à une seule paire de plans rectangulaires, de même taille et sans ouvertures. L'outil est inclus dans le logiciel gratuit IC-EMC (www.ic-emc.org).

Le calcul de cette réponse est d'une grande importance pour garantir la stabilité ou l'intégrité du réseau de distribution d'alimentation (problématique appelée *power integrity (PI)* en anglais) à l'échelle d'un circuit imprimé, condition indispensable pour garantir le bon fonctionnement des circuits électroniques. Cette problématique est particulièrement présente lors de la conception des cartes électroniques pour les applications digitales haute vitesse, produisant des appels de courant forts et rapides (plusieurs dizaines d'ampères en quelques dizaines/centaines de ps). La stabilité de l'alimentation (autrement dit, limiter la fluctuation entre les plans d'alimentation Vdd et Vss à l'échelle de la carte électronique) dépend non seulement des valeurs et du placement des condensateurs de découplage, des impédances des circuits intégrés (régulateur de tension, circuit digital à alimenter), mais aussi de la réponse de la paire de plans Vdd-Vss. Durant le TP de Machine learning pour la conception électronique, la problématique de sélection et de positionnement optimal des condensateurs de découplage sur une carte électronique pour atteindre un objectif acceptable de PI sera utilisée comme cas d'étude.

Ce document vise à décrire le principe général et le fonctionnement de l'outil "PCB Power-Ground Pair Rectangular Plane Model", que nous appellerons « PG Plane Model » dans la suite de ce document. La majeure partie du document décrit la version GUI (*Graphical User Interface*) de l'outil. Une version console existe, permettant un interfacement avec les fonctions d'optimisation utilisées lors des séances de TP de Machine learning pour la conception électronique. Avant de décrire le fonctionnement et le paramétrage de cet outil, quelques éléments théoriques sur la problématique Power Integrity et la structure typique du réseau de distribution d'un réseau d'alimentation seront présentés, ainsi que le modèle de calcul utilisé pour prédire la réponse de la paire de plans Vdd-Vss.

I. Brève introduction à la problématique de Power Integrity dans les réseaux de distribution d'alimentation des cartes électroniques

Une condition fondamentale pour assurer le bon fonctionnement des circuits intégrés (qu'ils soient numériques, analogiques ou radiofréquences) est de garantir des références de tension d'alimentation (Vdd) et de masse (Vss) stables dans le temps. Le fonctionnement des circuits (c'est-à-dire leur consommation de courant statique et dynamique) ainsi que les perturbations électromagnétiques extérieures peuvent conduire à des fluctuations lentes ou rapides des tensions Vdd et Vss appliquées sur les circuits intégrés. La notion de Power Integrity est liée à cette exigence de stabilité des tensions d'alimentation et de masse au niveau des circuits intégrés (IC pour *Integrated Circuits*) et des circuits imprimés (PCB pour *Printed Circuit Board*).

La stabilité des tensions d'alimentation est fortement liée à la structure et au placement-routage du réseau de distribution d'alimentation (PDN pour *Power Distribution Network*). La Figure 1 présente la structure et les composants typiques du PDN d'un PCB multicouche. La source d'énergie peut être une batterie, la tension secteur ou toute autre source. Celle-ci est généralement éloignée des circuits à alimenter et nécessite la présence de convertisseur(s) de puissance et de régulateur(s) de tension (VRM pour *Voltage Regulator Module*) afin d'ajuster la tension à appliquer sur les circuits et garantir sa stabilité quelle que soit la consommation des circuits. Cependant, comme le précise la Figure 2, les régulateurs de tension n'ont pas de bandes passantes suffisantes pour stabiliser les tensions d'alimentation face aux appels de courant des circuits rapides, notamment les circuits digitaux. A chaque front d'horloge, selon la taille du circuit, sa technologie et son activité interne, des pics de courant de plusieurs centaines de mA à dizaines d'ampères durant entre quelques centaines de picosecondes à plusieurs nanosecondes peuvent être générés et conduire à des fluctuations très rapides des tensions d'alimentation. Dans le domaine fréquentiel, ces appels de courant produisent des spectres large bande pouvant aller jusqu'à plusieurs centaines de MHz voire plusieurs GHz.

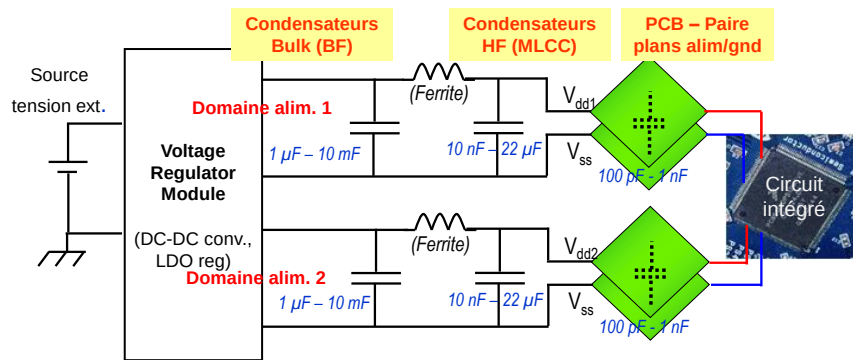


Figure 1 - Structure typique du réseau de distribution d'alimentation d'une carte électronique multicouche

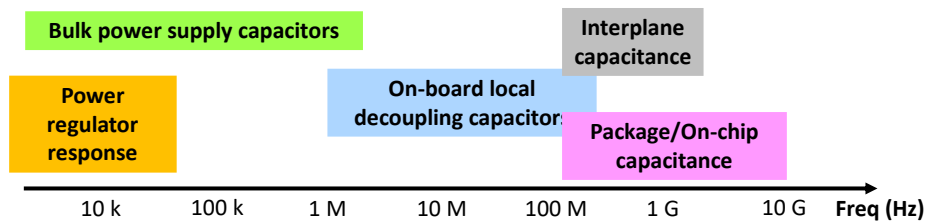


Figure 2 – Bande de fréquence d'influence typique des différents éléments du PDN

Des condensateurs sont ajoutés sur le PCB en sortie des régulateurs (appelés *Bulk capacitor*) et au plus près des broches des circuits (Condensateur de découplage HF ou *Local decoupling capacitor*) afin de servir de réservoir de charges. Plus un condensateur est placé proche d'un circuit, plus vite il pourra répondre à l'appel en courant de ce dernier et limiter les fluctuations des tensions V_{dd} et V_{ss} . La capacité d'un condensateur à répondre rapidement à un appel de courant peut être facilement analysée à travers sa courbe d'impédance complexe dans le domaine fréquentiel ($Z(f)$). Pour un courant et une fréquence donnée, plus l'impédance d'un condensateur est faible, plus la différence de potentiel à ses bornes sera faible. Ainsi, un condensateur monté entre V_{dd} et V_{ss} sera un condensateur de découplage efficace sur une bande de fréquence tant que son impédance sera faible. La Figure 3 compare les profils d'impédance de deux condensateurs électrolytiques de 100 μF (technologies aluminium et tantale) et de condensateurs céramiques multicouches (MLCC pour *MultiLayer Ceramic Capacitor*) de 100 nF (X7R 50 V) assemblés dans différents boîtiers SMD. Le premier présente une impédance faible entre 10 kHz et 10 MHz. Il est typiquement utilisé comme condensateur Bulk placé en sortie d'un VRM. Son impédance reste cependant trop importante au-dessus de quelques MHz pour découpler correctement un circuit intégré digital. Sans compter son boîtier volumineux qui ne permet pas de le monter au plus près du circuit. En raison de leur grande compacité et leurs propriétés, les condensateurs MLCC constituent la technologie usuelle pour découpler localement et en HF les circuits intégrés.

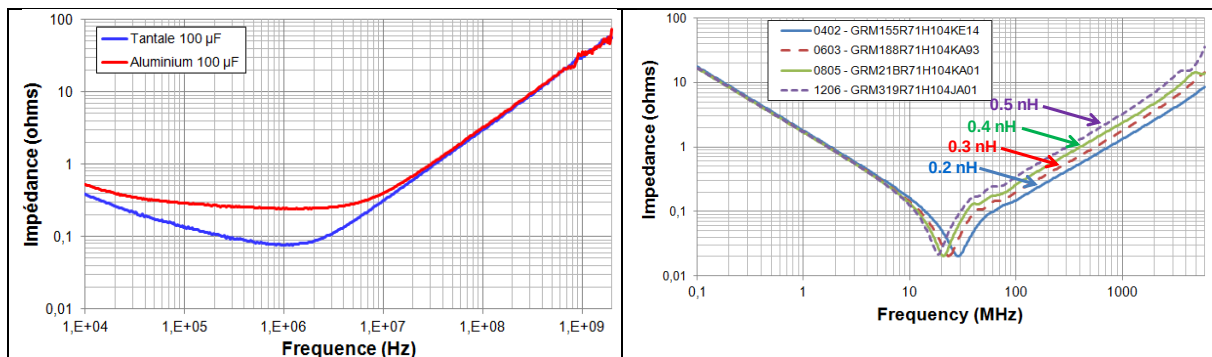


Figure 3 – Profil d'impédance complexe typique : condensateurs électrolytiques aluminium et tantale de 100 μF (à gauche) et MLCC (à droite)

Néanmoins, comme le montre la Figure 3, les condensateurs sont affectés d'une inductance parasite (ESL pour *Equivalent Serial Inductance*) qui accroît l'impédance au-delà d'une fréquence de résonance ($f_c = \frac{1}{2\pi\sqrt{LC}}$). Celle-ci dépend de la taille du boîtier du condensateur et de sa connexion au PDN. Ainsi, même en envisageant les boîtiers les plus miniatures (01002), les MLCC montés sur PCB ont une efficacité de découplage limitée à quelques centaines de MHz. Pour découpler au-delà, des condensateurs peuvent être directement reportés à l'intérieur des boîtiers des circuits intégrés. Des cellules capacitatives peuvent être aussi ajoutées à l'intérieur même de la puce (*on-chip capacitor*), mais au prix d'une surface plus grande de silicium.

L'efficacité des condensateurs de découplage est aussi fortement influencée par leur placement sur le PDN et le routage des interconnexions du PDN. Pour un PCB typique, celles-ci ne peuvent pas être considérées comme de simples équipotentielles au-delà de quelques dizaines de MHz, et doivent être considérées comme des lignes de transmission à partir de quelques centaines de MHz. Afin de minimiser leur résistance et leur inductance parasite, une structure de type plan est utilisée dans les PCB multicouches. En combinant deux couches internes adjacentes, elles forment une paire de plans Vdd et Vss qui, selon les dimensions et l'espacement, fournit une interconnexion faiblement inductive et présentant une capacité allant de quelques centaines de pF à quelques nF (*Interplane capacitance*).

Les différents éléments du PDN vont influencer l'impédance complexe $Z_{PDN}(f)$ qui sera vue entre les broches Vdd et Vss du circuit intégré. Pour le concepteur du PCB, la maîtrise de Z_{PDN} est l'unique levier permettant de garantir un PI acceptable. Afin d'étudier le PI à l'échelle d'un PCB, une analyse petit signal est réalisée. Celle-ci suppose que l'ensemble des composants connectés au PDN ont un comportement linéaire. En outre, l'analyse suppose un fonctionnement en régime établi (l'analyse lors des démarrages nécessite d'analyser la réponse transitoire du PDN, ce qui dépasse le but de ce TP). L'analyse du PI est réalisée à partir du modèle électrique équivalent petit signal présenté sur la Figure 4. L'ensemble des condensateurs de découplage, le VRM, les plans d'alimentation et de masse, les circuits intégrés sont modélisés par une impédance équivalente Z_{PDN} (ici, le PDN est modélisé par une seule impédance équivalente. Il est tout à fait possible de modéliser le PDN par un réseau d'impédances équivalentes associées aux différents éléments du PDN. Nous choisissons un modèle simple pour illustrer le principe). L'activité du circuit intégré à découpler génère un courant transitoire $I_{IC}(t)$. L'impédance Z_{PDN} n'étant pas nulle, l'activité du circuit conduit à une variation de la tension Vdd-Vss, notée $\Delta Vdd(t)$. Dans le domaine fréquentiel, la relation entre le courant produit par l'activité par le circuit et la fluctuation de la tension Vdd-Vss est donnée par :

$$\Delta Vdd(f) = Z_{PDN}(f) \cdot I_{IC}(f)$$

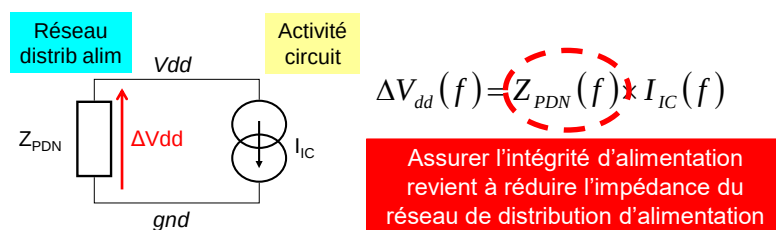


Figure 4 – Modèle électrique équivalent pour l'analyse PI sur le PDN d'un PCB

Ainsi, pour le concepteur du PCB, améliorer le PI (c'est-à-dire limiter $\Delta Vdd(f)$) revient à minimiser l'impédance du PDN sur la plage de fréquence où le circuit intégré produit un courant non négligeable. Comment déterminer alors une limite acceptable pour $Z_{PDN}(f)$? L'approche conventionnelle consiste à définir une impédance cible (Z_{Target}) en fonction de la fréquence de telle sorte que la variation de tension entre Vdd et Vss reste dans des proportions acceptables pour une consommation de courant nominal. Un critère habituel est de limiter la fluctuation de tension à 5% (voire 2 % ou 10 % selon que le critère est strict ou relâché) de la tension d'alimentation nominale. Ainsi, pour un circuit intégré alimentée sous 1 V, ΔVdd ne doit pas dépasser 50 mV. Le courant moyen I_{avg} , dans une condition de fonctionnement nominale, voire pire cas, est généralement considéré.

$$Z_{Target} = \frac{\Delta Vdd_{max}}{I_{avg}}$$

La maîtrise du PI passe ainsi par un objectif simple, illustré sur la Figure 5 :

$$Z_{PDN}(f) < Z_{Target}(f) \quad \forall f \in [f_{min}; f_{max}]$$

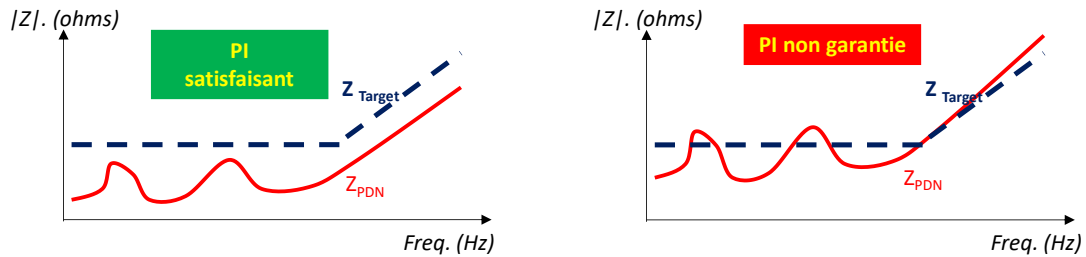


Figure 5 – Concept d'impédance cible

L'impédance Z_{PDN} n'est pas disponible au moment de la conception d'un PCB. L'évaluation de PI par l'approche précédente requiert donc le calcul de Z_{PDN} . Or, ce calcul n'est pas trivial, puisqu'il nécessite d'avoir les modèles équivalents HF des composants du PDN (VRM, condensateurs de découplage, circuits intégrés) et de simuler la réponse HF du PCB via un simulateur électromagnétique (EM) 3D. Le calcul passe donc par un processus de simulation relativement long, mixant simulation électrique et électromagnétique. Le flot de simulation typique est décrit sur la Figure 6. Il fait apparaître différents types de données pouvant être fournis par les constructeurs et deux types de simulateurs : électromagnétiques et électriques. La conception de la carte électronique étant custom, sa réponse doit être calculée par le concepteur, généralement en passant par un outil de simulation EM 3D. Le calcul précis de la réponse de la paire de plans Vdd-Vss est important. Jusqu'à plusieurs dizaines de MHz, il est généralement possible de considérer cette structure comme une équipotentielle. Cette simplification n'est plus valide à des fréquences plus élevées, elle ne permettra pas de modéliser correctement les phénomènes de propagation et les multiples modes de résonance qui impacteront la stabilité de l'alimentation (voir partie II). Afin de garantir un PI suffisant, le concepteur du PCB doit ajuster le dimensionnement et le routage des plans, sélectionner et positionner judicieusement l'ensemble des condensateurs de découplage HF.

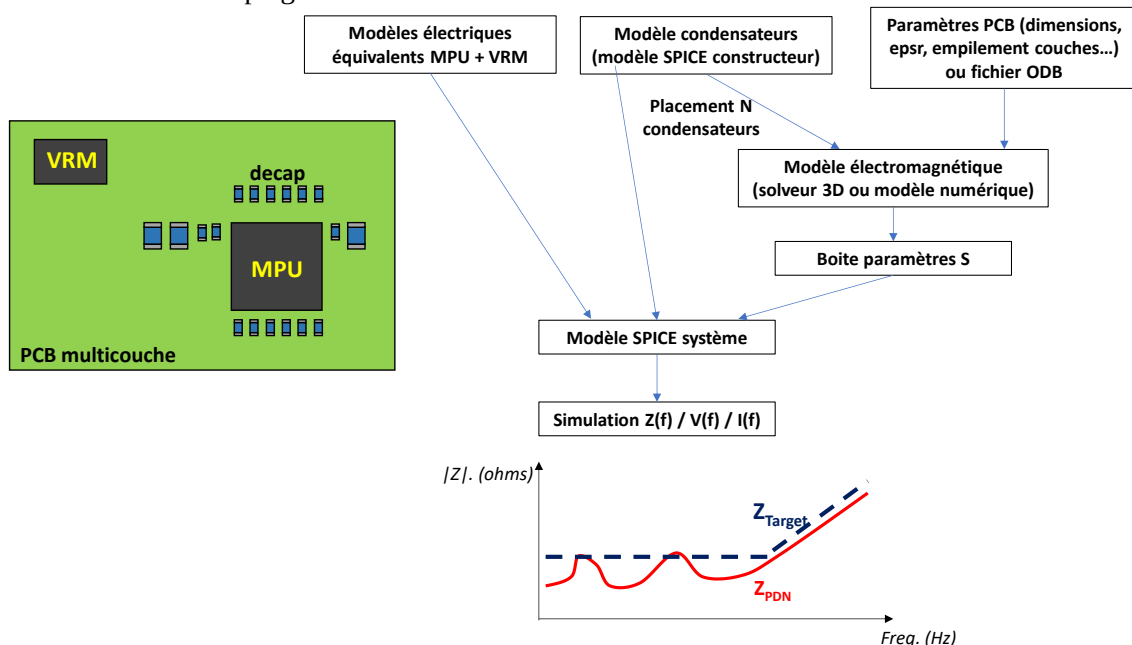


Figure 6 – Flot typique de simulation de l'impédance du réseau de distribution d'alimentation d'une carte électronique

II. Modèle de calcul de la réponse de la paire de plans Vdd-Vss

Afin de minimiser l'impédance Z_{PDN} et raccourcir les connexions des condensateurs de découplage au PDN, les références d'alimentation et de masse sont routées sous la forme de plans. Sur les cartes multicouches complexes, plusieurs couches peuvent être dédiées au routage de ces structures. Celles-ci peuvent occuper une partie d'une couche (voire toute une couche), être de forme variée et présenter des ouvertures (notamment pour le passage des vias).

Sans perdre en généralité, considérons une seule paire de plan d'alimentation-masse. Quel est le comportement électrique d'une telle paire ? Elle doit être vue comme un guide d'onde électromagnétique, ou comme une ligne de transmission à 2 dimensions. Tant que la structure est électriquement courte (dimensions petites devant la longueur d'onde $\lambda = \frac{c_0}{f}$ dans l'air, avec $c_0 = 3.10^8$ m/s et f la fréquence), cette structure est capacitive et la capacité entre deux plans rectangulaires est donnée par :

$$C_{plane} = \frac{\epsilon_0 \epsilon_r W L}{d}$$

Où ϵ_0 est la permittivité de l'air ($8.85.10^{-12}$ F/m), ϵ_r la permittivité relative du substrat diélectrique formant le PCB (typ. 4.2-4.6 pour un substrat epoxy – FR4), W et L sont les dimensions latérales des plans (supposées identiques), d est la distance interplan ou l'épaisseur du substrat. Un courant circulant entre ces 2 plans, une inductance est aussi présente, donnée par :

$$L_{plane} = \mu_0 d$$

Où μ_0 est la perméabilité de l'air ($4\pi.10^{-7}$ H/m). Cependant, lorsque la structure devient électriquement grande, ce comportement capacitif et inductif est distribué, conduisant à de nombreux modes de résonance (à chaque fois qu'une dimension est multiple de $\lambda/2$). Une meilleure manière d'appréhender cette structure est de la considérer comme une ligne de transmission à 2 dimensions (l'épaisseur est considérée comme électriquement petite). Lorsque cette structure est excitée, une onde électromagnétique est créée entre les 2 conducteurs. Celle-ci est stationnaire car les bords de cette structure étant ouverts. A l'instar d'une ligne de transmission 1D, cette structure présentera des résonances (marquées par une impédance tendant vers 0) et anti-résonances (marquées par une impédance tendant vers une grande valeur), comme l'illustre la Figure 7. Ce sont justement ces fréquences d'antirésonance qui peuvent être problématiques pour le PI.

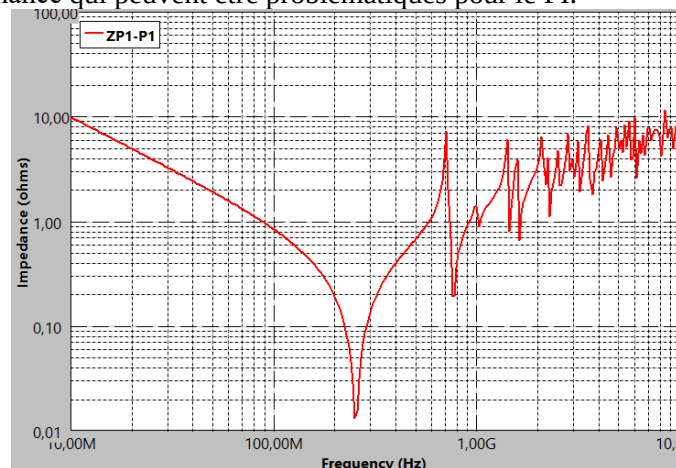


Figure 7 – Impédance complexe d'une paire de deux plans rectangulaires ($W=L=100$ mm, $d = 0.25$ mm, $\epsilon_r = 4.5$) simulée entre 10 MHz et 10 GHz

Ces paires de plans sont donc des structures complexes. Le calcul de leur réponse HF passe par la résolution des équations de Maxwell, ce qui n'est pas trivial. Plusieurs techniques existent pour calculer leurs réponses, de complexité croissante :

- Modèle analytique, limitée à des géométries simples

- Modèle électrique à constantes localisées (PEEC)
- Modèle électrique distribué (Transmission Matrix Method)
- Simulation électromagnétique 3D full-wave (FDTD, FEM par exemple)

Dans ce TP, nous considérons le cas relativement idéal d'une paire de plans Vdd-Vss rectangulaires sans ouverture (Figure 8). Malgré le caractère idéal du cas, il est suffisant pour appréhender la plupart des enjeux liés à la conception du PDN et son impact sur le PI. Pour réduire le temps de calcul, nous utiliserons un modèle analytique, appelé modèle de résonateur à cavité.

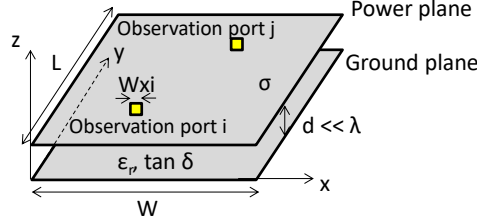


Figure 8 – Cavité rectangulaire 2D formée par une paire de plans d'alimentation et de masse séparés par une couche fine de diélectrique

Sans rentrer dans les détails, l'application des équations de Maxwell dans le cas d'une paire de plans rectangulaires séparés par une distance d électriquement courte ($d \ll \lambda$), conduit à l'équation différentielle suivante, appelée équation de Helmholtz. Celle-ci relie la tension u entre les plans et la densité de courant J_z injecté entre les plans.

$$(\nabla^2 + k^2)u + j\omega\mu d J_z$$

avec ∇ l'opérateur Laplacien, k le nombre d'onde, ω la pulsation et μ la perméabilité magnétique du milieu. Celle-ci peut être résolue en déterminant la fonction de Green de la structure, sous la forme de la matrice d'impédance $[Z_{PG}]$. Chaque terme Z_{PGij} de la matrice relie la tension au point j de la structure, lorsqu'un courant est appliqué au point i . Dans la suite, ces points i et j seront appelés ports, placés aux points (x_i, y_i) et (x_j, y_j) . L'impédance Z_{PGij} est calculée à l'aide de l'expression suivante :

$$Z_{PGij}(\omega) = \sum_{n=0}^{\infty} \sum_{m=0}^{\infty} \frac{N'_{mni} N'_{mni}}{j\omega L_{mn} + j\omega C_{mn} + G_{mn}}$$

avec :

$$\begin{aligned} N_{mni} &= \cos\left(\frac{m\pi x_i}{W}\right) \cos\left(\frac{n\pi y_i}{L}\right) \sin c\left(\frac{m\pi W_{xi}}{2W}\right) \sin c\left(\frac{n\pi W_{yi}}{2L}\right) \\ L_{mn} &= \frac{d}{2\pi F_{Cmn} W L \epsilon_0 \epsilon_r} \quad C_{mn} = \frac{W L \epsilon_0 \epsilon_r}{d} \quad N'_{mni} = C_m C_n N_{mni} \\ G_{mn} &= \frac{W L \epsilon_0 \epsilon_r}{d} 2\pi F_{Cmn} \left(\tan \delta + \frac{1}{d} \frac{1}{\sqrt{\pi F_{Cmn} \mu \sigma}} \right) \\ C_m, C_n &= \begin{cases} 1 & \text{if } m, n = 0 \\ \sqrt{2} & \text{otherwise} \end{cases} \end{aligned}$$

W_{xi} , W_{xj} , W_{yi} , W_{yj} sont les dimensions latérales des ports (supposés rectangulaires), $\tan \delta$ la tangente de pertes du substrat, σ la conductivité des couches conductrices, F_{Cmn} les fréquences de résonance des différents modes de résonance de la cavité. Les modes sont caractérisés par des indices m et n .

L'outil "PCB Power-Ground Pair Plane Model", dédié au calcul de la matrice d'impédance d'une paire de plans rectangulaires chargés par des condensateurs de découplage, exploite ce modèle en évaluant cette double sommation jusqu'à des valeurs maximales d'indices m et n , appelées dans la suite ordre du modèle. Augmenter l'ordre du modèle permet d'accroître la précision du modèle, mais au prix d'un temps de calcul allongé. L'impédance de la paire de plans peut ensuite être combinée avec les impédances des composants connectés à ces plans (VRM, IC, condensateurs de découplage) pour calculer l'impédance du PDN vue depuis chaque port. En l'occurrence, pour le problème de PI, l'impédance vue depuis les broches du circuit intégré à découpler est celle qui nous intéresse.

III. Présentation de la version GUI de l'outil "PG Plane Model"

L'outil est dédié au calcul de la réponse fréquentielle sous la forme d'une matrice de paramètres Z d'une paire de plans rectangulaires formant une cavité résonante avec des condensateurs de découplage placés en N ports sur la carte (N limité à 20). La géométrie du problème est précisée sur la Figure 8. L'outil permet non seulement de tracer l'impédance vue depuis différents ports placés sur la carte, mais aussi d'exporter la réponse de la structure sous la forme d'un fichier de paramètres S (fichier Touchstone) ou sous la forme d'un schéma électrique équivalent SPICE. Ceci afin d'importer ce modèle dans un modèle de simulation plus large. L'outil distingue deux types de ports : les ports observables et non-observables. A l'issue du calcul, l'impédance Z_{PDN} sera uniquement calculée entre les ports observables. Ils correspondent aux ports d'intérêt pour le concepteur, là où l'impédance du PDN doit respecter l'impédance cible, généralement aux bornes des circuits intégrés à découpler. Par contre, les charges peuvent être connectées sur les ports observables et non-observables.

Le flot est résumé sur la Figure 9. La première phase correspond à la définition du problème (géométrie de la paire de plans rectangulaires, des matériaux utilisés, la position et la définition des ports). On définit aussi l'impédance complexe des charges éventuellement connectées sur les ports. La configuration peut être enregistrée dans un fichier de configuration (.pgconf) et réimportée plus tard. Une fois le calcul configuré (bande de fréquence, ordre maximal), l'impédance Z_{PG} de la cavité rectangulaire formée par les plans Vdd-Vss entre tous les ports (observables et non-observables) est calculée. L'effet des charges des ports est ensuite pris en compte pour calculer l'impédance du PDN vue entre chaque port dit observable. Les résultats sont ensuite visibles sous la forme du tracé de Z_{PDN} en fonction de la fréquence. Le modèle calculé pour la cavité rectangulaire peut aussi être exporté sous la forme d'un fichier Touchstone (fichier d'échange de paramètres S) ou d'un sous-circuit SPICE, afin d'être inclus dans un simulateur électrique de type SPICE.

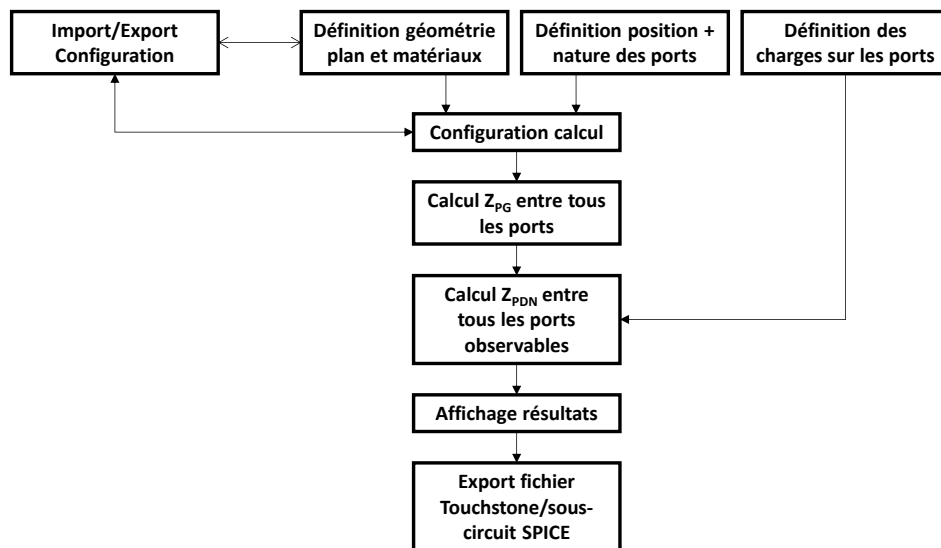


Figure 9 – Flot de calcul de l'impédance Z_{PDN} de l'outil PG Plane Model

Dans la suite, nous considérons que la carte n'est formée que de cette paire Vdd-Vss (les autres couches et interconnexions sont ignorées). On appelle charge l'impédance équivalente d'un composant qui sera connectée sur la carte. Celle-ci peut correspondre à un circuit intégré (le régulateur de tension, le circuit digital à découpler) ou un condensateur de découplage. Dans la pratique, ces charges sont reliées physiquement aux plans Vdd et Vss par des vias et pistes courtes. Ces connexions sont modélisées sous la forme d'un port, constitué de 2 terminaux carrés de dimension w_p , situés sur les

plans Vdd et Vss. Par défaut, aucune charge n'est connectée sur un port. Sur chaque port, jusqu'à trois charges peuvent être connectées.

L'outil PG Plane Model est intégré au logiciel gratuit IC-EMC, qui fonctionne sur Windows. Celui-ci est téléchargeable ici : www.ic-emc.org. Dézippez-le et lancez l'exécutable ic-emc.exe. Une fois lancé, cliquez sur le menu Tools et cliquez sur « PG Plane Model » pour ouvrir l'outil. La fenêtre présentée sur la Figure 10 s'affiche.

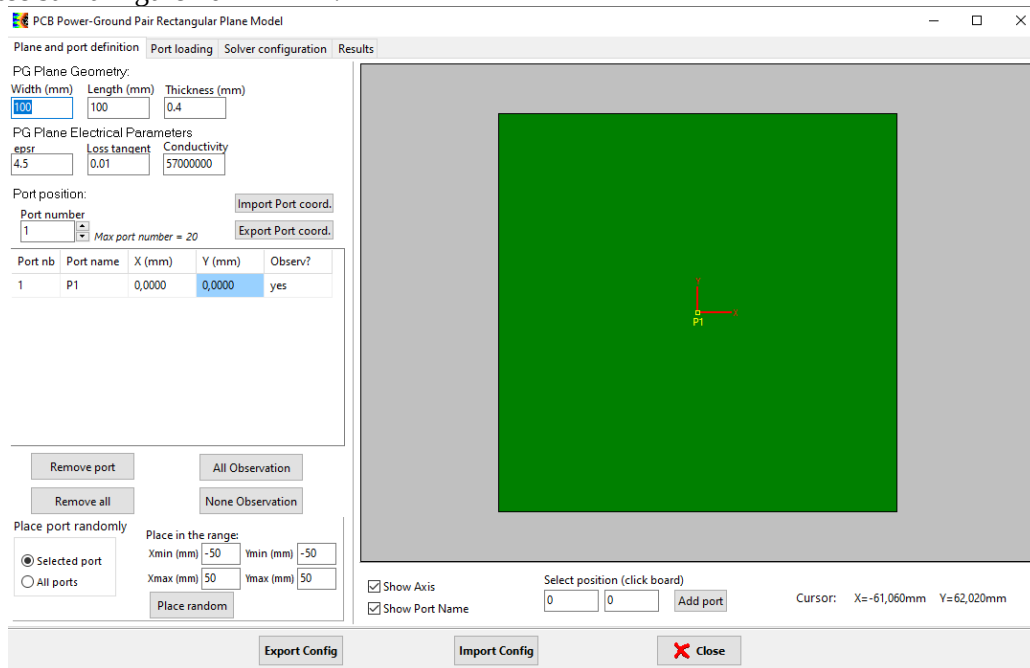


Figure 10 – Outil PG Plane Model à l'ouverture

L'outil se présente sous la forme de 4 onglets visible en haut à gauche, correspondant aux 4 étapes principales du flot de calcul :

- Définition des dimensions géométriques du plan, de ces matériaux et positions des ports
- Définition des charges connectées sur les différents ports
- Configuration de l'outil de calcul
- Affichage et exports des résultats

Les prochaines parties détaillent chacun de ces onglets.

1. Onglet Plane and Port Definition

Le premier onglet « Plane and Port Definition » (Figure 11) est dédié à :

- la définition de la géométrie de la paire de plans Vdd-Vss : dimensions latérales données par les champs Width et Length, séparation donnée par l'épaisseur du substrat donnée par le champ Thickness.
- la définition des matériaux : la permittivité relative du substrat (epsr), ses pertes diélectriques (loss tangent), la conductivité des conducteurs (Conductivity) typiquement du cuivre.
- la définition des ports : le nom du port, la position (x ; y) du port (il doit naturellement se trouver sur le plan), et son observabilité.

Pour faciliter la définition du problème, l'interface graphique de droite montre la position des ports sur la surface de la carte. Les ports observables apparaissent en jaune, les non-observables en violet. **Le nombre de ports est limité à 20 !** Le nombre de ports est contrôlable par le contrôle haut-bas Port-number. Pour ajouter un nouveau port, incrémenter de 1 le nombre de ports. Les propriétés des

ports sont modifiables en cliquant directement dans le tableau Port position, visible en dessous de ce contrôle.

Remarque : quand vous modifiez un champ ou une valeur dans un tableau, validez la saisie en appuyant sur la **touche Entrée**.

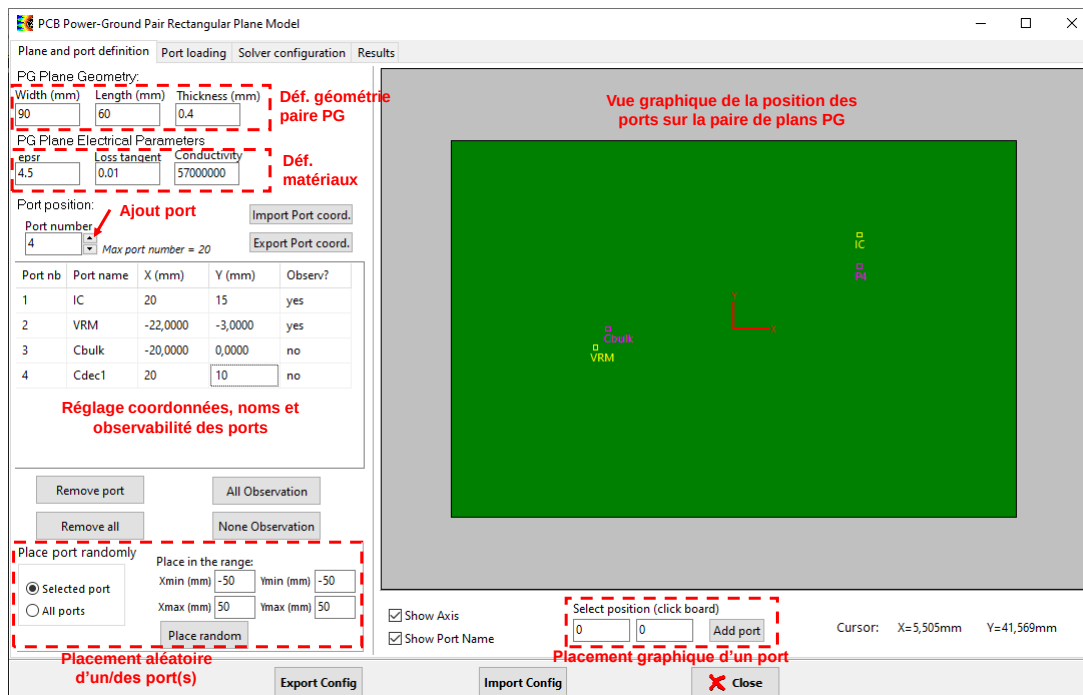


Figure 11 – Fenêtre de définition de la paire de plans et des ports

Le bouton Remove Port permet de supprimer le port sélectionné (sélectionnant en cliquant la ligne associée sur le tableau de définition des ports). Le bouton Remove all supprime l'ensemble des ports. Pour activer ou désactiver l'observabilité de tous les ports, cliquez sur les boutons All Observation ou None Observation. Les coordonnées des ports peuvent être saisies manuellement, ou sélectionnées graphiquement : cliquez sur l'interface graphique. La position associée apparaît sous l'interface graphique, dans les champs Select position. Sélectionnez un port en cliquant sur une ligne du tableau Port Position. Cliquez enfin sur le bouton Add port, les nouvelles coordonnées seront attribuées au port sélectionné. Enfin, il est possible d'attribuer aléatoirement des positions aux ports (Place port randomly) à l'intérieur d'une zone définie par les champs Place in the range.

Il est aussi possible d'exporter et d'importer les coordonnées des ports dans un fichier texte, en cliquant sur les boutons Import Port Coord et Export Port Coord. Le format de ce fichier est simple :

- Une première ligne comportant les noms des ports séparés par des ;
- Une seconde ligne comportant les coordonnées (x ;y) des différents ports, toutes séparées par un ;

2. Sauvegarde et import des configurations

La définition de la géométrie, des ports et des paramètres de calcul peut être fastidieux si elle est répétée à chaque utilisation de l'outil. Pour éviter de les redéfinir à chaque nouveau lancement de l'outil, il est possible de sauvegarder ces paramètres dans un fichier de configuration (extension .pgconf) qui pourra être importer ultérieurement. L'export de la configuration se fait en cliquant sur le bouton Export Config, situé en bas de l'écran. Elle pourra être rappelée en cliquant sur le bouton Import Config.

3. Onglet Port Loading

Le premier onglet « Plane and Port Definition » (Figure 12) est dédié au réglage des charges connectées sur les ports. Sur tous les ports, il est possible de connecter jusqu'à 3 charges. Même si ces

3 charges ne peuvent pas être physiquement positionnées sur le même port, cette astuce permet de simuler plus rapidement 3 composants qui seraient montées à proximité. Trois types de charge peuvent être configurées :

- None : l'état par défaut des charges connectées sur les ports. Ce type indique qu'un circuit ouvert est connecté sur le port.
- RLC : un circuit équivalent RLC est connecté sur le port, qui correspond au modèle électrique HF le plus basique d'un condensateur. Ce type de charges permet de modéliser facilement les condensateurs de découplage.
- S parameter : le comportement fréquentiel de la charge est décrit par des paramètres S importés sous la forme d'un fichier de paramètres S. Ce fichier peut être fourni par le constructeur d'un composant (notamment pour les condensateurs MLCC) ou obtenu par mesure. Cette approche permet de décrire précisément le comportement un composant sous la forme d'un modèle boîte noire.

Durant le TP, nous ne considérerons que des charges de type RLC (ou None) pour garantir des simulations rapides.

Le tableau Load list donne une vue d'ensemble du nombre et de la nature de charges connectées sur chaque port. Le détail est donné dans le tableau Load details. Pour ajouter une charge sur un port, cliquez sur la ligne correspondante du tableau Load list. Le port sélectionné apparaît dans le champ Selected port. Cliquez sur Add Load autant de fois que vous voulez ajouter une charge (elles seront appelées Load1, Load2 et Load3). Sélectionnez la charge que vous voulez configurer puis choisissez l'onglet correspondant au type de charge attendu : RLC, S parameter, No load. Entrez les caractéristiques et cliquez sur le bouton Set Load.

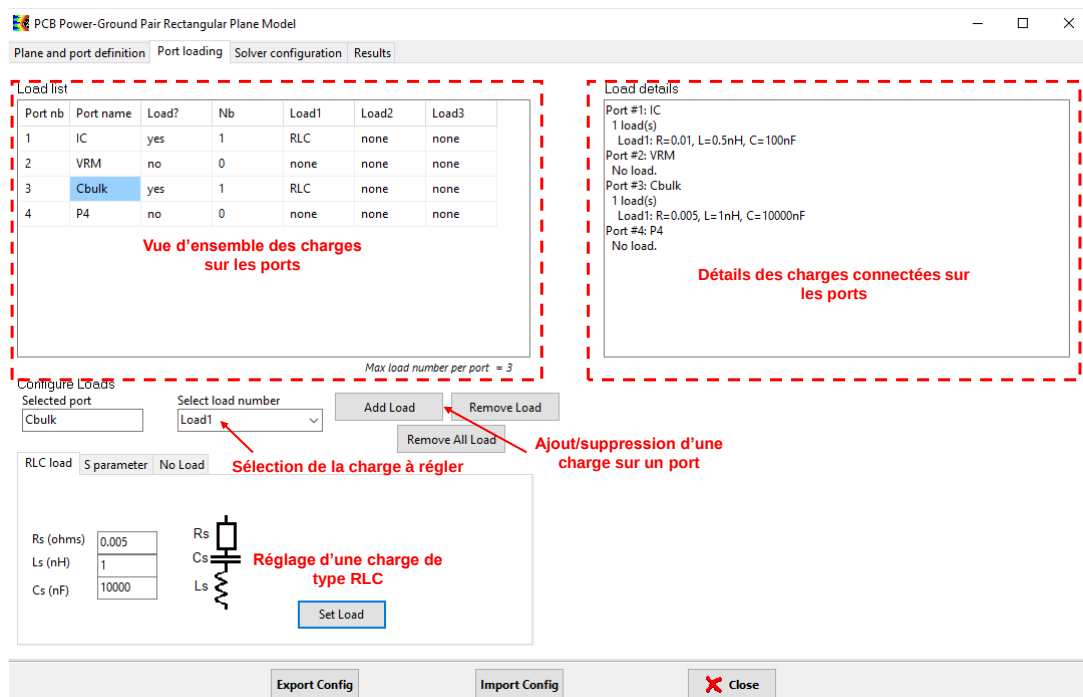


Figure 12 – Fenêtre de définition des charges connectées sur les ports

4. Onglet Solver configuration

Le troisième onglet « Solver configuration » est dédié au paramétrage et au lancement du calcul de l'impédance du PDN (Figure 13). Le réglage doit être fait afin de trouver un compromis entre la précision et le temps de calcul. Les paramètres à régler sont :

- Réglage de l'ordre du modèle (Order m et Order n). Augmenter l'ordre améliore la précision du calcul, mais allonge le temps de calcul. L'ordre est limité à 1000.

- Réglage du balayage en fréquence : type de balayage (lineaire, logarithmique, défini par un fichier Touchstone), les bornes et le nombre de points de fréquence. Augmenter le nombre de points de fréquence augmente la résolution fréquentielle du modèle mais accroît le temps de calcul. Le nombre de points de fréquence est limité à 500. La réponse du PDN étant large bande, il est préférable d'employer un balayage fréquentiel logarithmique.
- Les options : elles permettent de réduire le temps de calcul au prix d'une perte de précision. L'option Simple Sum simplifie l'expression utilisée pour calculer Z_{PG} pour remplacer la double sommation par une simple sommation. Le calcul des paramètres extradiagonaux de la matrice ZPG ne requiert pas nécessairement des valeurs élevées pour l'ordre, contrairement aux paramètres diagonaux. L'option Adaptive Order permet d'ajuster l'ordre selon qu'un terme diagonal ou extra-diagonal soit calculé. Durant ce TP, il est conseillé d'activer ces deux options pendant les phases d'optimisation pour garantir des temps de calcul acceptables.

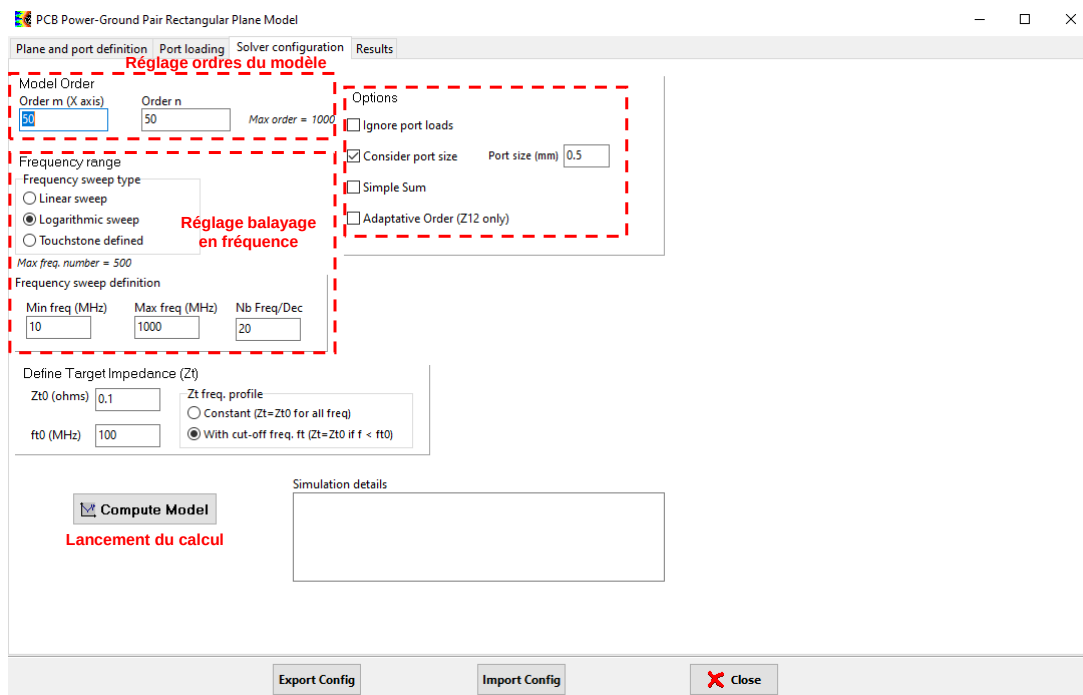


Figure 13 – Fenêtre de configuration du calcul

Une fois le calcul configuré, celui-ci est lancé en cliquant sur le bouton **Compute Model**. A l'issue du calcul, le statut et le temps de calcul sera affiché dans le champ Simulation Details.

La partie Define Target Impedance est utilisée pour définir un profil d'impédance cible, qui sera affiché dans l'onglet Results. Deux types de profil peuvent être configurés :

- Constant : l'impédance cible est constante quelle que soit la fréquence et donnée par le champ Zt0.
- Défini par un profil passe-bas d'ordre 1 : $Z_t(f) = Z_{t0} \frac{f}{f_{t0}}$. L'impédance cible est constante et donnée par le champ Zt0 pour toute fréquence inférieure à la fréquence de coupure donnée par le champ ft0. Au-delà, l'impédance cible augmente en 20 dB/dec. Cette définition permet de considérer le fait que le spectre du courant lié à l'activité des circuits intégrés est borné et, in fine, de relâcher la contrainte sur Z_{PDN} à haute fréquence.

5. Onglet Results

Le dernier onglet « Results » est dédié à l'affichage et l'export des résultats (Figure 14). Seules les impédances visibles entre les ports observables peuvent être affichées.

Pour afficher une nouvelle courbe, sélectionnez les deux ports entre lesquels vous souhaitez tracer l'impédance (champ Port 1 et Port 2), puis cliquez sur le bouton Add Result. La nouvelle courbe

s'affichage sur la zone graphique à droite, ses propriétés (visibilité et couleur) s'affichent dans le tableau Displayed Result. Pour supprimer définitivement une courbe, sélectionnez la ligne correspondante du tableau Displayed Curves et cliquez sur le bouton Remove. En cliquant sur le bouton Memo, vous mémoriser une courbe, ce qui peut aider à comparer les résultats de plusieurs simulations successives.

Remarque : jusqu'à 5 courbes de simulation et 5 courbes de mémorisation peuvent être affichées simultanément.

Le réglage du graphique (bornes, zoom, échelle lin/log) se fait via les boutons et champs situés sous le graphique.

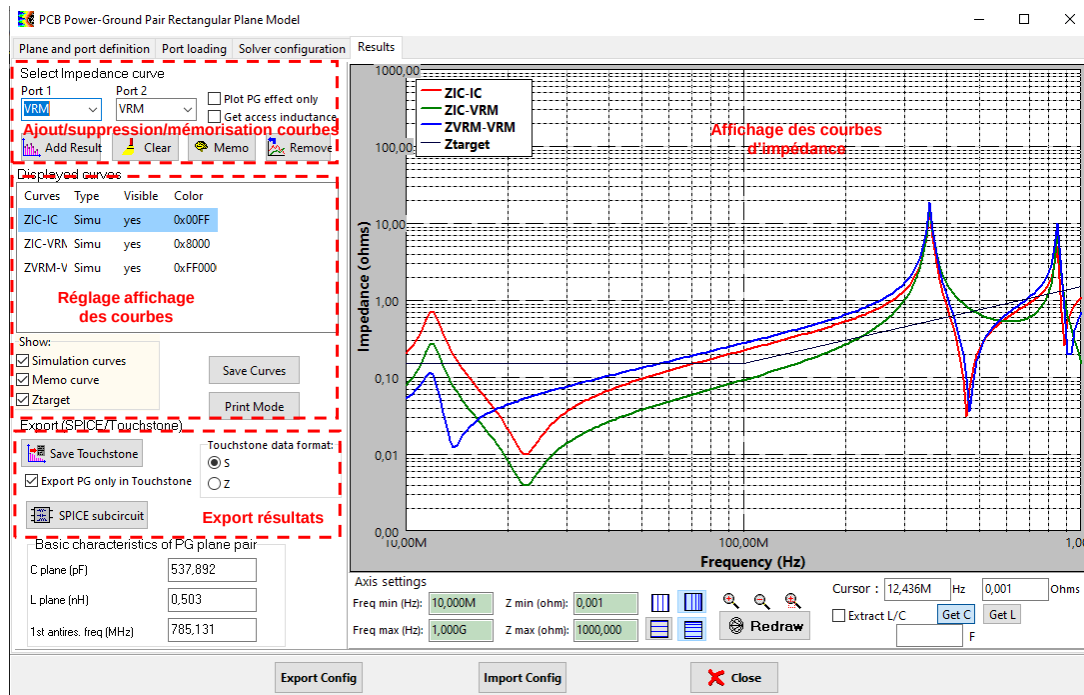


Figure 14 – Fenêtre d'affichage et d'export des résultats

Les impédances calculées peuvent être comparées avec l'impédance cible définie sur l'onglet précédent, à condition que la case Ztarget soit cochée. Les impédances tracées correspondent à l'impédance Z_{PDN} , qui tient compte de l'impédance Z_{PG} de la paire de plans Vdd-Vss et des charges connectées sur l'ensemble des ports. En cochant la case Plot PG effect only, l'effet des charges n'est plus pris en compte et seule l'impédance Z_{PG} est affichée. L'ensemble des courbes affichées peuvent être sauvegardées dans un fichier texte pour un traitement ultérieur en cliquant sur le bouton Save Curves.

Le modèle de la paire de plans Vdd-Vss peut être sauvegardé pour une utilisation dans un simulateur électrique de type SPICE. Deux formats sont possibles :

- Sous la forme d'un fichier de paramètres S au format Touchstone (*.sNp, où N est le nombre de ports). Pour cela, cliquez sur le bouton Save Touchstone, en sélectionnant si vous voulez sauvegarder les données en paramètres S ou Z
- Sous la forme d'un sous-circuit SPICE. Pour cela, cliquez sur le bouton SPICE subcircuit.

IV. Présentation de la version Console de l'outil "PCB Power-Ground Pair Plane Model"

L'outil PG Plane Model du logiciel IC-EMC est adapté à une utilisation manuelle via une interface graphique. Cependant, il n'est pas adapté à une utilisation itérative dans le cadre d'un processus automatique d'optimisation ou d'apprentissage réalisé par un programme externe. En effet, ce dernier doit appeler un grand nombre de fois un outil de simulation calculant l'impédance de la paire de plans Vdd/Vss. Le lancement d'IC-EMC et son interface graphique vont ralentir considérablement l'exécution du processus d'optimisation ou d'apprentissage.

C'est pourquoi une application console (fonctionnant dans une fenêtre de commande Windows) a été développée : PGPlane_Calculator.exe a été développée. Cette application réutilise le cœur de calcul de l'outil PG Plane Model, mais sans l'interface graphique utilisateur. Il peut être appelé par n'importe quel programme externe. Les données d'entrée sont fournies par des fichiers texte, les résultats de calcul sont disponibles aussi dans des fichiers texte. La Figure 15 illustre son utilisation par un programme d'optimisation ou d'apprentissage externe. Ce dernier appelle un grand nombre de fois un outil de simulation externe (simulation engine) dédiée au calcul du problème physique considéré (ici, le calcul de l'impédance de la paire de plans Vdd-Vss avec ses condensateurs de découplage). Chaque appel de l'outil de simulation externe correspond à une nouvelle combinaison de positions de condensateurs de découplage. L'application PGPlane_Calculator.exe est appelée pour calculer la réponse de la paire de plans Vdd-Vss pour une combinaison de placement de condensateurs, ou pour plusieurs combinaisons de placements de condensateurs permettant ainsi un mode appelé **vectorisation**.

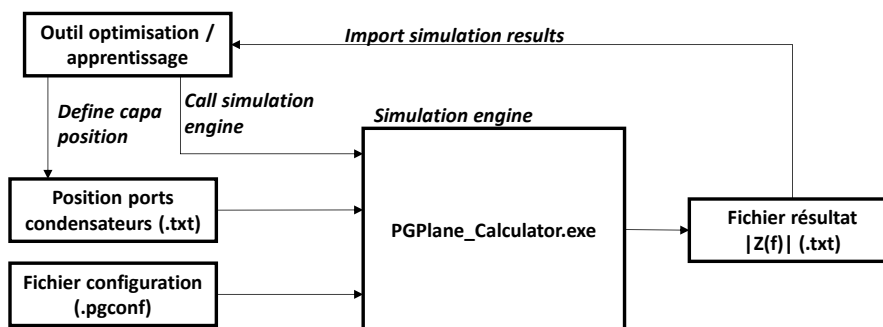


Figure 15 – Utilisation de l'outil PGPlane_Calculator.exe dans un processus d'optimisation ou d'apprentissage

L'application PGPlane_Calculator.exe a besoin de deux fichiers d'entrée pour lancer le calcul :

- Le fichier de configuration .pgconf, donnant la géométrie du problème, la position des ports (observables et non-observables), les charges, les paramètres de calcul (fréquences, ordres, options de calcul). Ce fichier est unique pendant toute la séquence d'optimisation ou d'apprentissage du modèle. Le fichier .pgconf peut être généré par l'outil PG Plane Model du logiciel IC-EMC (voir partie IV.2).
- La position des condensateurs de découplage est passée par un fichier .txt. Celui-ci est généré par l'outil d'optimisation ou d'apprentissage

A l'issue du calcul, l'application PGPlane_Calculator.exe calcule l'impédance vue depuis le premier port observable (pour chaque fréquence de calcul définie dans le fichier .pgconf). Le résultat est écrit dans un fichier de résultat (au format .txt) qui est importé par l'outil d'optimisation/d'apprentissage.

Les parties suivantes détaillent l'appel de l'application PGPlane_Calculator.exe, le format des fichiers d'entrée et de sortie, et donnent un exemple d'appel de l'application par un programme Python.

1. Appel de l'application console PGPlane_Calculator.exe

L'application PGPlane_Calculator.exe peut être appelée depuis la fenêtre de commande Windows ou bien appelé par un programme externe. L'application doit être appelée avec 4 arguments, sous la forme :

PGPlane_Calculator.exe NbCalcul Fichier_config.pgconf Fichier_Ports.txt Fichier_resultats.txt

Si les arguments ne sont pas passés lors de l'appel ou si les arguments sont incorrects, l'application PGPlane_Calculator.exe ne s'appliquera pas correctement.

- Le premier argument (NbCalcul) est un entier qui indique le nombre de combinaisons de positions de condensateurs que l'application va calculer. Si NbCalcul > 1, alors l'outil d'optimisation/d'apprentissage fonctionne en mode vectorisation.
- Le second argument (Fichier_config.pgconf) passe le fichier de configuration à l'application PGPlane_Calculator.exe. Il s'agit du fichier .pgconf, qui peut être généré à partir de l'outil PG Plane Model du logiciel IC-EMC. L'argument donne le nom du fichier avec son chemin d'accès.
- Le troisième argument (Fichier_Ports.txt) donne le nom et le chemin d'accès d'un fichier texte donnant la position (x;y) des ports que l'on souhaite déplacer. Ces ports doivent avoir été définis dans le fichier .pgconf. Dans notre TP, il s'agira de la position des condensateurs de découplage dont on cherche à optimiser la position. C'est donc votre propre programme Python qui devra générer ce fichier. Le format du fichier est le suivant :
 - Une entête indiquant les noms des ports que l'on souhaite déplacer, séparés un ' ; '. Ces noms doivent être ceux qui ont été définis dans le fichier .pgconf.
 - L'entête est suivi de NbCalcul lignes donnant les positions (x ;y) des ports définis dans l'entête. Les coordonnées sont définies en mètres.
 - Exemple :
Port1;Port2
xport1;yport1;xport2;yport2
xport1;yport1;xport2;yport2
xport1;yport1;xport2;yport2
- Le quatrième argument (Fichier_resultats.txt) fournit le nom et le chemin d'accès du fichier de sauvegarde des résultats. Si ce fichier n'existe pas, l'application PGPlane_Calculator.exe le créera. Ce fichier de sortie donne le module de l'impédance vue depuis le premier port observable défini dans le fichier .pgconf, calculée pour les NbF fréquences définies aussi dans le fichier .pgconf. A l'issue du calcul, les résultats seront écrits dans le fichier texte sous le format d'une matrice avec NbF lignes et NbCalcul colonnes.

Remarque : il est déconseillé d'avoir des espaces ou des caractères spéciaux dans les chemins d'accès ou les noms des fichiers appelés par l'application PGPlane_Calculator.exe.

2. Exemple d'appel de l'application console PGPlane_Calculator.exe par un programme Python

Pour appeler l'application PGPlane_Calculator.exe depuis un programme Python et contrôler son exécution (notamment pour attendre la fin de l'exécution de l'application), il est conseillé d'utiliser le module subprocess de Python.

Voici un exemple (à adapter) pour pouvoir lancer l'application PGPlane_Calculator.exe depuis un programme Python. Au préalable, celle-ci doit avoir été téléchargée et installée sur votre machine à un endroit connu.

```
import subprocess

#lien vers l'exécutable et ses arguments
#Attention : pas d'espace dans les noms de fichier !!!
exe_path = r'C:\ PGPlane_Calculator.exe'
```



```
config_file = r'C:\ CasTest_4decap.pgconf'  
data_file = r'C:\port_file.txt'  
Out_File = r'C:\out.txt'  
  
#lancement du subprocess  
args = [exe_path,str(Nb_calcul),config_file,data_file,Out_File]  
subprocess.run(args)
```